

SKALA

AI 서비스를 위한 SW 기초 Full-stack Engineering

스마트 데이터 이해 및 활용

2026.6

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

스마트 데이터의 이해 및 활용

1. 데이터베이스 개요
2. 관계형 모델 핵심 개념
3. ERD
4. 개념적→논리적→물리적 모델 설계
5. 스키마 전략 & RDBMS 비교
6. SQL 기초 — DDL
7. SQL 기초 — DML
8. 종합실습 — 1

스마트 데이터터의 이해 및 활용

9. Day 1 핵심 복습 & DBMS 생태계

10. SQL 중급 — 집계 & 그룹화

11. SQL 고급 — JOIN 알고리즘

12. JOIN 종류 상세

13. 서브쿼리 & 집합 연산자

14. CTE · View · Materialized View

15. Windows Function — 분석 함수

16. 종합 실습 — 2

스마트 데이터터의 이해 및 활용



17.인덱스 설계 및 최적화

18.실행계획 분석

19.SQL 튜닝 & 느린 쿼리 식별

20.MVCC & 트랜잭션 격리 수준

21.Lock & Deadlock 관리

22.고급 DB 설계

23.종합 실습 - 3

스마트 데이터의 이해 및 활용

24. Stored Procedure & 함수

25. Trigger & 이벤트 처리

26. Cloud DB 개요

27. 서버리스 & 분산 Cloud DB

28. 데이터 웨어하우스 & 분석 DB

29. 현재의 트렌드

30. 보안 및 권한 관리

31. 백업·복구 & 고가용성

32. 모니터링 및 운영

33. 종합실습 - 4

스마트 데이터의 이해 및 활용

1. 데이터베이스 개요
2. 관계형 모델 핵심 개념
3. ERD
4. 개념적→논리적→물리적 모델 설계
5. 스키마 전략 & RDBMS 비교
6. SQL 기초 — DDL
7. SQL 기초 — DML
8. 종합실습 — 1

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

1. 데이터 베이스 개요

- 필요성 · 발전 과정 · ACID · WAL · Isolation Level

필요성과 발전 과정

파일 시스템 시대

- 개별 애플리케이션마다 독자적 파일 사용
- 중복 데이터, 일관성 보장 어려움
- 동시 접근 불가 → 단일 사용자 중심

계층형 DB (Hierarchical DB)

- 트리 구조 기반 (예: **IBM IMS**)
- 빠른 탐색 가능하나 유연성 부족
- **1:N** 관계에 최적화, **N:M** 관계 표현 어려움

관계형 DBMS (RDBMS)

- **1970년대 Codd**의 관계형 모델 제안 이후 주류
- **SQL** 도입으로 표준화 및 범용성 확보
- 다양한 최적화 기법 (인덱스, 트랜잭션, 정규화)

분산/클라우드 DB

- 빅데이터, 글로벌 서비스 확산으로 확장성/가용성 중시
- **NewSQL, NoSQL**과의 혼합 환경 출현
- 예: **Google Spanner, AWS Aurora, CockroachDB**

DB 발전은 단순한 저장 방식이 아니라 데이터 무결성과 일관성 vs 확장성 간의 균형 싸움

DB 발전 과정

시대/구분	대표 기술	핵심 특징	한계
파일시스템 (1960s)	운영체제 파일	개별 앱마다 독자 파일	- 중복 - 불일치 - 동시접근 불가
계층형 DB (1970s)	IBM IMS	- 트리구조 - 빠른 탐색	1:N 최적 - N:M 표현 어려움
관계형 DBMS (1970s~)	Oracle, MySQL, PostgreSQL	- 관계형 모델 - SQL 표준화	- JOIN 비용 - 수평 확장 어려움
NoSQL (2000s~)	MongoDB, Redis, Cassandra	- 유연한 스키마 - 수평 확장	강한 일관성 보장 어려움
분산/Cloud DB (2010s~)	Spanner, Auroa, CockroachDB	ACID + 수평 확장 (New SQL)	높은 운영 복잡도

【 ACID 속성 】

속성	설명
Atomicity (원자성)	트랜잭션의 모든 작업이 완전히 수행되거나 전혀 수행되지 않아야 함 (송금 = 출금 + 입금 → 둘 다 성공 or 둘 다 취소)
Consistency (일관성)	트랜잭션 수행 전후에 DB는 일관된 상태를 유지
Isolation (격리성)	트랜잭션이 동시에 실행될 때 서로 간섭하지 않도록 보장
Durability (지속성)	트랜잭션이 성공적으로 완료되면 그 결과는 영구적으로 반영

데이터베이스 트랜잭션이 지켜야 할 4가지 기본 원칙

DBMS별 ACID 지원 차이

DBMS ACID 차이점

- **ACID**는 모든 **DBMS**에서 "교과서적으로는 지원" — 엔진/설정에 따라 보장 수준이 다름
- **MySQL InnoDB**: 완전한 **ACID** 지원 (Redo/Undo Log, MVCC)
- **MyISAM** 엔진: Rollback 불가, FK 없음
→ 사실상 **ACID** 미지원 (현재는 InnoDB가 기본)
- **PostgreSQL**: MVCC 기반, WAL, SSI (Serializable Snapshot Isolation)
- 모든 격리 수준 지원, 가장 표준에 가까운 구현
- **Oracle**: Read Committed 기본, 매우 안정적인 상용 **ACID** 구현
- **Cloud DB (Aurora/RDS)**: 분산 환경에서 **ACID** + 수평 확장
- **Aurora**: 스토리지 레벨 6방향 복제 → 장애 시 빠른 복구
- 핵심 포인트: DB 선택 시 **ACID** 보장 수준과 성능 트레이드오프 반드시 확인

DBMS별 ACID 지원 차이

DBMS	Atomicity	Consistency	Isolation	Durability	특징
Oracle	강력 지원	Strict Consistency	다양한 Isolation Level, 기본적으로 Read Consistent	Redo/Undo 로 완벽 보장	엔터프라이즈급 표준
PostgreSQL	MVCC 기반 보장	MVCC로 강력한 Consistency	Serializable 포함 다양한 Level	WAL 기반	오픈소스 중 가장 "Oracle-like"
MySQL (InnoDB)	완전 지원	무결성 제약 준수	Repeatable Read 기본, Phantom Read는 발생 가능	Double-write Buffer	엔진에 따라 차이, InnoDB만 ACID
MySQL (MyISAM)	(Rollback 불가)	제약 약함	Locking 기반	비지속성	사실상 ACID 미지원
SQL Server	완전 지원	제약조건, FK 엄격 적용	Read Uncommitted ~Serializable까지	Transaction Log	Isolation Level 세부 튜닝 가능
Cloud DB (예: AWS Aurora, Google Spanner)	지원	Consistency 조정 가능 (Strong / Eventual)	다지역 트랜잭션 지원	분산 WAL/Consensus 기반	분산 환경에서 "일관성 vs 가용성" 선택 가능 (CAP Trade-off)

ACID는 모든 DBMS에서 "교과서적으로는 지원", 엔진/설정/클라우드 옵션에 따라 실제 보장 수준이 다름

[참고] WAL (Write-Ahead Logging) — 지속성 구현 원리

-- PostgreSQL WAL 동작 방식

BEGIN;

UPDATE account SET balance = balance - 10000 WHERE id = 'A';

-- 1단계: WAL 버퍼에 변경 내용 기록 (로그 우선 원칙)

-- 2단계: Shared Buffer (메모리)에 페이지 변경 반영

-- 3단계: 아직 디스크 데이터 파일에는 미기록 (dirty page)

UPDATE account SET balance = balance + 10000 WHERE id = 'B';

COMMIT;

-- COMMIT 로그를 WAL에 기록 후 fsync(디스크 동기화)

-- 이후 Checkpoint 시점에 데이터 파일에 반영

-- 장애 복구 시:

-- pg_wal/ 디렉토리의 로그를 순서대로 재실행 (Redo)

-- COMMIT되지 않은 트랜잭션은 Undo 처리

-- WAL 관련 설정 확인 (postgresql.conf)

-- wal_level = replica

(복제 레벨)

-- synchronous_commit = on

(on: 안전 / off: 성능↑, 최대 200ms 손실 가능)

-- checkpoint_timeout = 5min

(체크포인트 주기)

WAL 핵심

"로그가 먼저(WAL)"

데이터 파일보다 로그를 먼저 써야
장애 시 복구 가능

[참고] WAL (Write-Ahead Logging) / Consensus

【 WAL (Write-Ahead Logging) 】

데이터베이스에서 변경 사항을 실제 데이터 파일에 쓰기 전에 로그(저널)에 먼저 기록하는 방식

- "로그가 먼저(WAL)"라는 이름 그대로

구분	설명
동작 과정	• 클라이언트가 UPDATE 실행 • DB는 즉시 데이터 파일을 고치지 않고, 먼저 WAL 로그에 기록 • 로그가 디스크에 안전하게 기록되면 "COMMIT 성공" 응답 • 실제 데이터 파일 반영은 나중에 배치/버퍼 관리자가 수행
장점	• 장애 발생 시, 로그를 기반으로 Redo(다시 적용) / Undo(취소) 가능 • 쓰기 성능 개선 (순차 기록 → 랜덤 I/O 최소화) • PostgreSQL, SQL Server, Oracle, MySQL(InnoDB) 등 대부분 DBMS가 사용

[참고] WAL (Write-Ahead Logging) / Consensus

【 Consensus 기반 (합의 알고리즘) 】

분산 DB(Cloud DB, 분산 트랜잭션 DB)에서는 여러 노드가 동일한 데이터를 가진 상태여야 함

- 네트워크 지연/장애가 발생해도 모든 노드가 같은 결과(합의)를 가지도록 하는 알고리즘 사용

구분	설명
대표적인 Consensus 알고리즘	• Paxos: 이론적으로 가장 유명한 분산 합의 알고리즘 • Raft: 구현이 단순해 실제 제품(AWS Aurora, Etcd, CockroachDB 등)에서 널리 활용 • Google Spanner: Paxos + TrueTime API (전역 시계 동기화) 활용
동작 과정	• 클라이언트가 트랜잭션 요청 • Leader 노드가 로그에 기록 (→ 여기서도 WAL!) • 다른 Follower 노드들과 "투표"를 통해 과반수가 같은 로그에 동의해야 Commit 확정 • 모든 노드에 같은 순서의 로그(트랜잭션)가 쌓이므로 Strong Consistency 보장

[참고] WAL (Write-Ahead Logging) / Consensus

【 Consensus 기반 (합의 알고리즘) 】

분산 DB(Cloud DB, 분산 트랜잭션 DB)에서는 여러 노드가 동일한 데이터를 가진 상태여야 함

- 네트워크 지연/장애가 발생해도 모든 노드가 같은 결과(합의)를 가지도록 하는 알고리즘 사용

구분	설명
장점	• 분산 환경에서도 데이터 일관성(Consistency) 유지 • 장애 발생 시 Leader 재선출 가능 → 고가용성
단점	• 분산 환경에서도 데이터 일관성(Consistency) 유지 • 네트워크 지연이 늘어나면 성능 저하 (특히 글로벌 DB에서 latency 증가)

[참고] WAL (Write-Ahead Logging) / Consensus

방식	WAL (Write-Ahead Logging)	Consensus
적용 범위	단일 DBMS, 단일 노드 중심	분산 DB, 멀티 노드 환경
목적	장애 복구, 트랜잭션 보장	분산 노드 간 동일한 상태 유지
동작 원리	로그 먼저 기록 후 데이터 반영 (트랜잭션 실행 → WAL 로그 기록 → 디스크 저장 → COMMIT 응답 → 나중에 데이터 파일 반영)	여러 노드가 투표를 통해 합의 (Leader → Followers에 로그 전파 → 다수 동의 후 Commit 확정)
대표 사례	PostgreSQL, Oracle, SQL Server	Google Spanner, AWS Aurora, CockroachDB

- **WAL = “한 DB 안에서 장애 대비 안전장치”**
- **Consensus = “여러 DB가 흩어져 있어도 모두 같은 결과를 보장하는 안전장치”**

[참고] Isolation Level

트랜잭션
동시성에서
발생하는
대표 현상

- **Dirty Read**
: 커밋 전 값 읽음
- **Non-Repeatable Read**
: 같은 행을 두 번 읽었더니 값이 달라짐
(다른 트랜잭션이 그 행을 **UPDATE/DELETE**)
- **Phantom Read**
: 같은 조건으로 두 번 조회했더니 행의 개수가 달라짐
(다른 트랜잭션이 새 행 **INSERT** 또는 범위 내 행을 추가/삭제)

[참고] Isolation Level_동시성 이상 현상 허용 여부

격리 수준	Dirty Read	Non-Repeatable Read	Phantom Read	DBMS 기본값
Read Uncommitted	허용	허용	허용	거의 미사용
Read Committed	방지	허용	허용	PostgreSQL, Oracle
Repeatable Read	방지	방지	허용(표준) → InnoDB는 Gap Lock으로 방지	MySQL, InnoDB
Serializable	방지	방지	방지	성능 저하 주의

[참고] Isolation Level_ANSI 격리 수준(이상 현상 허용 여부)

Isolation Level	Dirty	Non-Repeatable	Phantom	비고
READ UNCOMMITTED	허용	허용	허용	거의 사용 X
READ COMMITTED (RC)	차단	허용	허용	기본값인 DBMS 多
REPEATABLE READ (RR)	차단	차단	원칙적 허용	구현마다 Phantom 처리 차이
SERIALIZABLE	차단	차단	차단	가장 엄격(순차 실행과 동등)

**표준 정의상 RR은 Phantom을 허용하지만,
DBMS 구현에 따라 범위 잠금/스냅샷으로 실질적으로 막히는 경우 존재**

[참고] Isolation Level_DBMS별 기본값과 구현 차이

DBMS	기본	특징
Oracle	READ COMMITTED	- MVCC로 문장 단위 read consistency 제공 → 같은 SELECT 안에서는 일관된 스냅샷 - SERIALIZABLE은 스냅샷 기반으로 충돌 시 ORA-08177 발생 ("serialize 불가" 재시도 필요)
PostgreSQL	READ COMMITTED	- MVCC. REPEATABLE READ는 스냅샷 격리(SI) 성격 → 같은 트랜잭션 내 반복 SELECT는 새 행(팬텀)을 보지 않음 - 완전 직렬성을 위해 SERIALIZABLE(SSI) 사용 : 필요 시 트랜잭션을 롤백해 직렬성 보장
MySQL (InnoDB)	REPEATABLE READ	- MVCC의 consistent read로 일반 SELECT는 같은 스냅샷을 보므로 팬텀을 사실상 보지 않음 - 단, 잠금 읽기(SELECT ... FOR UPDATE / FOR SHARE)나 UPDATE/DELETE 범위 조건에선 next-key lock(레코드+갭 잠금)으로 팬텀을 차단 (동시에 경합/대기 증가 가능)
SQL Server	READ COMMITTED (락 기반)	- 옵션: READ_COMMITTED_SNAPSHOT(행 버전) 켜면 RC에서도 과거 버전 읽기 가능 - SNAPSHOT(세션/트랜잭션 레벨) 별도 제공 - SERIALIZABLE은 범위 잠금으로 팬텀 차단
Cloud		- Aurora MySQL/PostgreSQL: 각 호환 엔진 규칙을 따름(상동) - Spanner: TrueTime/분산 합의로 외부적 직렬성(Strong/Serializable) 보장

[참고] Phantom Read

테이블 **orders(amount)**에서 **amount >= 100** 인 건수

【 Session 1 】

```
BEGIN;
SELECT COUNT(*) FROM orders WHERE amount >= 100; -- 결과: 5
-- 같은 조건으로 조금 있다가 다시 셈
```

【 Session 2 (동시 처리 트랜잭션) 】

```
BEGIN;
INSERT INTO orders(id, amount) VALUES (999, 150);
COMMIT;
```

【 다시 Session 1 】

```
SELECT COUNT(*) FROM orders WHERE amount >= 100; -- 결과가?
```

- **READ COMMITTED**
: 결과 5 → 6 으로 증가 → 팬텀 발생
- **REPEATABLE READ**
(일반적 스냅샷 구현, 예: InnoDB/PG)
: 5 → 5 유지
(같은 트랜잭션 내 같은 스냅샷을 보기 때문)
- 단, Session 1이 잠금 읽기(SELECT ... FOR UPDATE)나 같은 범위를 UPDATE/DELETE로 잡으면 범위 잠금으로 Session 2의 INSERT가 대기/차단되어 팬텀 자체를 사전 차단
- **SERIALIZABLE**
: 팬텀 원천 차단
(범위/프레딕트 잠금 또는 SSI)
: Session 2의 INSERT는 Session 1이 끝날 때까지 대기하거나 충돌로 롤백됨.

팬텀은 “값이 변해서”가 아니라
“조건에 맞는 새 행이 생겨서
결과 개수가 바뀌는 현상”

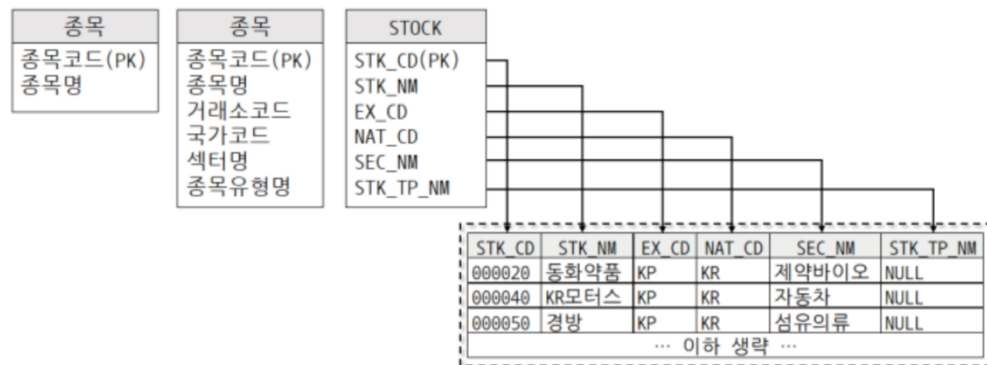
AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

2. 관계형 모델 핵심 개념

- 테이블, 키 (PK, FK, UK), 관계 (1:1, 1:N, N:M)
- 정규화 (1NF~3NF)

관계형 데이터 모델링이란?

현실 세계의 데이터를 관계형 포맷으로 변환하는 과정



관계형 데이터 모델링이란?

- 현실 세계의 데이터를 관계형 포맷으로 변환하는 과정 – 관계형 데이터 모델링
- 컴퓨터 프로그램을 통해서 일을 하기 위해서는 1) 현실 세계의 데이터를 컴퓨터가 이해할 수 있도록 변환 해야 하고 2) 해당 데이터에 대해서 여러 명이 동시에 통합적으로 작업할 수 있도록 구성이 필요
- 관계 (Relation)란, 통상 열과 행을 지니는 형태로 표현하며 대표적 예는 “표”

관계형 데이터 모델링이란?

테이블 (Table/Relation)

- 행(Row/Tuple)과 열(Column/Attribute)로 구성된 2차원 구조
- 각 행(Row): 하나의 데이터 인스턴스
- 각 열(Column): 특정 속성, 동일 데이터 타입

기본키 (PK, Primary Key)

- 각 행을 고유하게 식별하는 컬럼(또는 컬럼 조합)
- **NULL** 불가, 중복 불가 / 자동으로 인덱스 생성
- 예: 학생 테이블의 "학번"

외래키 (FK, Foreign Key)

- 다른 테이블의 **PK**를 참조하는 컬럼
- 테이블 간 관계를 표현, 참조 무결성 보장
- 예: 성적 테이블의 "학번"
→ 학생 테이블의 "학번(PK)" 참조

고유키 (UK, Unique Key)

- 중복 불가, **NULL**은 허용
- 예: 이메일 주소, 주민등록번호, 직원코드

관계형 모델 핵심 개념

【 테이블 (Table) 】

- 데이터를 행(Row, Tuple)과 열(Column, Attribute)로 구성한 2차원 구조
 - 각 행(Row)
 - : 하나의 레코드(데이터 인스턴스)를 의미
 - 각 열(Column)
 - : 특정 속성(Attribute)을 의미
 - 동일한 데이터 타입을 가진 값들이 한 컬럼에 저장됨

학생 테이블

학번(PK)	이름	학과	학년
2024001	홍길동	컴퓨터공학	1
2024002	이순신	경영학	2
2024003	강감찬	컴퓨터공학	3

관계형 모델 핵심 개념

【 키 (Key) 】

- 관계형 데이터베이스에서 데이터 무결성과 식별성을 보장하기 위한 제약 조건

키 구성

기본 키 (PK, Primary Key)

- 각 행을 고유하게 식별할 수 있는 컬럼 (혹은 컬럼 조합)
- **Null 불가, 중복 불가**
- 예: 학생 테이블의 "학번"

외래 키 (FK, Foreign Key)

- 다른 테이블의 기본 키를 참조하는 컬럼
- 테이블 간 관계(**relationship**)를 표현
- 예: 성적 테이블의 "학번" → 학생 테이블의 "학번(PK)" 참조

고유 키 (UK, Unique Key)

- 중복은 허용하지 않지만, **PK와 달리 Null은 허용 가능**
- 예: 이메일 주소, 주민등록번호

[참고] 외래키 제약 조건 옵션 비교

옵션	동작	사용 시나리오
ON DELETE CASCADE	부모 삭제 시 자식도 함께 삭제	주문 삭제 시 주문 항목도 자동 삭제
ON DELETE SET NULL	부모 삭제 시 자식의 FK를 NULL로	직원 퇴사 시 담당 고객의 담당자를 NULL로 처리
ON DELETE RESTRICT	자식이 있으면 부모 삭제 차단	카테고리 삭제 전 상품 먼저 삭제 필요
ON DELETE NO ACTION	RESTRICT와 유사 제약 체크 시점 차이	Oracle, SQL Server 기본값
ON UPDATE CASCADE	부모 PK 변경 시 자식 FK도 자동 변경	자연키(학번) 변경 전파 (대체키로 권장)

관계형 모델 핵심 개념

【 관계 (Relationship) 】

- 테이블 간의 연관성을 표현

관계

1:1 관계 (One-to-One)

- 한 레코드가 다른 테이블의 하나의 레코드와만 연결
- 예: [학생] ↔ [학생증]

1:N 관계 (One-to-Many)

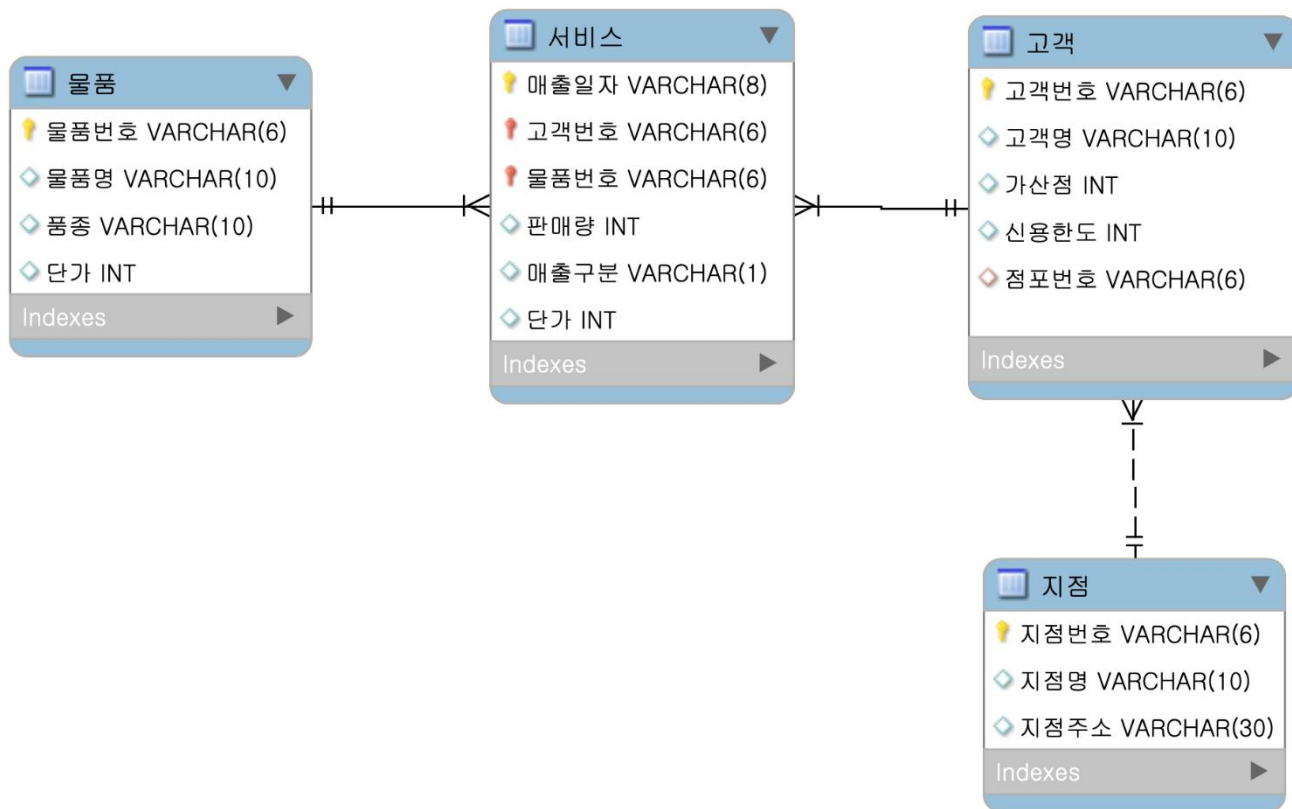
- 한 레코드가 다른 테이블의 여러 레코드와 연결
- 가장 흔한 관계
- 예: [학생] 1명 ↔ [성적] 여러 개

N:M 관계 (Many-to-Many)

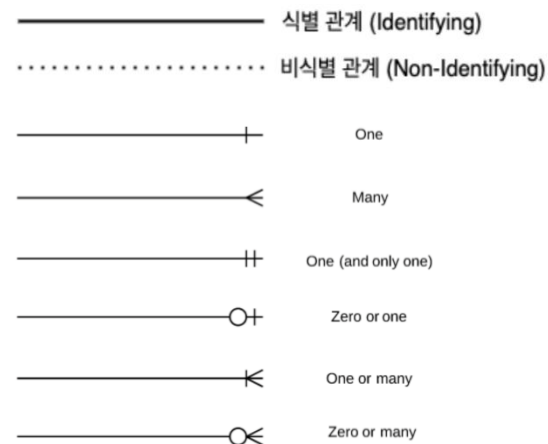
- 여러 레코드가 다른 테이블의 여러 레코드와 연결
- 일반적으로 중간 테이블(교차 엔티티)로 구현
- 예: [학생] ↔ [강좌] 관계 → "수강" 테이블 필요

관계형 데이터 모델링이란?

일반적 조직 (공기업/사기업) 업무에서, 제품 및 서비스를 만들어서 시장/현장에 출시한다고 가정하여 논리 **ERD**를 설계한다면



범례 (IE 표기법)



관계형 데이터 모델링이란?

사번 (PK)	직위	직위코드	전공코드 (PK)	역할상세	연락처	근무지	자격증	회사코드	회사명
20-243679	중사	SG2	SNIPER	저격	010-XXX-XXXX	미국	스쿠버	UDT	3707
23-023456	병장	SG0	MEDIC	의료	010-XXX-XXXX	한국	응급구조사	특수전여단	1179
22-138907	하사	SG1	DEMOLITION	폭파	010-XXX-XXXX	캐나다	화공기사	특전사령부	707

※ 식별자 (Primary Key)가 부여된 상태를 1정규형이라고도 합니다.

관계형 데이터 모델링에서 PK (Primary Key) 부여 예시

- 회사 업무 앱을 만들어 역할별 특수임무 TF팀을 구성하고 운영한다고 가정
- 회사이니 사번으로 특정인을 구분할 수 있을 것이고, 역할은 전공코드로 구분 가능
- ⇒ 직원별 역할은 "사번과 전공코드"로 구분이 가능

관계형 데이터 모델링이란?

사번 (PK)	직위	직위코드	전공코드 (PK)	연락처	근무지	자격증	회사코드	회사명
20-243679	중사	SG2	SNIPER	010-XXX-XXXX	미국	스쿠버	UDT	3707
23-023456	병장	SG0	MEDIC	010-XXX-XXXX	한국	응급구조사	특수전여단	1179
22-138907	하사	SG1	DEMOLITION	010-XXX-XXXX	캐나다	화공기사	특전사령부	707
21-135987	중사	SG2	SNIPER	010-XXX-XXXX	한국	고공강하	특수전여단	1179

전공코드 (PK)	역할상 세
SNIPER	저격
MEDIC	의료
DEMOLITION	폭파

※ 식별자 (**Primary Key**)에 대한 부분 종속된 내용을 분리한 상태를 제 2 정규형이라고 합니다.
분리된 테이블에서 기존 테이블로 연결해서 다시 찾아갈 수 있는 기준 열 (Column)을
외래키 (**Foreign Key**)라고 합니다.

관계형 데이터 모델링에서 FK (Foreign Key) 부여 예시

- 직원을 한 명 더 추가하면서 생각해 보니 사번, 이름, 연락처는 바로 수첩에서 찾을 수 있었지만,
- 전공코드로 구분되는 임무상세 정보가 중복되고 추후 재정의될 수도 있으니 특수임무 TF팀 전체 목록 관리에서 분리 후 관리

관계형 데이터 모델링이란?

인사정보

※ 식별자 (Primary Key)아 아닌 데 부분 종속된 내용을 분리한 상태를 제 3 정규형이라고 합니다.

사번 (PK)	직위코드	전공코드 (PK)	연락처	근무지	자격증	회사코드
20-243679	SG2	SNIPER	010-XXX-XXXX	미국	스쿠버	UDT
23-023456	SG0	MEDIC	010-XXX-XXXX	한국	응급구조사	특수전여단
22-138907	SG1	DEMOLITION	010-XXX-XXXX	캐나다	화공기사	특전사령부
21-135987	SG2	SNIPER	010-XXX-XXXX	한국	고공강하	특수전여단

전공정보

전공코드 (PK)	역할상 세
SNIPER	저격
MEDIC	의료
DEMOLITION	폭파

직위정보

직위코드 (PK)	역할상 세
SG2	중사
SG0	병장
SG1	하사
SG2	중사

회사정보

회사코드 (PK)	역할상 세
UDT	3707
특수전여단	1179
특전사령부	707
특수전여단	1179

관계형 데이터 데이터 모델링 완성

- 식별자는 아니지만 회사와 직위도 각각 따로 관리가 가능.
단, 분리를 안 할 경우 회사와 동일한 직위가 동일한 직원을 추가할 경우에 회사와 직위가 중복 되므로 주의 필요

관계형 데이터 모델링과 SQL 관계

전공정보

전공코드 (PK)	역할상세
SNIPER	저격
MEDIC	의료
DEMOLITION	폭파

관계형 데이터 모델링과, SQL간의 관계

- 완료된 데이터 모델링을 토대로 **DBMS (Database Management System)**에 테이블을 생성하고 데이터를 입력/조회/삭제/수정 가능
⇒ **SQL (Structured Query Language)** 언어 사용
- 데이터 모델링 없이 바로 테이블을 생성할 수는 있으나,
데이터 양이 많아지거나 복잡해지면 데이터 중복 및 오류 등
품질 문제가 발생

```
CREATE TABLE 전공정보 (
    전공코드 Varchar(10) Primary Key,
    역할상세 Varchar(40)
);
```

```
INSERT INTO 전공정보 (전공코드, 역할상세)
VALUES ('SNIPER', '저격');
```

No SQL 데이터 모델링 이해

SQL vs NoSQL?

• SQL

- 정형 데이터(관계형 데이터모델) 설계 후, 생성된 테이블을 사용하기 위한 표준 언어
- 요구사항을 모델링하여 테이블로 정의하여 사용 시
→ 관계형 데이터 모델링으로 처리

• NoSQL

- 비정형 데이터(문서, 키-값, 그래프, 인메포리, 로그검색 등)에 접근할 때 사용하는 언어를 의미
- 사전 명시된 요구사항이 아니거나 뚜렷하지 않거나 관련이 없는 데이터를 빠르게 처리하는데 유리

SQL이 적합한 부분

- ✓ 데이터 일관성과 온라인 처리
- ✓ 엄격한 ACID (원자성, 일관성, 고립성, 지속성) 인 경우

NoSQL이 적합한 부분

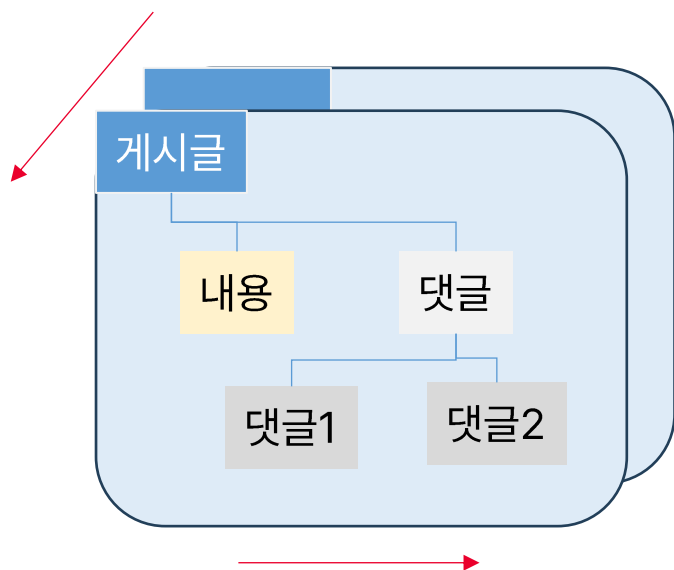
- ✓ 가용성과 성능 중심 처리
- ✓ 일부 중복, 불일치를 허용하는 완화된 ACID

트랜잭션을 보장하는 **ACID** 수준에 따라
SQL/NoSQL 선택 적용 가능

구분	RDBMS	NoSQL
장/단점	• 데이터 무결성, 정확성 보장 • 정규화된 테이블과 소규모 트랜잭션이 있음 • 확장을 위해서는 일부 제약이 있음	• 데이터의 무결성과 정확성을 보장하지 않음 • 웹 환경의 다양한 정보 검색 · 저장에 유리
특징	• 다양하고 복잡한 쿼리 지원 • Update, Delete, Join 연산 가능 • 트랜잭션의 ACID (원자성, 일관성, 고립성, 지속성) 보장 • 고정 스키마가 있음 • 테이블을 디자인한 후 쿼리를 작성	• 단순한 쿼리 지원에 유리 • 직접 수정 · 삭제 보다는 입력으로 대체 가능 • 강한 일관성은 고려하지 않음 • 데이터 서버의 추가 및 삭제, 데이터 분산에 유연 • 어떤 쿼리 결과가 필요한지 먼저 정의한 후 그 결과를 얻을 수 있도록 테이블 디자인

No SQL 데이터 모델링 이해

SQL에서는 데이터의 관계에 집중해서 데이터 분석을 진행하는 반면에, NoSQL은 표현되는 데이터와 정렬되는 특징에 집중해서 분석을 진행



데이터 분석 방법

- ✓ 게시판에서 게시글은 작성 시간순으로 정렬되는 특징
- ✓ 게시글과 내용간, 1:1로 대응
- ✓ 게시글과 댓글간, 1:N으로 대응
- ✓ 댓글은 게시글 내에서 시간순으로 정렬됨
- ✓ 정렬 순서 특징 집중 →

관계형 데이터 모델 검증

관계형 데이터 모델 검증은 설계된 모델이 실제 데이터 저장 및 관리 요구사항을 충족하는지 확인하는 중요한 단계

Time	Crit glares	Databated	Databas validate
ts-database entry...	1	Nowall.tak1	Tex: lif
ts-validation entry...	0	Nowall.tak2	Tex: lif
ts-validation entry...	110	Nowall.tak4	Tex: lif
ts-vatabase.entry...	111	Nowall.tak4	Tex: lif
ts-validation.entry...	111	Nowall.tak3	Tex: lif
ts-validation.entry...	181	Nowall.tak4	Tex: lif
ts-vatabase.entry...	101	Nowall.tak4	Tex: lif
ts-validation entry...	101	Nowall.tak4	Tex: lif
ts-validation entry...	105	Nowall.tak2	Tex: lif
ts-validbase.entry...	101	Nowall.tak4	Tex: lif
ts-validation.entry...	101	Nowall.tak3	Tex: lif
ts-vatabase.entry...	101	Nowall.tak3	Tex: lif
ts-validation entry...	111	Nowall.tak4	Tex: lif
ts-validaton.entry...	101	Nowall.tak4	Tex: lif
ts-vatabase.entry...	101	Nowall.tak4	Tex: lif
ts-validation entry...	100	Nowall.tak3	Tex: lif
ts-vatabase.entry...	107	Nowall.tak3	Tex: lif
ts-validation.entry...	100	Nowall.tak2	Tex: lif
ts-validation entry...	100	Nowall.tak4	Tex: lif
ts-validation entry...	5	Nowall.tak3	Tex: lif
ts-validationed.entry...	107	Nowall.tak3	Tex: lif
ts-validation entry...	117	Nowall.tak4	Tex: lif

검증항목	설명
데이터 무결성	• 정확하고 일관된 데이터 저장 및 관리
정규화	• 데이터 중복 최소화 및 데이터 무결성 향상
성능	• 데이터 액세스 및 처리 속도
보안	• 데이터 보호 및 무단 접근 방지

데이터 무결성, 정규화, 성능, 보안 등 다양한 측면을 검토하여
모델의 완성도를 높입니다.

정규화 과정 소개

1

데이터 중복 제거

- 정규화는 데이터 중복을 최소화하여 데이터 무결성을 높이고 저장 공간을 절약하는 과정
- 데이터 일관성을 유지하고 데이터 관리 효율을 높이는 데 도움

2

관계형 데이터 모델 개선

- 데이터베이스 설계 단계에서 관계형 데이터 모델의 구조를 개선하고 데이터를 효율적으로 관리
- 데이터 중복을 줄이고 데이터 일관성을 높여 데이터베이스의 성능과 안정성을 향상

3

정규화 단계

- 5단계의 정규화 단계를 거침 (1NF, 2NF, 3NF, BCNF, 5NF)
- 각 단계별로 데이터 중복을 제거하고 데이터 무결성을 강화

4

데이터베이스 설계 핵심

- 효율적인 데이터베이스 설계를 위한 핵심적인 과정
- 데이터베이스 성능 향상과 데이터 무결성 유지를 위해 필수적으로 수행

관계형 모델 핵심 개념

관계

【 정규화 (Normalization) 】

- 데이터 중복을 최소화하고, 데이터 무결성을 보장하기 위한 설계 규칙

제1정규형 (1NF: First Normal Form)

- 각 컬럼은 원자값(**Atomic Value**)을 가져야 함 (반복되는 그룹 불가)
- 예: "수강과목 = DB, 네트워크" → 별도 테이블로 분리해야 함

제2정규형 (2NF: Second Normal Form)

- 1NF 만족 + 부분 함수 종속 제거
- 복합키의 일부에만 종속된 속성을 분리
- 예: [성적(학번, 과목코드, 성적)] 테이블에서 "학생이름"은 학번에만 종속 → 학생 테이블로 분리

제3정규형 (3NF: Third Normal Form)

- 2NF 만족 + 이행적 종속 제거
- 기본 키가 아닌 속성이 다른 속성에 의존하지 않도록 분리
- 예: [학생(학번, 학과코드, 학과명)] → 학과명은 학과코드에 종속 → "학과" 테이블로 분리

제 1 정규형

비정규화된 테이블:

고객ID	고객이름	주문1	주문2	주문3
1	홍길동	TV	냉장고	세탁기
2	김철수	컴퓨터	스마트폰	

제1정규화된 테이블:

고객ID	고객이름	주문
1	홍길동	TV
1	홍길동	냉장고
1	홍길동	세탁기
2	김철수	컴퓨터
2	김철수	스마트폰

제 1 정규형

• 중복 제거

- 각 컬럼은 하나의 값만 보유 (반복되는 값을 각 행으로 분리)
- 모든 셀에 단일값만 남도록 테이블 재구성
- 컬럼 내에 중복된 값이 없어야 함

• 원자성

- 각 컬럼은 더 이상 분할할 수 없는 최소 단위의 데이터로 구성

• 관계형 데이터 모델 기초

- 정규화의 첫 번째 단계로, 기본적인 데이터 무결성을 확보

• 위반

- 한 셀에 여러 값이 들어가거나, 반복되는 그룹이 있으면 1NF를 위반

제 2 정규형

비정규화된 테이블:

주문ID	고객ID	고객이름	제품ID	제품명
1	1	홍길동	101	TV
2	1	홍길동	102	냉장고
3	2	김철수	103	세탁기
4	2	김철수	104	컴퓨터

고객 테이블:

고객ID	고객이름
1	홍길동
2	김철수

주문 테이블:

주문ID	고객ID	제품ID
1	1	101
2	1	102
3	2	103
4	2	104

제 2 정규형

- **종속성 제거**
 - 제2 정규형은 제1 정규형을 만족
 - 비키(Non-key) 속성이 기본키의 일부분에만 종속되는 경우 제거
- **기본키 완전 종속**
 - 기본키가 복합키일 경우 모든 컬럼이 기본키 전체에만 완전 종속
 - 비키 속성은 기본키 전체에 완전히 종속 (부분적 함수 종속 제거)
- **부분 종속 제거**
 - 기본키의 일부분에만 종속되는 비키 속성은 별도의 테이블로 분리하여 부분 종속 제거

제 2 정규형

제품 테이블:

제품ID	제품명
101	TV
102	냉장고
103	세탁기
104	컴퓨터

제 2 정규형

- 종속성 제거
 - 제2 정규형은 제1 정규형을 만족
 - 비키(Non-key) 속성이 기본키의 일부분에만 종속되는 경우 제거
- 기본키 완전 종속
 - 기본키가 복합키일 경우 모든 컬럼이 기본키 전체에만 완전 종속
 - 비키 속성은 기본키 전체에 완전히 종속 (부분적 함수 종속 제거)
- 부분 종속 제거
 - 기본키의 일부분에만 종속되는 비키 속성은 별도의 테이블로 분리하여 부분 종속 제거

제 3 정규형

고객ID	고객이름
1	홍길동
2	김철수

주문 테이블:

주문ID	고객ID	제품ID
1	1	101
2	1	102
3	2	103
4	2	104

제품 테이블:

제품ID	제품명
101	TV
102	냉장고
103	세탁기
104	컴퓨터

제 3 정규형

정의

- 제 3 정규형은 제 2 정규형을 만족
- 비키(비기본키) 속성이 기본키에 의존하지 않고 다른 비키에 함수적으로 종속되지 않는 상태
- 비키가 기본키에 의존하지 않고 다른 비키의 값으로 결정될 수 없어야 함

제 3 정규형

고객ID	고객이름
1	홍길동
2	김철수

주문 테이블:

주문ID	고객ID	제품ID
1	1	101
2	1	102
3	2	103

제품 테이블:

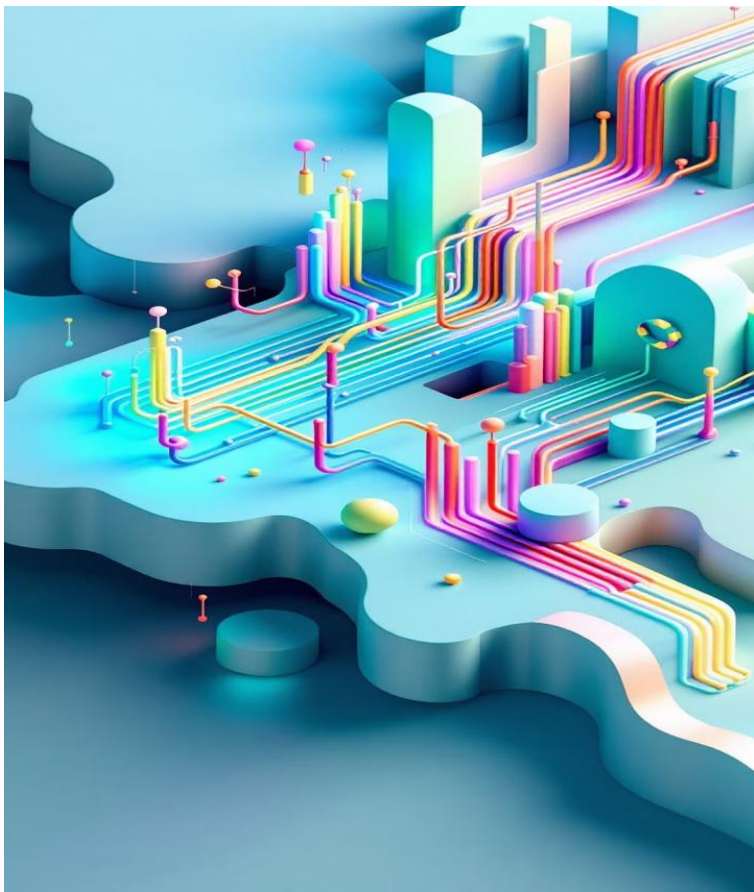
제품ID	제품명	공급업체명
101	TV	삼성
102	냉장고	LG
103	세탁기	LG
104	컴퓨터	애플

제 3 정규형

정의

- 제 3 정규형은 제 2 정규형을 만족
- 비키(비기본키) 속성이 기본키에 의존하지 않고 다른 비키에 함수적으로 종속되지 않는 상태
- 비키가 기본키에 의존하지 않고 다른 비키의 값으로 결정될 수 없어야 함

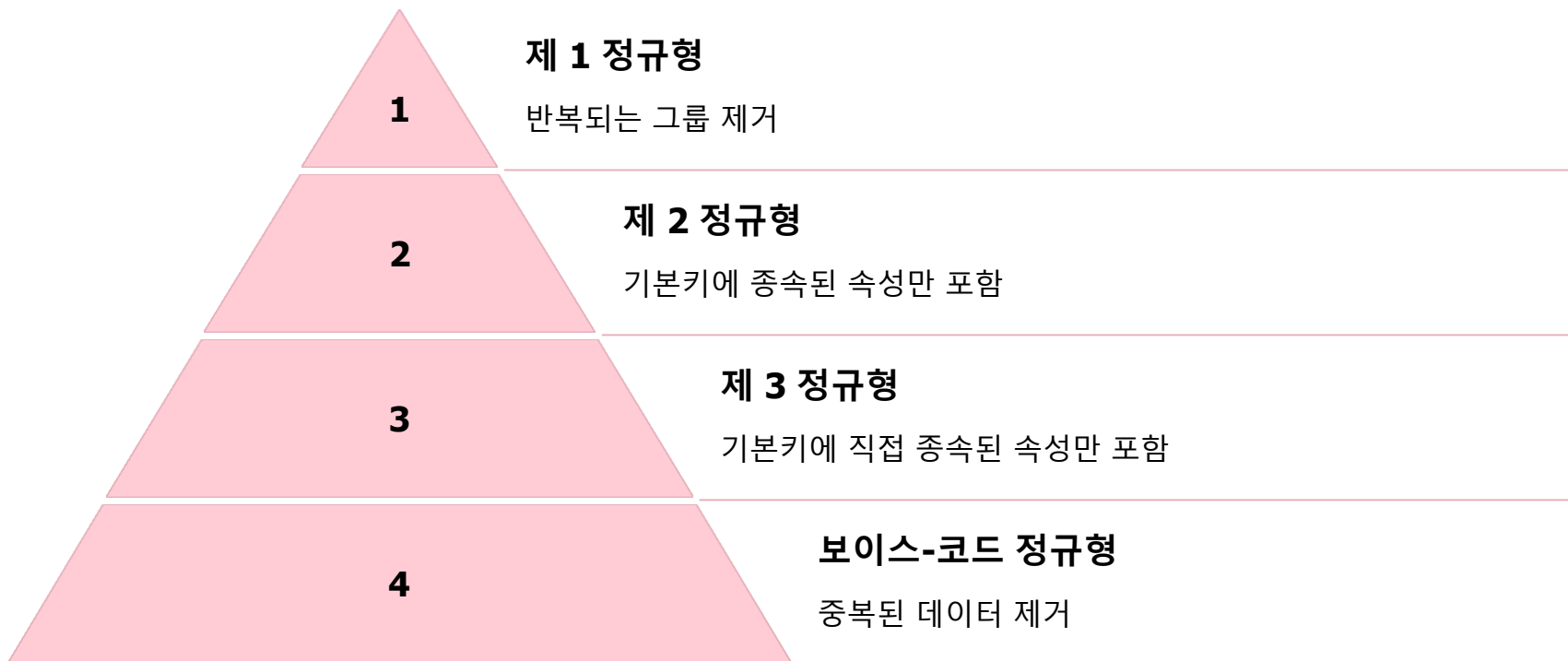
보이스-코드 정규형



보이스-코드 정규형

- ✓ **종속성 제거**
 - 보이스-코드 정규형은 모든 종속성을 제거하는 가장 높은 수준의 정규화 형태
- ✓ **최소 중복성**
 - 데이터 중복을 최소화하여 데이터 무결성을 보장
 - 유지 관리를 간소화
- ✓ **성능 향상**
 - 중복 데이터가 줄어들어 데이터 검색 및 업데이트 성능이 향상
- ✓ **복잡성 증가**
 - 데이터베이스 설계 및 관리가 복잡해지고 성능 저하를 유발

정규화 과정 종합 정리



정규화는 데이터베이스 설계 단계에서 데이터 중복성을 제거하고 무결성을 높이는 중요한 과정
정규화 과정을 통해 데이터베이스 효율성을 향상시키고 데이터 일관성을 유지

정규화 과정 종합 정리

정규형	조건 및 목적	변환 방법 요약
1NF	모든 컬럼이 원자값(Atomic Value)	반복/복합값 분해
2NF	1NF + 부분 함수 종속 제거(기본키 전체에만 종속)	부분 종속 속성 분리
3NF	2NF + 이행적 함수 종속 제거(비기본키 간 종속 제거)	이행 종속 속성 분리

정규화를 통해 데이터 중복을 줄이고, 삽입/삭제/갱신 이상을 방지

정규화 (Normalization) — 왜 필요한가?

정규화

- 데이터 중복 최소화, 이상(Anomaly) 현상 방지, 무결성 향상
- 비정규화 테이블에서 발생하는 현상
 - 삽입 이상 (Insertion Anomaly)
: 일부 데이터 저장 시 불필요한 데이터도 저장
 - 삭제 이상 (Deletion Anomaly)
: 데이터 삭제 시 다른 중요 정보도 삭제
 - 갱신 이상 (Update Anomaly)
: 중복 저장된 데이터 중 일부만 수정
→ 불일치 발생

정규화 단계

- **1NF → 2NF → 3NF → BCNF**
(보통 3NF 또는 BCNF까지 적용)
- 과도한 정규화
→ **JOIN** 비용 증가
→ 성능 고려 시
반정규화(Denormalization)
전략 도입

정규화 단계별 요약

정규형	조건	제거 대상	예시
1NF	각 컬럼은 원자값(Atomic Value)만	• 반복 그룹, 다중값 속성	• "과목 = DB, 네트워크" → 별도 행으로 분리
2NF	1NF + 부분 함수 종속 제거	• 복합키 일부에만 종속된 속성	• (학번+과목코드) → 학생이름은 학번에만 종속
3NF	2NF + 이행적 종속 제거	• 비-키 속성이 다른 비-키 속성에 종속	• 학번→학과코드→학과명 이행 종속 제거
BCNF	3NF + 결정자는 반드시 후보키	• 비후보키 결정자 제거	• 모든 함수 종속의 결정자가 후보키여야 함

정규화 전/후 비교 – SQL 예시

【 SQL 예시 】

-- 비정규화 (1NF 위반: 다중값 컬럼)

```
CREATE TABLE student_bad (
  student_id INT PRIMARY KEY,
  name VARCHAR(50),
  subjects VARCHAR(200) -- "DB, 네트워크, OS" ← 원자값 아님!
);
```

Checkpoint

- ✓ 정규화는 설계 단계에서 데이터 중복·이상을 방지하는 핵심 원칙

-- 1NF + 2NF + 3NF 적용 후 (분리된 테이블)

```
CREATE TABLE students (
  id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  major_id INT REFERENCES majors(id) -- 3NF: 학과명 분리
);
```

```
CREATE TABLE majors (
  id INT PRIMARY KEY,
  code VARCHAR(20) UNIQUE,
  name VARCHAR(100) -- major_id → name (이행종속 제거)
);
```

```
CREATE TABLE enrollments ( -- 1NF: 과목 분리 + 2NF 교차테이블
  student_id INT REFERENCES students(id),
  course_id INT REFERENCES courses(id),
  score DECIMAL(5,2),
  PRIMARY KEY (student_id, course_id) -- 복합 PK
);
```

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

3. ERD — Entity, Attribute, Relationship

- 엔티티 · 속성 · 관계 · ERD 작성 방법 · IE 표기법

실습 환경 구성

PostgreSQL (권장)

- 오픈소스 RDBMS, 표준 SQL 호환, JSONB/GIS/Vector 확장 풍부
- postgresql.org/download
- 기본 포트: 5432 (Mac: 5433)
- 실습 버전 : PostgreSQL 14 or 17
(17 이하 권장)

ERD 설계 도구 (온라인)

- dbdiagram.io
 - DBML 문으로 ERD 빠르게 작성
 - SQL 내보내기
- ERDCloud (erdcloud.com)
 - 한국어 UI, 팀 협업
- Lucidchart
 - 실시간 협업, 다양한 다이어그램

DBeaver (GUI 클라이언트): 무료, 멀티 DB 지원, ERD 시각화, 쿼리 편집기

Entity (엔티티, 개체) — 개념 및 종류

엔티티

- 데이터베이스에서 관리하고자 하는 대상 객체
 - 유형 엔티티: 사람, 사물, 장소 (학생, 교수, 강의실)
 - 무형 엔티티 : 개념, 사건, 행위 (수강신청, 주문)

엔티티 집합 (Entity Set)

- 동일한 특성을 가진 엔티티들의 모임
- 모든 학생들의 집합 = "학생" 엔티티 집합

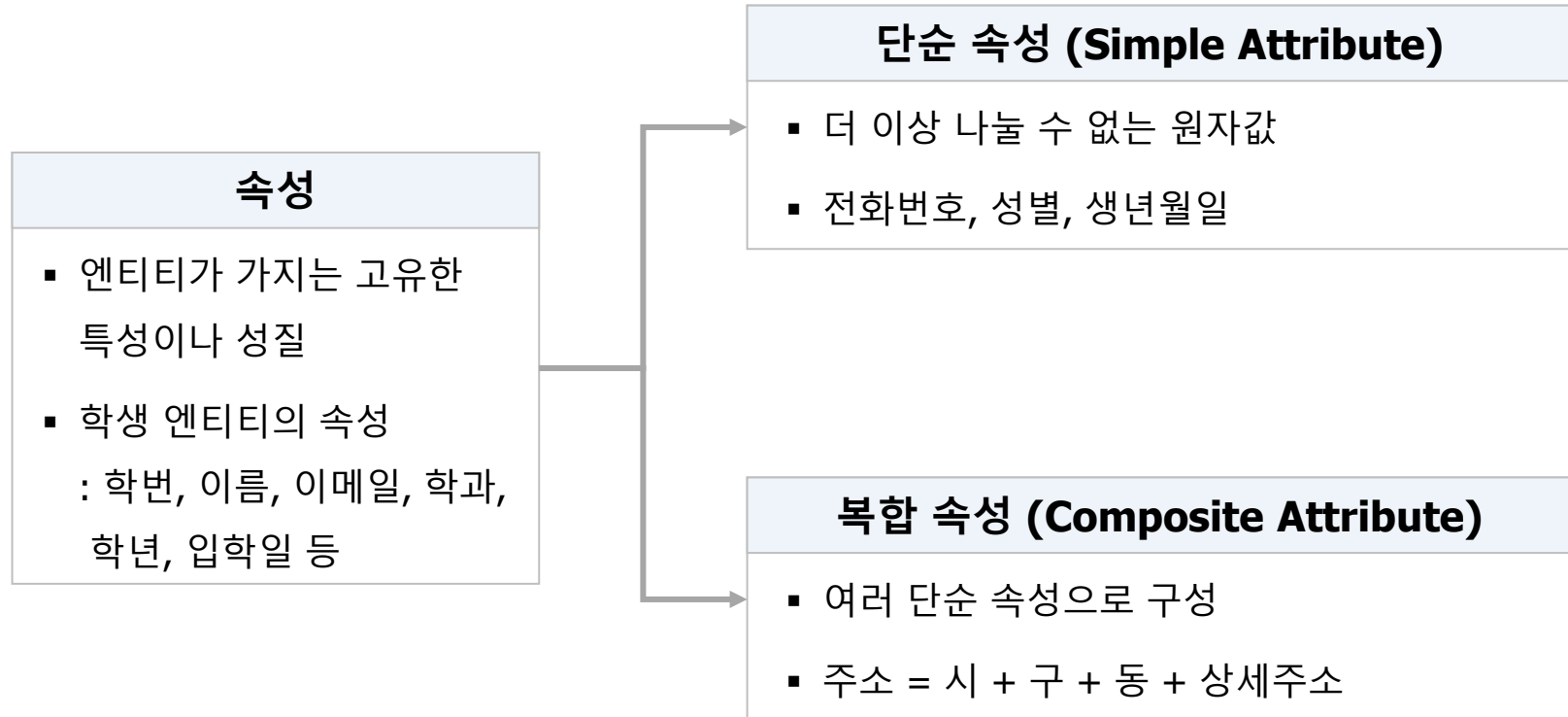
강한 엔티티 (Strong Entity)

- 독자적인 PK 존재, 혼자서도 식별 가능
- 학생(학번), 강좌(강좌코드), 교수(교수번호)

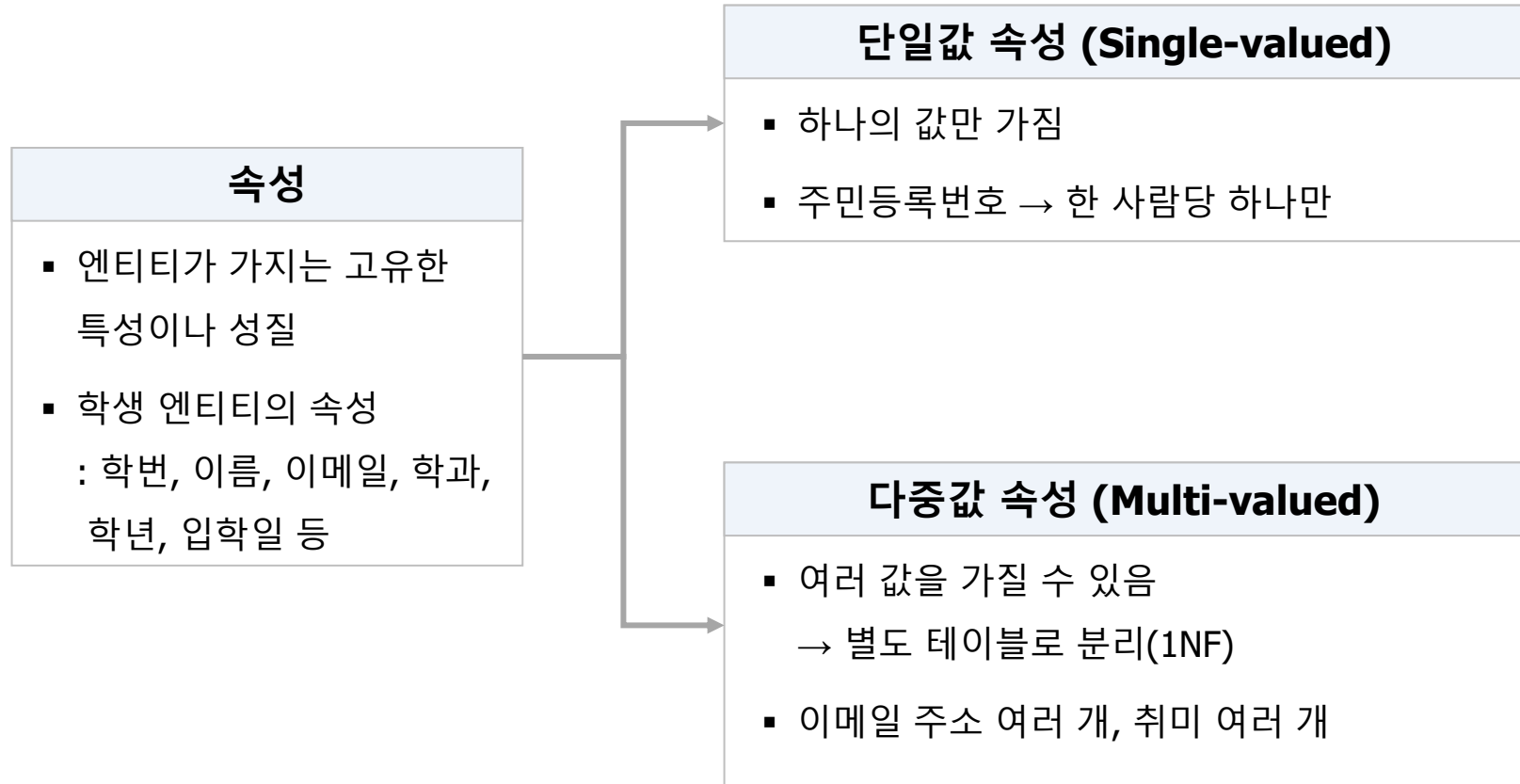
약한 엔티티 (Weak Entity)

- 독자적 PK 없음, 다른 강한 엔티티에 의존해야 식별 가능
 - OrderItem — OrderID + ItemNo 복합키
(Order 없이 존재 불가)
 - ERD에서 이중 사각형으로 표현,
식별 관계는 굵은 선
 - 부모 엔티티 삭제 시 자식도 삭제
(ON DELETE CASCADE)

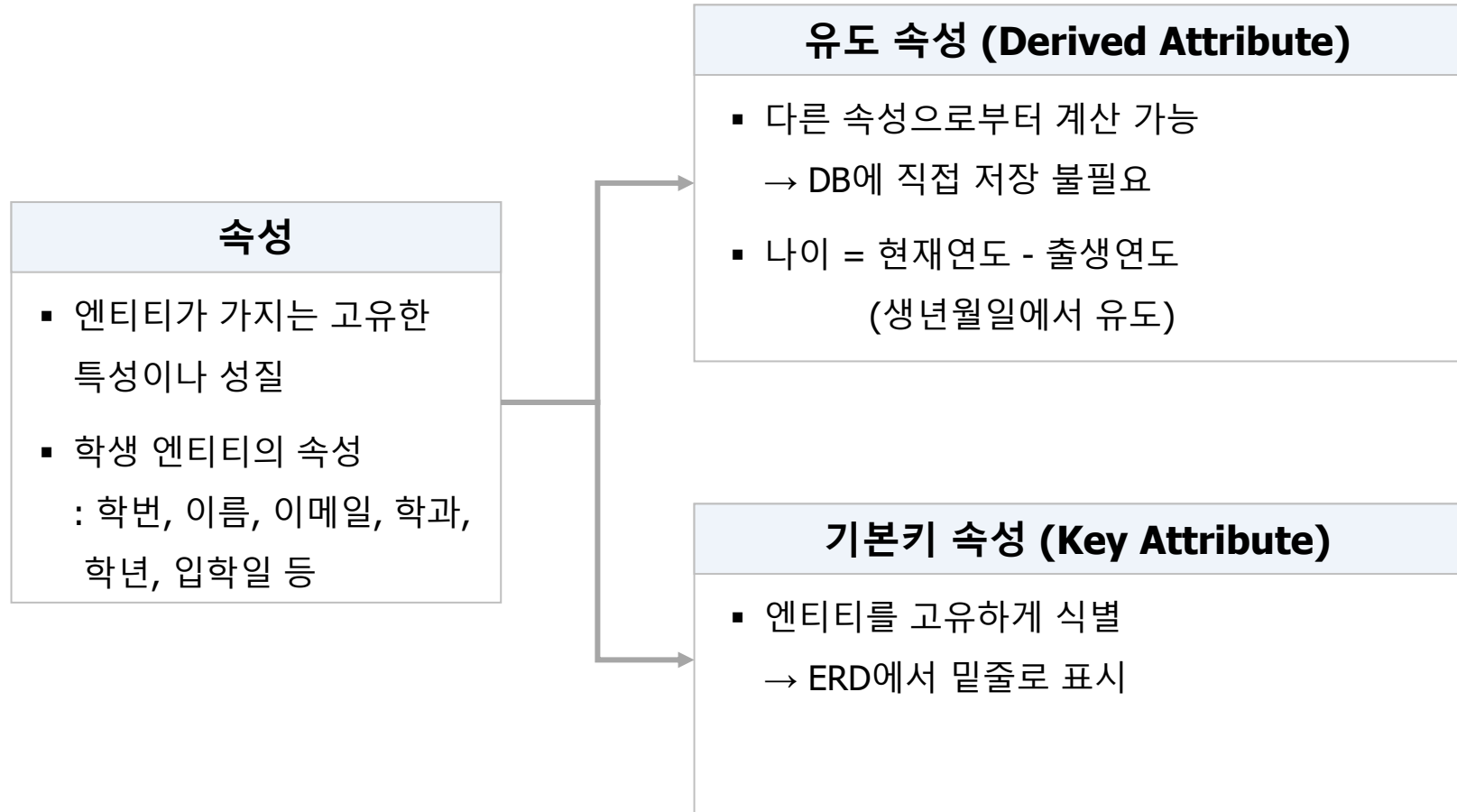
Attribute (속성) — 종류 및 특성



Attribute (속성) — 종류 및 특성



Attribute (속성) — 종류 및 특성



【 작성 단계 】

요구사항 수집	엔티티 도출	속성 정의	관계 설정	카디널리티 명시	ERD 도구 시각화	검토 및 수정
- 시스템에서 관리해야 할 정보, 업무 흐름을 파악 - 학생 관리 시스템에서는 학생, 강의, 교수, 과제 등	- 관리대상의 주요 객체 추출 후 엔티티로 정의 (주로 명사) - 학생(Student), 강의(Course), 교수(Professor)	- 각 엔티티가 가져야 할 특징(속성)을 정리 - 학생 → 학번, 이름, 전공 - 강의 → 강의코드, 강의명	- 엔티티 간의 연관성 파악, 관계 정의 - 학생과 강의는 '수강한다' - 교수와 강의는 '담당한다'	- 관계의 수 (1:1, 1:N, N:M 등)를 명확히 표시 - 한 학생은 여러 강의를 수강 - 한 강의는 여러 학생이 수강 → 학생-강의 N:M	- 도형 - 사각형 : 엔티티 - 타원 : 속성 - 마름모 : 관계 다이어그램 - Lucidchart, GitMind, Aquerytool, DBdiagram 등 다양한 무료 도구 활용	- 작성한 ERD를 팀원과 함께 검토 - 데이터 무결성 및 요구사항 반영 여부를 확인

IE 표기법

IE 표기법 (까마귀발 표기법, Crow's Foot)

IE 표기

- | — 하나(One)
- ≤ (까마귀발) — 여럿(Many)
- ○ — 없거나 하나(Zero or One)
- || — 반드시 하나(One and Only One)

관계선 종류

- 식별 관계 (실선)
 - FK가 자식의 PK 일부
 - 부모 없이 자식 존재 불가 (약한 엔티티)
- 비식별 관계 (점선)
 - FK가 자식의 일반 컬럼
 - 부모 없이도 자식 존재 가능

dbdiagram.io DBML 문법으로 ERD 작성

【 SQL 예시 】

```
// dbdiagram.io (DBML - Database Markup Language)
// SQL 없이 빠르게 ERD 작성 → SQL DDL 자동 생성
```

```
Table students {
  id      bigint   [pk, increment, note: 'PK']
  student_no varchar(20) [unique, not null, note: '학번']
  name     varchar(100) [not null]
  email     varchar(200) [unique, not null]
  major_id int      [ref: > majors.id, note: 'FK → 학과']
  grade     smallint [not null, note: '1~4학년']
  created_at timestamp [default: 'now()']
}
```

```
Table majors {
  id int [pk, increment]
  code varchar(20) [unique, not null]
  name varchar(100) [not null]
}
```

```
Table enrollments { // N:M 교차 테이블
  student_id bigint [ref: > students.id]
  course_id int [ref: > courses.id]
  score decimal(5,2)
  enrolled_at date [not null]
  Indexes {
    (student_id, course_id) [pk] // 복합 PK
  }
}
```

```
// 관계 방향: > 다대일, < 일대다, - 일대일, <> 다대다
```

Checkpoint

- ✓ **dbdiagram.io**
 - 왼쪽에 **DBML** 코드 작성
 - 오른쪽에 **ERD** 자동 생성
- **PNG/SQL** 내보내기 가능

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

4. 개념적 → 논리적 → 물리적 모델 설계

- 3단계 설계 흐름 · 약한 엔터티 · N:M 관계 설계

개념적 vs 논리적 데이터 모델링

개념적 데이터 모델링

- 비즈니스 요구사항을 추상화
- **DBMS**에 독립적인 수준
- 주요 활동
 - 엔터티 식별 (명사 추출)
 - 관계 정의 (**1:N, N:M**)
 - **ERD** 작성 (이해관계자와 소통)
 - 복합 속성 처리
 - 약/강 엔터티 구분
- 산출물: **ERD** 다이어그램

논리적 데이터 모델링

- 개념 모델 → 논리 구조로 변환
- **DBMS**에 독립적
- 주요 활동
 - 엔터티 → 테이블 매핑
 - **PK/FK** 설정
 - 정규화 (**1NF~BCNF**) 적용
 - 반정규화 전략 검토
 - 상속 관계 모델링
(**Single Table vs Class Table**)
- 산출물: 논리 **ERD**, 데이터 사전

물리적 데이터 모델링 핵심 항목

1**특정 DBMS 환경에 최적화된 물리적 스키마 설계****2****데이터 타입 결정: DBMS별 최적 타입 선택**

- ✓ 금액: NUMERIC(10,2) — 부동소수점 오류 방지
- ✓ 시간: TIMESTAMPTZ (타임존 포함) — 글로벌 서비스 필수
- ✓ 문자열: VARCHAR(n) vs TEXT — PostgreSQL은 TEXT 제한 없음

3**인덱스 설계: WHERE/JOIN/ORDER BY 컬럼에 인덱스 추가**

- ✓ B-Tree (기본), Hash (등호만), GIN (JSONB/배열), GiST (GIS)

물리적 데이터 모델링 핵심 항목

4

파티셔닝: 대용량 데이터를 연도/지역별로 분할 저장

5

클러스터형 vs 비클러스터형 인덱스: 데이터 물리적 배치 방식

6

저장 공간 최적화: 압축, 컬럼 기반 저장(Columnstore), 아카이빙 전략

7

트랜잭션 로드 테스트: 동시 사용자를 고려한 Lock 전략 (낙관적/비관적)

낙관적 Lock vs 비관적 Lock

낙관적 Lock (Optimistic)

- 데이터 저장 시 충돌이 드물다고 가정
- 트랜잭션 종료 시 버전 비교로
- 충돌 감지 (버전 컬럼 활용)
- 적합 상황
 - 읽기 위주 작업
 - 갱신 빈도 낮음
 - 긴 트랜잭션 (사용자 세션)
예: 블로그 댓글 수정
- 구현: `updated_at` 또는 `version`
- 컬럼으로 충돌 감지

비관적 Lock (Pessimistic)

- 트랜잭션 시작 시 Lock 획득
- 다른 트랜잭션의 접근 차단
- 적합 상황
 - 고충돌 환경
 - 은행 이체, 좌석 예매
 - 쓰기 집약적 시스템
 - 짧은 트랜잭션 (ms 단위)
- 구현
 - `SELECT ... FOR UPDATE`
 - `SELECT ... FOR UPDATE NOWAIT`

약한 엔티티와 N:M 관계 설계 패턴

약한 엔티티 설계 원칙

- 독자적 PK 無
→ 강한 엔티티의 PK
+ 자신의 부분키로
복합 PK 구성
- **ON DELETE CASCADE** 필수
 - 부모(강한 엔티티) 삭제 시
자식(약한 엔티티)도 삭제
 - OrderItem(OrderID, ItemNo)
: Order 없이 존재 불가
 - Dependent(EmployeeID,
DependentName)
: Employee 없이 존재 불가

N:M 관계

- 교차(중간) 테이블 설계
 - 두 엔티티의 FK를 포함하는
교차 테이블 필수
 - 복합 PK(FK1 + FK2) 또는
대체 PK(surrogate key) 선택
 - Non-Clustered Index 추가로
양방향 검색 성능 최적화
 - 교차 테이블에 추가 속성 가능
→ 수강(enrolled_at, score),
주문상품(quantity, unit_price)

인덱스 전략

- 대량 데이터 시
- 전략 (빠른 검색 목적)
 - 어떤 컬럼으로 **WHERE** 조건을
거느냐에 따라 인덱스를
양방향으로 만들어두는 전략
 - Enrollment 테이블
: INDEX(student_id, course_id)
+ INDEX(course_id, student_id)
 - 파티셔닝
: 학기별로 Enrollment 테이블
분할

약한 엔티티

엔티티의 속성에 따라 강한 엔티티/약한 엔티티로 구분

약한 엔티티(Weak Entity)

자기 혼자서는 고유하게 식별(구분)할 수 없는 엔티티
다른 엔티티(강한 엔티티)와 같이 있어야 식별 가능

- 독자적인 기본 키 없음
- 상위 엔티티의 키 + 자신의 속성으로 복합 키를 구성

- 이름이나 코드만으로는 유일하지 않으므로 혼자서는 구분 불가
- 부모 엔티티 삭제 시 자식도 삭제
- 복합키(PK+FK) 시 성능/가독성 고려

약한 엔티티는 혼자서 구분될 수 없고,
반드시 다른 강한 엔티티에 의존해서
존재하는 데이터 구조

[예시] 약한 엔티티

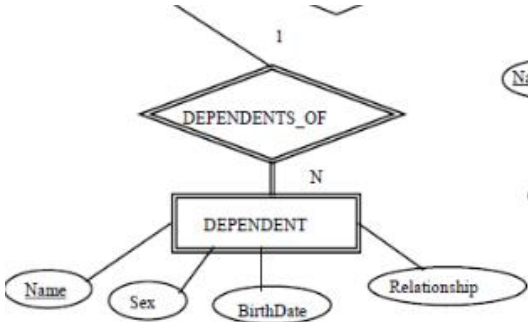
엔티티의 속성에 따라 강한 엔티티/약한 엔티티로 구분

엔티티	설명	비고
order	주문 자체(주문번호, 고객명 등)	강한 엔티티
orderitem	주문 상세 항목 (상품명, 수량 등)	약한 엔티티

- OrderItem은 주문(Order) 없이는 존재 불가
- OrderItem의 기본 키는 (OrderID, ItemNo) 같이 복합 키로 구성

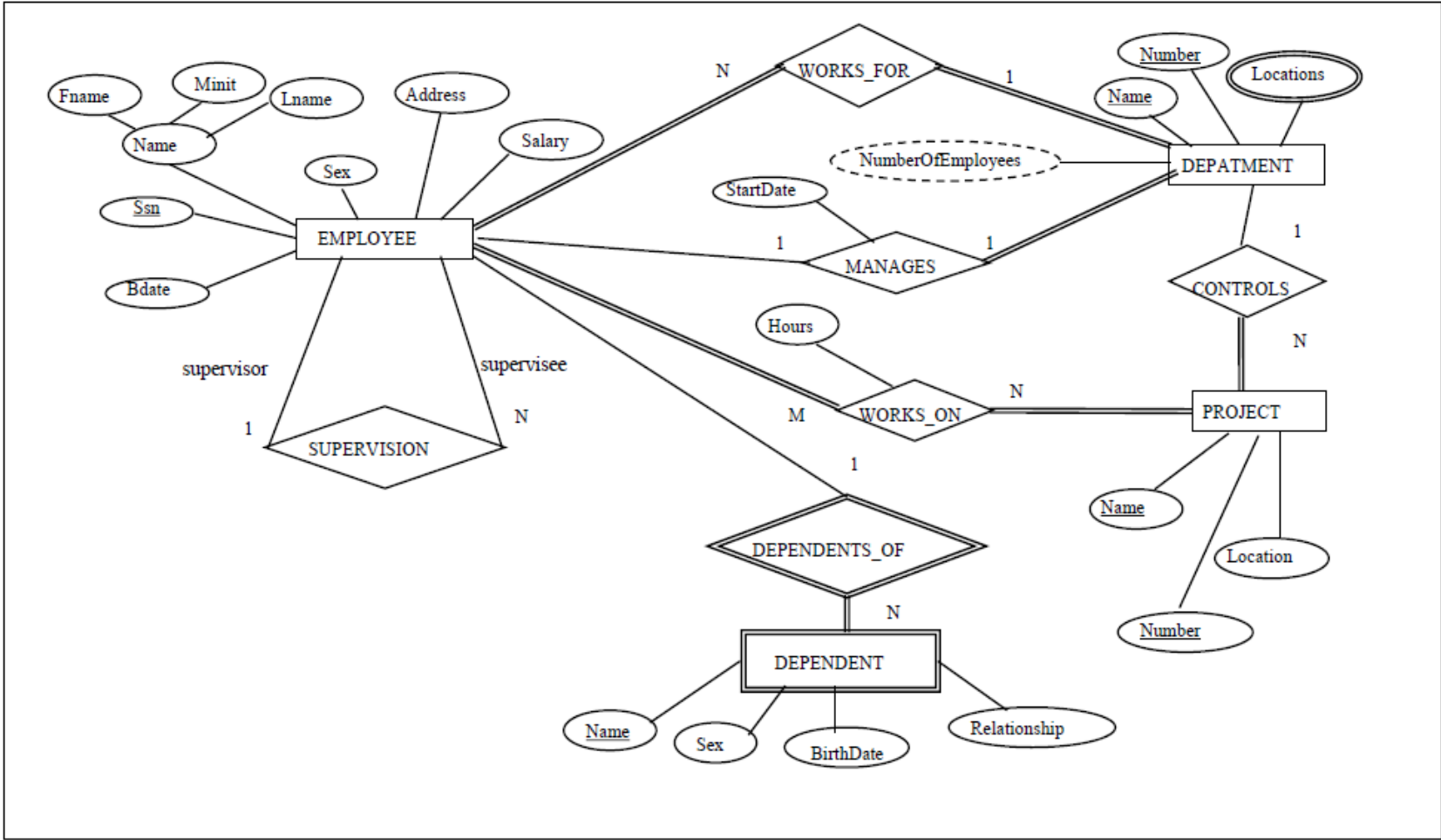
【 ERD에서 약한 엔터티 표현 】

- 약한 엔터티는 이중 사각형(double rectangle)으로 표현
- 식별 관계(Identifying Relationship)는 굵은 선으로 연결
- 상위 엔터티의 PK + 하위 엔터티의 일부 속성 = 복합 PK



[예시] 약한 엔티티

엔티티의 속성에 따라 강한 엔티티/약한 엔티티로 구분



[예시] 약한 엔티티

Table Creation(SQL statements in Oracle)

```

create table EMP(
    eid                int,
    ename              varchar2(20) NOT NULL,
    sex                char,
    job                varchar2(20),
    address            varchar2(20),
    super_eid          int,
    did                int default 300,
    salary              number(10,2),
    bonus              number(10,2),
    emp_start_date      date,
    check (salary > bonus),
    primary key(eid),
    foreign key (super_eid) references emp(eid)
);

create table DEPT(
    did                integer,
    dname              varchar2(20),
    location            varchar2(20),
    mgr_eid             integer,
    mgr_start_date      date,
    primary key(did),
    unique (ename),
    foreign key (mgr_eid) references emp(eid)
);

alter table emp add constraint did_ref
foreign key (did) references dept(did);

```

```

create table PROJ(
    pid                int,
    pname              varchar2(20),
    location            varchar2(20),
    did                integer,
    proj_start_date      date,
    primary key(pid),
    unique(pname),
    foreign key (did) references dept(did)
);

create table WORKS_ON(
    eid                int,
    pid                int,
    hours              number(10,2),
    primary key(eid, pid),
    foreign key (eid) references emp(eid),
    foreign key (pid) references proj(pid)
);

create table DEPENDENTS(
    eid                integer,
    dp_name            varchar2(20),
    sex                char,
    birth_date          date,
    relationship        varchar2(20),
    primary key(eid, dp_name),
    foreign key (eid) references emp(eid)
);

```

- int : 자동으로 NUMBER(38)로 변환된다. == integer
- varchar2(20) : 가변길이 최대 20 bytes. == varchar2(20 byte)
varchar2(20 char) : 가변길이 최대 20 자.
- char(5) : 고정길이 5 bytes. == char(5 byte)
char(5 char) : 고정길이 5 자.
- char : 고정길이 1 byte == char(1 byte)
- number : 실수 표현
- number(10,2) : 전체 10 자리, 소숫점 이하 2 자리, 그 이하는 반올림.
- number(10) : 정수 최대 10 자리. 소수점 이하는 반올림.
- date : YYYY/MM/DD HH24:MI:SS

primary key: 기본(基本) 키, foreign key: 외래(外來) 키
constraint: 제약조건

```

insert into emp values (10, '일지매', 'M', 'SALESMAN', '대구', NULL, 100, 2000.80, 120.4, '2010-3-21');
insert into emp values (20, '춘향', 'F', '인사', '서울', 10, 200, 1000.4, 10.06, to_date('2018/12/31 03:15:32', 'yyyy/mm/dd hh24:mi:ss'));

```

[예시] 약한 엔티티

```
CREATE TABLE Dependent (  
  EmployeeID INT,  
  DependentName VARCHAR(50),  
  BirthDate DATE,  
  PRIMARY KEY (EmployeeID, DependentName),  
  FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID)  
  ON DELETE CASCADE  
);
```

- 설계 시 고려사항

- Cascading Delete: 강한 엔티티 삭제 시 약한 엔티티 연쇄 삭제 여부 결정

다대다(M:N) 관계

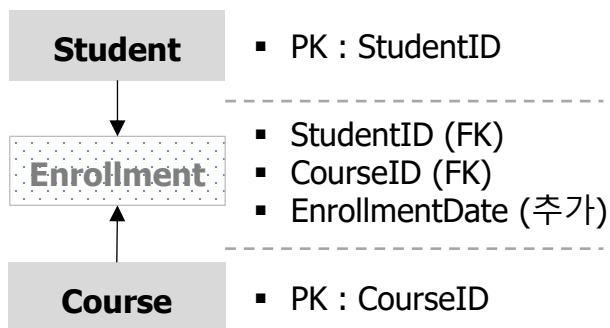
다대다 관계 설정 시 두 엔터티가 다수의 레코드를 연결하므로 이에 대한 해결 방안 필요

다대다

기본 구조 문제점

- 두 엔터티가 상호 다수의 레코드를 연결
- Student ↔ Course, Order ↔ Product
- 직접 구현 불가 → 조인 테이블 필요

조인테이블 설계 전략



성능 최적화/해결 방안

- 인덱스 전략 (**Non-Clusterd Index**추가)
- 파티셔닝
(대량 데이터 시 Enrollment 테이블을 학기별로 분할)

- 자동 증가 ID: 단순화 but 의미 없는 키
- 복합 키: (**StudentID, CourseID**)
→ 데이터 무결성 강화

약한 엔티티 및 N:M 교차 테이블 DDL

【 SQL 예시 】

-- 약한 엔티티 예시: OrderItem (Order 없이 존재 불가)

```
CREATE TABLE order_items (
  order_id INT REFERENCES orders(id) ON DELETE CASCADE,
  item_no  SMALLINT NOT NULL,          -- 부분키 (약한 엔티티)
  product_id INT REFERENCES products(id),
  quantity INT NOT NULL CHECK (quantity > 0),
  unit_price NUMERIC(10,2) NOT NULL,
  PRIMARY KEY (order_id, item_no)      -- 복합 PK
);
```

-- N:M 교차 테이블: 학생-강좌 수강신청

```
CREATE TABLE enrollments (
  student_id BIGINT REFERENCES students(id) ON DELETE CASCADE,
  course_id  INT  REFERENCES courses(id) ON DELETE CASCADE,
  enrolled_at DATE NOT NULL DEFAULT CURRENT_DATE,
  score      NUMERIC(5,2) CHECK (score BETWEEN 0 AND 100),
  PRIMARY KEY (student_id, course_id),    -- 복합 PK로 중복 방지
  CONSTRAINT chk_score CHECK (score IS NULL OR score BETWEEN
0 AND 100)
);
```

-- 양방향 조회 최적화 인덱스

```
CREATE INDEX idx_enroll_course ON enrollments
(course_id, student_id);
```

Checkpoint

- ✓ **where** 조건이 어느 컬럼을 자주 쓰느냐에 따라서 **Index**는 별도로 조정 (단, **Non-Cluster Index** 권장)

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

5. 스키마 전략 & RDBMS 비교

- 멀티 프로젝트 설계 · MySQL · PostgreSQL · Oracle · Cloud DB

스키마 기반 멀티 프로젝트 설계 전략

스키마

- 테이블/뷰/함수 등 **DB 객체**를 그룹화하는 논리적 네임스페이스

Bridge Model 구조 예시

- **maindb/global**
(공통: users, products, exchange_rates)
- **maindb/prj_alpha**
(프로젝트 A: orders, promotions)
- **maindb/prj_beta**
(프로젝트 B: inventory, suppliers)

멀티 테넌시 3가지 패턴

- **Silo Model**
 - 테넌트별 완전 독립 **DB 인스턴스**
→ 보안 최강, 운영 비용 최대
- **Pool Model**
 - 단일 **DB** + **tenant_id** 컬럼으로 구분
→ 운영 단순, 데이터 혼재
- **Bridge Model (PostgreSQL 권장 방식)**
 - 단일 **DB 인스턴스**
+ 테넌트별 스키마 분리

주요 장점

- 물리적 자원 효율성 (단일 인스턴스), 스키마 단위 접근 제어, 버전 관리
- 성능 최적화
 - 프로젝트별 독립적인 인덱스/파티셔닝 전략 적용
- **Vector DB** 연계 시 프로젝트별 임베딩 저장 스키마 분리 설계 가능

주요 RDBMS 특징 비교

DBMS	라이선스	기본 격리	JSON 지원	주요 강점	활용 시나리오
MySQL/MariaDB	• 오픈소스	• Repeatable Read	• JSON (8.0+)	• 빠름, 배우기 쉬움	• 웹서비스, 스타트업
PostgreSQL	• 오픈소스	• Read Committed	• JSONB (강력)	• SQL 표준, GIS, AI/ML	• 분석, 금융, AI 서비스
Oracle	• 상용	• Read Committed	• 21c+ 지원	• 고가용성, 안정성	• 은행, 대기업 ERP
SQL Server	• 상용	• Read Committed	• JSON 함수	• MS 생태계, BI	• ERP, CRM, PowerBI
AWS Aurora	• 관리형(유료)	• Read Committed	• MySQL/PG 호환	• MySQL 대비 3~5배 빠름	• 글로벌 서비스
GCP Cloud SQL	• 관리형(유료)	• PG/MySQL 기본	• PG/MySQL 호환	• BigQuery 연계	• ML 파이프라인

MySQL / MariaDB

특징

- **경량 & 빠른 성능**
→ 웹 서비스에서 초기에 많이 사용
- **LAMP 스택의 핵심 DB**
(Linux, Apache, MySQL, PHP/Python)
- **InnoDB 스토리지 엔진** → 트랜잭션, 외래키(FK), MVCC 지원
- **MariaDB는 MySQL의 오픈소스 분기(Fork)로, 커뮤니티 중심 개발**

장점

- 배우기 쉽고, 설치/운영 부담이 적음
- 전 세계 웹사이트에서 기본적으로 사용 (WordPress, Drupal 등)

활용사례

- 쇼핑몰, 블로그, 스타트업 웹서비스의 DB
- **MariaDB는 MySQL과 호환되면서 추가 기능을 빨리 반영** (JSON, Window Functions)

단점

- 복잡한 대규모 시스템에서는 성능/확장성 한계 (수천 TPS 이상)
- 고급 기능은 부족 (예: 분산 트랜잭션, 고급 보안)

PostgreSQL

특징

- "가장 진보한 오픈소스 RDBMS"
- 표준 SQL 호환성이 뛰어나고, ANSI SQL:2016까지 잘 반영
- 확장성: JSONB, PostGIS(지리정보 GIS), 사용자 정의 함수/자료형
- MVCC 기반으로 동시성 처리에 강함

장점

- 복잡한 쿼리, 데이터 분석, GIS, JSON 같은 반정형 데이터 처리에 유리
- 오픈소스인데도 엔터프라이즈 급 기능 제공

활용사례

- 일부 기업들이 Postgres 기반으로 전환 중
- GIS 기반 서비스(배달앱 경로, 위치 서비스)
- 금융/핀테크 쪽에서 점차 도입 확대

단점

- 상대적으로 운영 난이도가 있음 (튜닝, 학습 곡선이 높음)
- DB 관리 경험이 부족한 경우 관리가 까다로울 수 있음

[참고] ANSI SQL:2016

데이터베이스 언어 SQL의 공식 국제 표준(버전 2016년판)

【 배경 】

- SQL은 1970년대 IBM에서 처음 만들어졌고,이후 국제 표준화 기구(ANSI → ISO)가 공식 표준으로 제정
- 표준 이름 : ISO/IEC 9075:2016 (ANSI SQL:2016)

ANSI (American National Standards Institute)
: 미국 표준화 기관 — 미국 표준 버전 SQL

ISO/IEC (International Organization for Standardization / International Electrotechnical Commission)
: 국제 표준화 기관 — 전 세계 공통 버전 SQL

주요 내용

- **ANSI SQL**
 - 모든 DBMS가 따라야 하는 공통 규격(표준 문법)
- 왜 중요?
 - 여러 RDBMS를 다루는 개발자라면, ANSI SQL 문법으로 작성하면 이식성(Portability)이 좋음
- **ANSI SQL**
 - : SELECT, JOIN, GROUP BY, HAVING, CASE WHEN 같은 기본 문법
- **Non-ANSI SQL**
 - : LIMIT(MySQL) vs TOP(SQL Server) DB마다 문법이 다른 것

[참고] ANSI SQL:2016

데이터베이스 언어 SQL의 공식 국제 표준(버전 2016년판)

【 SQL 표준의 역사 】

버전	연도	주요 특징
SQL-86	• 1986	• 최초 ANSI 표준 제정
SQL-89	• 1989	• 소폭 개정
SQL-92	• 1992	• 대부분 DBMS의 기본 표준 (SELECT, JOIN 등)
SQL:1999	• 1999	• 객체지향 확장 (UDT), 트리거, RECURSIVE WITH
SQL:2003	• 2003	• XML 지원, 윈도우 함수(OVER, RANK, ROW_NUMBER)
SQL:2008	• 2008	• MERGE, TRUNCATE, FETCH FIRST n ROWS 등
SQL:2011	• 2011	• Temporal Table(시간 기반 이력 관리) 도입
SQL:2016	• 2016	• JSON 표준 함수 지원, Polymorphic Table Function 등
SQL:2023	• 2023	• 최신 판 (표준 JSON Schema, 더 많은 함수 확장 등)

[참고] ANSI SQL:2016

SQL:2016은 현대 DB의 방향인 JSON & 비정형 데이터 지원에 집중

【 SQL:2016 주요 추가 내용 】

기능	설명
JSON 지원	JSON_VALUE(), JSON_QUERY(), JSON_OBJECT() 등 표준화된 함수
Polymorphic Table Function (PTF)	윈도우 함수처럼 동적으로 테이블 구조 변형 가능
Row Pattern Recognition	시계열 데이터 패턴 탐지 (MATCH_RECOGNIZE 절)
LATERAL JOIN 개선	하위쿼리와 상호 참조 가능
Enhanced temporal features	시간 이력 관리 기능 강화

【 실제 DBMS 지원 정도 】

기능	PostgreSQL	MySQL	Oracle	SQL Server
JSON 함수 (SQL:2016 스타일)	O 부분 지원 (jsonb + 함수)	O 부분 지원 (JSON_EXTRACT, 비표준명)	O Oracle 21c부터 표준 함수 지원	O 부분적 (OPENJSON, 비표준명)
MATCH_RECOGNIZE	X	X	(Oracle 12c+)	X
Temporal Table	O (SYSTEM VERSIONING)	O (8.0.2+)	O	O
Merge	O	O	O	O

[참고] ANSI SQL vs 주요 DBMS 문법 비교

구문 / 기능	ANSI SQL (표준)	MySQL / MariaDB	PostgreSQL	Oracle	SQL Server
상위 N행 조회	FETCH FIRST n ROWS ONLY	LIMIT n	FETCH FIRST n ROWS ONLY (또는 LIMIT n)	FETCH FIRST n ROWS ONLY (12c+)	TOP n (예: SELECT TOP 10 *)
NULL 치환	COALESCE(expr1, expr2)	동일	동일	NVL(expr1, expr2) (비표준)	ISNULL(expr1, expr2) (비표준)
문자열 연결				CONCAT(a,b) 또는 `a	
현재 시각	CURRENT_TIMESTAMP	NOW() (동의어)	NOW() / CURRENT_TIMESTAMP	SYSTIMESTAMP	SYSUTCDATETIME() / GETDATE()
AUTO_INCREMENT	GENERATED ALWAYS AS IDENTITY	AUTO_INCREMENT	GENERATED ... AS IDENTITY 또는 SERIAL	IDENTITY (12c~) 또는 SEQUENCE	IDENTITY(1,1)
조건문	CASE WHEN ... THEN ... ELSE ... END	동일	동일	동일	동일
LIMIT + OFFSET	OFFSET n ROWS FETCH FIRST m ROWS ONLY	LIMIT m OFFSET n	둘 다 지원	(12c+)	OFFSET n ROWS FETCH NEXT m ROWS ONLY (2012+)

[참고] ANSI SQL vs 주요 DBMS 문법 비교

구문 / 기능	ANSI SQL (표준)	MySQL / MariaDB	PostgreSQL	Oracle	SQL Server
문자 길이	CHAR_LENGTH(str)	CHAR_LENGTH()	CHAR_LENGTH()	LENGTH()	LEN()
날짜 차이	DATEDIFF() 없음 → timestamp1 - timestamp2	DATEDIFF() (비표준)	AGE() 또는 뽀셈 연산	MONTHS_BETWEEN()	DATEDIFF() (비표준)
논리값(Boolean)	BOOLEAN	TINYINT(1)	진짜 BOOLEAN	없음 (NUMBER(1)로 대체)	BIT
문자 대소문자 변경	UPPER(), LOWER()	동일	동일	동일	동일
문자열 부분추출	SUBSTRING(str FROM pos FOR len)	SUBSTRING(str, pos, len)	SUBSTRING(str FROM pos FOR len)	SUBSTR(str, pos, len)	SUBSTRING(str, pos, len)
IFNULL / NVL / COALESCE	COALESCE() (표준)	IFNULL()	COALESCE()	NVL()	ISNULL()
MERGE (UPSERT)	MERGE INTO ... USING ...	INSERT ... ON DUPLICATE KEY UPDATE (비표준)	INSERT ... ON CONFLICT ... DO UPDATE	표준 MERGE	MERGE
트랜잭션 시작/종료	BEGIN, COMMIT, ROLLBACK	동일	동일	동일	동일

[참고] ANSI SQL vs 주요 DBMS 문법 비교

구문 / 기능	ANSI SQL (표준)	MySQL / MariaDB	PostgreSQL	Oracle	SQL Server
문자열 검색	LIKE '%abc%'	동일	동일	동일	동일
JSON 접근	JSON_VALUE() (표준)	JSON_EXTRACT()	attrs->>'key', jsonb_extract_path_text()	JSON_VALUE() (21c~ 표준 지원)	JSON_VALUE() / OPENJSON()
윈도우 함수	OVER(PARTITION BY ... ORDER BY ...)	동일	동일	동일	동일
RECURSIVE CTE	WITH RECURSIVE ...	동일	동일	(11gR2+)	(2005+)
DELETE ALL	DELETE FROM table_name;	동일	동일	동일	동일
TRUNCATE	TRUNCATE TABLE table_name;	동일	동일	동일	동일
TEMP TABLE	CREATE TEMPORARY TABLE	동일	동일	GLOBAL TEMPORARY TABLE	#temp (로컬), ##temp (전역)

[참고] ANSI SQL vs 주요 DBMS 문법 비교

구분	표준 준수	비표준 문법 예시
완전 표준	SELECT, JOIN, GROUP BY, HAVING, CASE, COALESCE, FETCH FIRST n ROWS ONLY	-
DB 전용 문법	MySQL: LIMIT, IFNULL / SQL Server: TOP, ISNULL / Oracle: NVL, ROWNUM / PostgreSQL: SERIAL	DB 이식 시 주의 필요
추천 전략	ANSI SQL 기반으로 작성하고, LIMIT/OFFSET, NULL 처리, 날짜함수, JSON 함수만 DB별로 분기	

- 이식성 중요한 프로젝트면 **ANSI SQL + 호환 함수로 통일**
- 보고서/배치 등 **DB 고정**이면 해당 **DB 고유함수** 써도 **OK**
- **ORM (예: JPA, SQLAlchemy)**는 내부적으로 이런 차이를 자동 흡수
- 쿼리 튜닝할 땐 **DB 전용 함수/힌트**를 써야 함 (성능용).

Oracle

특징

- 대기업 시장을 장악한 전통 강자
- **RAC (Real Application Clusters)**
: 여러 서버가 동시에 **DB**를 공유하여
고가용성/확장성 제공
- 고급 기능: 보안, 성능 최적화, 분산
트랜잭션, **Data Guard**(재해복구)

장점

- 금융, 공공기관, 글로벌 기업에서 신뢰도
검증 완료
- 매우 안정적이고 복잡한 트랜잭션에 적합

활용사례

- 은행 계좌 관리 시스템
- 글로벌 제조/**ERP** 시스템
- 대규모 엔터프라이즈 시스템

단점

- 비용이 매우 비쌈 (라이선스, 유지보수)
- 오픈소스/클라우드 대체제에 비해 폐쇄적

Microsoft SQL Server

특징

- **MS 생태계와 밀접하게 통합**
(.NET, Visual Studio, Windows Server와 최적화)
- **OLAP (Online Analytical Processing)**
: BI(비즈니스 인텔리전스), 데이터 분석 강점
- **SSIS/SSAS/SSRS** → 데이터 통합, 분석,
리포팅 기능 제공

장점

- **MS 환경(Windows, Active Directory, PowerBI)**과 완벽 연계
- 엔터프라이즈 환경에서 쉽고 빠른
분석/리포팅 가능

활용사례

- 기업 내 **ERP/CRM** 시스템
- 금융 리포트 시스템
- **BI + PowerBI** 기반 경영 데이터 분석

단점

- **Windows 종속적**
(최근엔 **Linux** 지원 확대 중)
- 오픈소스 대비 자유도가 떨어짐

Cloud Database (관리형 RDBMS)

AWS RDS

- **MySQL, PostgreSQL, Oracle, SQL Server** 등 다양한 엔진 제공
- 장점: 자동 백업, 장애 복구, 운영 편리성
- 단점: 직접 서버 제어 불가(루트 접근 제한)
- 활용: 스타트업 ~ 대기업 클라우드 전환 시 활용

GCP Cloud SQL

- **Google**의 관리형 DB, 자동 백업/확장
- 장점: **BigQuery, Dataflow**와 연계가 쉬움
- 활용: 데이터 분석, 머신러닝 파이프라인과 함께 사용

Amazon Aurora

- **MySQL/Postgres** 호환 + **AWS** 자체 최적화
- 성능: **MySQL** 대비 3~5배 빠름 (스토리지 분산 구조)
- 활용: 글로벌 서비스 확장, 빠른 읽기 성능 요구되는 환경

Azure Database

- **SQL Server** 기반, **Microsoft** 생태계와 긴밀히 연동
- 장점: **Active Directory, PowerBI, Office365**와 통합
- 활용: **MS** 기반 **ERP/CRM SaaS** 서비스

주요 포인트

MySQL /MariaDB

- 웹서비스 스타트업 기본 DB
- 빠르고 배우기 쉽다

PostgreSQL

- 최신 오픈소스 RDBMS의 대표주자
- JSON+GIS 처리 강력

Oracle

- 금융/대기업 시스템, 고성능과 안정성
- 단, 비용 부담 큼

SQL Server

- MS 기업 환경에서 최적화, BI/데이터 분석 강점

Cloud DB

- 관리형 DBMS, 운영 부담 줄이고 글로벌 확장에 최적

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

6. SQL 기초 — DDL

▪ CREATE DATABASE · CREATE TABLE · ALTER · 데이터 타입 · 제약조건

CREATE DATABASE — DBMS별 문법 비교

【 SQL 예시 】

-- MySQL / MariaDB

```
CREATE DATABASE skala_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_general_ci; -- 한국어/이모지 지원
```

-- PostgreSQL (권장)

```
CREATE DATABASE skala_db
  WITH ENCODING = 'UTF8'
  LC_COLLATE = 'en_US.UTF-8'
  LC_CTYPE = 'en_US.UTF-8'
  TEMPLATE = template0;
```

-- PostgreSQL 스키마 생성 및 검색 경로 설정

```
c skala_db
CREATE SCHEMA app;           -- 애플리케이션 스키마
CREATE SCHEMA audit;        -- 감사 로그 스키마
SET search_path = app, public; -- 기본 스키마 순서
```

-- SQL Server

```
CREATE DATABASE SkalaDB
  COLLATE Korean_100_CI_AS_SC_UTF8;
```

-- Oracle (DBA 수행, 사용자=스키마 개념)

```
CREATE USER skala_user IDENTIFIED BY "StrongPW1!"
  DEFAULT TABLESPACE users QUOTA UNLIMITED ON
  users;
GRANT CREATE SESSION, CREATE TABLE, CREATE
  SEQUENCE TO skala_user;
```

Checkpoint

- ✓ UTF-8 및 한국어 콜레이션 설정을 처음부터 올바르게
- ✓ 나중에 바꾸면 모든 데이터 재변환 필요

CREATE DATABASE: “컨테이너” 만들기

벤더마다 데이터베이스의 의미/단위가 조금씩 다름

MySQL / MariaDB

- 하나의 서버 안에 여러 **database(schema)**를 만들고 그 안에 테이블 생성
- 문자셋/콜레이션을 꼭 지정!

```
-- MySQL 8 / MariaDB 권장
CREATE DATABASE appdb
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_general_ci; -- 한국어/다국어 무난
```

PostgreSQL

- 서버(**cluster**) → 여러 **database** → 각 **database** 안에 여러 **schema**
- 인코딩/로케일을 함께 정할 것

```
CREATE DATABASE appdb
  WITH ENCODING 'UTF8'
  LC_COLLATE 'en_US.UTF-8'
  LC_CTYPE 'en_US.UTF-8'
  TEMPLATE template0;
```

CREATE DATABASE: “컨테이너” 만들기

벤더마다 데이터베이스의 의미/단위가 조금씩 다름

SQL Server

- 인스턴스 → 여러 **database** → 각 **database** 안에 **schema**(기본: **dbo**)
- 한국어/**UTF-8** 쓰려면 콜레이션을 지정

```
CREATE DATABASE AppDb  
COLLATE Korean_100_CI_AS_SC_UTF8;
```

Oracle

- “데이터베이스”는 인스턴스/스토리지 단위(관리자가 생성)
- 개발자는 보통 사용자=스키마를 만들어 씀

```
-- DBA가 수행(개발 학습용 예시)  
CREATE USER appuser IDENTIFIED BY "StrongPW1!"  
  DEFAULT TABLESPACE users QUOTA UNLIMITED ON users;  
GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE TO appuser;
```

CREATE TABLE: 테이블 설계의 표준 뼈대

상품(products) 테이블을 각 DBMS 문법/관례에 맞게 만든 예시

id(정수, 식별자) / name(문자) / price(금액: 소수 고정 자리수) / attrs(JSON 속성) / created_at(생성 시각)

**MySQL /
MariaDB
(InnoDB
전제)**

```
CREATE TABLE products (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(200) NOT NULL,
  price DECIMAL(10,2) NOT NULL CHECK (price >= 0),    -- MySQL 8.0.16+ / MariaDB 10.2+ 유효
  attrs JSON NULL,                                     -- MariaDB는 내부적으로 LONGTEXT + JSON 함수
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB
DEFAULT CHARSET = utf8mb4;
```

- **Tip: CHECK는 MySQL 8.0.16 미만에선 파싱만 하고 무시되던 역사 있음**
(버전 확인 필수)
- **MariaDB의 JSON은 물리적으로 LONGTEXT로 인덱싱 시 가상컬럼+인덱스 패턴 사용**

CREATE TABLE: 테이블 설계의 표준 뼈대

상품(products) 테이블을 각 DBMS 문법/관례에 맞게 만든 예시

id(정수, 식별자) / name(문자) / price(금액: 소수 고정 자리수) / attrs(JSON 속성) / created_at(생성 시각)

Postgre SQL

```
CREATE TABLE products (  
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, -- SERIAL 대신 최신 표준  
  name VARCHAR(200) NOT NULL,  
  price NUMERIC(10,2) NOT NULL CHECK (price >= 0),  
  attrs JSONB, -- 강력한 JSONB 연산/인덱스  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
-- JSONB 특정 키 인덱싱 예시  
CREATE INDEX idx_products_attrs_color  
  ON products ((attrs->>'color'));
```

CREATE TABLE: 테이블 설계의 표준 뼈대

상품(products) 테이블을 각 DBMS 문법/관례에 맞게 만든 예시

id(정수, 식별자) / name(문자) / price(금액: 소수 고정 자리수) / attrs(JSON 속성) / created_at(생성 시각)

SQL Server

```
CREATE TABLE dbo.Products (  
  Id BIGINT IDENTITY(1,1) PRIMARY KEY,  
  Name NVARCHAR(200) NOT NULL,  
  Price DECIMAL(10,2) NOT NULL  
    CONSTRAINT CK_Products_Price CHECK (Price >= 0),  
  Attrs NVARCHAR(MAX) NULL,    -- JSON 전용 타입 없음: NVARCHAR에 저장, JSON_VALUE/OPENJSON 사용  
  CreatedAt DATETIME2 NOT NULL  
    CONSTRAINT DF_Products_CreatedAt DEFAULT SYSUTCDATETIME()  
);
```

CREATE TABLE: 테이블 설계의 표준 뼈대

상품(products) 테이블을 각 DBMS 문법/관례에 맞게 만든 예시

id(정수, 식별자) / name(문자) / price(금액: 소수 고정 자리수) / attrs(JSON 속성) / created_at(생성 시각)

Oracle

```
CREATE TABLE products (
  id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY, -- 12c+ IDENTITY
  name VARCHAR2(200 CHAR) NOT NULL,                        -- CHAR semantics 권장
  price NUMBER(10,2) NOT NULL,
  attrs CLOB,                                                -- 12c~: IS JSON 제약, 21c~: JSON 타입
  created_at TIMESTAMP WITH TIME ZONE DEFAULT SYSTIMESTAMP NOT NULL,
  CONSTRAINT ck_products_price CHECK (price >= 0)
);

-- 12c 이상에서 JSON 유효성
ALTER TABLE products ADD CONSTRAINT ck_products_attrs_json
CHECK (attrs IS JSON);
```

데이터 타입 차이 핵심 정리

정수/식별자

- MySQL: **INT AUTO_INCREMENT**
- PostgreSQL: **GENERATED ... AS IDENTITY(표준) 또는 SERIAL**
- SQL Server: **BIGINT IDENTITY(1,1)**
- Oracle: **IDENTITY(12c+) 또는 SEQUENCE+TRIGGER(구버전)**

문자열

- MySQL/MariaDB: **VARCHAR(n), TEXT/MEDIUMTEXT/LONGTEXT**
- PostgreSQL: **VARCHAR(n)와 TEXT**
(사이즈 제한 없음, 실무에선 TEXT 많이 사용)
- SQL Server: **VARCHAR/NVARCHAR(유니코드),**
대용량은 **NVARCHAR(MAX)**
- Oracle: **VARCHAR2(n BYTE | CHAR), 대용량은 CLOB**

데이터 타입 차이 핵심 정리

이진

- MySQL/MariaDB: BLOB 계열
- PostgreSQL: BYTEA(일반적), 대용량은 Large Object API
- SQL Server: VARBINARY, 대용량은 VARBINARY(MAX)
- Oracle: BLOB

날짜/시간

- MySQL: DATETIME, TIMESTAMP(타임존 없음),
CURRENT_TIMESTAMP
- PostgreSQL: TIMESTAMP [WITH TIME ZONE]=TIMESTAMPTZ, now()
- SQL Server: DATETIME2 권장, GETDATE()(로컬),
SYSUTCDATETIME()(UTC)
- Oracle: TIMESTAMP [WITH TIME ZONE], SYSTIMESTAMP

데이터 타입 차이 핵심 정리

불리언

- PostgreSQL: **BOOLEAN**(진짜 불리언)
- SQL Server: **BIT**
- MySQL: **BOOLEAN**은 **TINYINT(1)**의 동의어
- Oracle: SQL 레벨에선 없음 → **NUMBER(1)+CHECK**로 대체

JSON

- MySQL 8+: **JSON**(네이티브)
- MariaDB: **JSON** 표기지만 실제로 **LONGTEXT** + **JSON** 함수/검증
- PostgreSQL: **JSONB**(인덱싱/연산 강력), **JSON**(원문 보존)
- SQL Server: 전용 타입 없음(**NVARCHAR** 저장) + **OPENJSON**, **JSON_VALUE**
- Oracle: 12c~ **IS JSON**로 유효성, 21c~ 전용 **JSON** 타입 도입

데이터 타입 차이 핵심 정리

ENUM /ARRAY

- **MySQL/MariaDB: ENUM** 있음(편하지만 확장성/이식성 주의)
- **PostgreSQL: ENUM, ARRAY** 모두 강력 지원
- **SQL Server/Oracle: 직접 지원 X(참조 테이블 + FK로 모델링 권장)**

```
-- 상태값을 따로 테이블로 만들
CREATE TABLE status_types (
  status_code VARCHAR2(20) PRIMARY KEY, -- Oracle
  description VARCHAR2(100)
);

INSERT INTO status_types VALUES ('ACTIVE', '활성 사용자');
INSERT INTO status_types VALUES ('BLOCKED', '차단됨');
INSERT INTO status_types VALUES ('PENDING', '승인 대기');

-- users 테이블이 status_code를 참조
CREATE TABLE users (
  id NUMBER PRIMARY KEY,
  email VARCHAR2(200) NOT NULL,
  status_code VARCHAR2(20) NOT NULL,
  CONSTRAINT fk_users_status FOREIGN KEY (status_code)
    REFERENCES status_types(status_code)
);
```

제약조건: 무결성의 4대장

구분	특징/추가옵션
NOT NULL	빈 값 금지
DEFAULT	- 값 없으면 이걸로 예) <code>DEFAULT CURRENT_TIMESTAMP, DEFAULT 0</code> - 함수 이름은 벤더별로 다름(예: SQL Server는 <code>GETDATE()/SYSUTCDATETIME()</code>).
CHECK	- 수학/논리식으로 값 제한 - MySQL 8.0.16+ / MariaDB 10.2+ / PostgreSQL / SQL Server / Oracle에서 사용 - 성능상 과도한 복잡식은 지양, 핵심 규칙만!
FOREIGN KEY(FK)	- 참조 무결성 - 자식 → 부모(PK/UNIQUE) 참조. 삭제/수정 동작을 <code>ON DELETE/ON UPDATE</code>로 정의 - PostgreSQL: <code>ON DELETE/UPDATE CASCADE/SET NULL/RESTRICT</code> 지원, <code>DEFERRABLE</code> 가능 - MySQL/MariaDB(InnoDB): <code>ON DELETE/UPDATE</code> 지원(버전/엔진 주의) - SQL Server: <code>ON DELETE/UPDATE</code> 지원, <code>DEFERRABLE</code> 없음 - Oracle: <code>ON DELETE</code>만(<code>ON UPDATE</code> 미지원), <code>DEFERRABLE</code> 가능 - 실무 팁: FK 컬럼에 인덱스를 고려(조인/삭제 성능) 일부 DB는 자동/강제 안 될 수 있음

이식성(Portable SQL) 팁

DBMS마다 타입 이름이 제각각이어서, 호환성 좋은 표준 타입을 쓰라는 뜻

타입은 표준/보편형으로
INTEGER, NUMERIC(p,s), VARCHAR(n), TIMESTAMP 중심

예약어/식별자 따옴표 지양
벤더별 따옴표가 다름(", ` , []) + 스네이크케이스 소문자 권장

DEFAULT의 함수명 통일 불가 → 마이그레이션 스크립트에서 분기

CHECK/ENUM 남용 X: 규칙이 바뀌면 스키마 변경 불가
많으면 **Lookup** 테이블 + **FK**로

JSON은 스키마-리스지만, 접근 경로가 자주 쓰이면 가상컬럼/인덱스로 최적화

자주 겪는 일

MySQL 구버전 CHECK 무시: 8.0.16 미만이면 동작 안 함

Oracle의 ON UPDATE FK 미지원: 모델 설계로 해결 (애플리케이션에서 업데이트 금지 등)

대용량 텍스트의 DEFAULT: MySQL TEXT/BLOB엔 DEFAULT 불가

**타임존 혼선: 서버/클라이언트 TZ가 다르면 TIMESTAMP WITH TIME ZONE(PG/Oracle)
또는 UTC 고정(SQL Server SYSUTCDATETIME) 전략**

문자셋/문자열을 어떻게 비교하고 정렬할 지를 프로젝트 초기에 통일

체크리스트(스키마 리뷰용)

PK는 단일/숫자/불변인가? (자연키 대신 대체키 권장)

FK에 필요한 인덱스가 있는가?

금액/수량은 DECIMAL/NUMERIC으로 스케일을 고정했는가?

시간 컬럼은 기본값/타임존 정책이 일관적인가?

JSON 컬럼은 꼭 필요한가? 자주 쓰는 키에 인덱스 계획이 있는가?

CHECK/ENUM이 변경 가능성을 과도하게 제한하진 않는가?

ALTER TABLE — 컬럼 추가/수정 및 운영 주의사항

【 SQL 예시 】

-- 컬럼 추가 (일반적으로 안전)

```
ALTER TABLE students ADD COLUMN phone VARCHAR(20);
```

```
ALTER TABLE students ADD COLUMN birth_date DATE;
```

-- 컬럼 타입 변경 (주의: 대용량 테이블에서는 전체 재작성 발생)

```
ALTER TABLE students ALTER COLUMN phone TYPE  
VARCHAR(30); -- PostgreSQL
```

-- NOT NULL 추가 (기존 NULL 없는 경우만)

```
ALTER TABLE students ALTER COLUMN phone SET NOT NULL;
```

-- 제약조건 추가

```
ALTER TABLE students  
ADD CONSTRAINT chk_email_format CHECK (email LIKE  
'%@%');
```

-- 컬럼 삭제 (신중히 — CASCADE: 의존 뷰/인덱스 함께 삭제)

```
ALTER TABLE students DROP COLUMN IF EXISTS phone  
CASCADE;
```

-- 인덱스 생성 (운영 중 잠금 없이)

```
CREATE INDEX CONCURRENTLY idx_students_grade ON  
students (grade);
```

-- **CONCURRENTLY**: PostgreSQL에서 테이블 잠금 없이
인덱스 생성 (운영 환경 권장)

-- **TRUNCATE** vs **DELETE**

```
TRUNCATE TABLE temp_data; -- 빠름, 자동 커밋, 트리거  
미실행
```

```
DELETE FROM temp_data; -- 느림, 롤백 가능, 트리거  
실행, WHERE 가능
```

Checkpoint

- ✓ 운영 중 DDL 변경(대용량 테이블)
 - pt-online-schema-change(MySQL)
 - pg_repack(PostgreSQL)

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

7. SQL 기초 — DML

▪ **SELECT · INSERT · UPDATE · DELETE · 함수 · CASE WHEN**

INSERT / UPDATE / DELETE 기본 패턴

【 SQL 예시 】

-- INSERT (단건)

```
INSERT INTO students (student_no, name, email, major_id,
grade)
VALUES ('2025001', '홍길동', 'hong@skala.ai', 1, 1);
```

-- INSERT (다건)

```
INSERT INTO courses (code, title, credits) VALUES
('CS101', '데이터베이스', 3),
('CS102', '알고리즘', 3),
('CS103', '운영체제', 3);
```

-- UPDATE (WHERE 필수! 없으면 전체 행 수정)

```
UPDATE students
SET grade = grade + 1,
updated_at = now()
WHERE enrolled = TRUE
AND grade < 4;
```

-- DELETE (WHERE 필수! 없으면 전체 행 삭제)

```
DELETE FROM enrollments
WHERE student_id IN (
SELECT id FROM students WHERE enrolled = FALSE
);
```

주의 사항

✓ MySQL

- 기본 autocommit ON → 즉시 커밋됨

✓ PostgreSQL

- BEGIN; 없이 DML → 자동 커밋

✓ 대용량 DELETE는 LIMIT + 루프로 나눠서
처리 권장

SELECT 기본 구조 및 논리적 실행 순서

【 SQL 예시 】

```
-- 논리적 실행 순서 (작성 순서와 다름!)
-- FROM → JOIN → WHERE → GROUP BY → HAVING
-- → SELECT → ORDER BY → LIMIT

SELECT
  s.student_no,
  s.name,
  m.name                AS major_name,
  UPPER(s.email)        AS email_upper,
  EXTRACT(YEAR FROM s.created_at) AS join_year,
  COALESCE(s.phone, '미등록')    AS phone_display,
  CASE s.grade
    WHEN 1 THEN '신입생'
    WHEN 4 THEN '졸업반'
    ELSE s.grade || '학년'
  END                    AS grade_label
FROM students s
  JOIN majors m ON m.id = s.major_id  -- 2. JOIN 처리
WHERE s.enrolled = TRUE              -- 3. 행 필터 (집계 전)
  AND s.grade BETWEEN 1 AND 4
ORDER BY s.grade DESC, s.name ASC    -- ORDER BY에
서만 SELECT 별칭 사용 가능
LIMIT 20 OFFSET 0;                  -- 페이지네이션
```

주의 사항

- ✓ **WHERE절에서 SELECT 절의 별칭(alias) 사용 불가**
- ✓ **SELECT는 WHERE보다 나중에 처리됨**

자주 쓰는 문자열 / 날짜 / NULL 처리 함수

【 SQL 예시 】

-- 문자열 함수

```
SELECT UPPER('hello'),      -- HELLO
       LOWER('WORLD'),     -- world
       LENGTH('abc'),      -- 3
       SUBSTRING('apple' FROM 1 FOR 3), -- app (표준 SQL)
       TRIM(' hi '),       -- hi
       REPLACE('hello world', 'world', 'DB'), -- hello DB
       CONCAT('SK', 'ALA', '4기'); -- SKALA4기
```

-- 날짜/시간 함수 (PostgreSQL)

```
SELECT CURRENT_DATE,      -- 오늘 날짜
       CURRENT_TIMESTAMP, -- 현재 일시 + 타임존
       EXTRACT(YEAR FROM now()), -- 연도 추출
       DATE_TRUNC('month', now()), -- 월의 첫날
       now() - INTERVAL '30 days', -- 30일 전
       TO_CHAR(now(), 'YYYY-MM-DD'); -- 포매팅
```

-- NULL 처리

```
SELECT COALESCE(NULL, NULL, '세 번째'), -- 세 번째 (첫 비-NULL)
       NULLIF(10, 10),                  -- NULL (같으면 NULL)
       NULLIF(10, 20),                  -- 10 (다르면 첫 번째)
       CASE WHEN val IS NULL THEN 0 ELSE val END;
```

주의 사항

- ✓ **COALESCE**
- SQL 표준 (모든 DBMS)
- ✓ **NVL** : Oracle 전용
- ✓ **ISNULL** : SQL Server 전용

WHERE 절 — 조건 패턴 및 NULL 주의사항

비교 연산자

- = <> != < > <= >=
- WHERE grade = 3 / WHERE grade <> 3 / WHERE score > 80

범위/목록 조건

- WHERE grade BETWEEN 2 AND 4 (inclusive, 2와 4 포함)
- WHERE major_id IN (1, 3, 5) / WHERE major_id NOT IN (2, 4)

LIKE 패턴 검색

- WHERE name LIKE '김%' → 인덱스 사용 가능 (prefix)
- WHERE name LIKE '%길%' → 인덱스 미사용! 대용량 시 전문검색 인덱스 필요

NULL 처리

(! = NULL X. IS NULL 사용)

- WHERE major_id IS NULL / WHERE major_id IS NOT NULL
- NULL = NULL → UNKNOWN (TRUE가 아님) → WHERE로 필터되지 않음
- NULL + 5 = NULL / COALESCE(NULL, 0) = 0

AND / OR / NOT 조합 시 우선순위

- NOT > AND > OR → 괄호로 명확하게 표현

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

8. 종합실습 - 1

- 학사관리시스템 DB 설계 → 구축 → 조회

종합 실습 - 방향

학습 목표

- PostgreSQL 환경 구성 및 연결
- 학사 관리 시스템 ERD 설계
- DDL 작성으로 테이블 생성
- DML로 샘플 데이터 입력 및 조회
- WHERE/함수/CASE WHEN 활용 쿼리 작성

실습 단계

- PostgreSQL 접속 확인 (psql 또는 DBeaver)
- CREATE DATABASE / CREATE SCHEMA 실행
- ERD 설계
- CREATE TABLE — 제약조건 포함 DDL 작성
- INSERT INTO (테이블 데이터는 최소 10건 이상)
- SELECT + WHERE + ORDER BY 기초 조회
- COALESCE / CASE WHEN / 날짜 함수 활용
- 수강신청 교차 테이블 JOIN 조회

과정 기간 내 활용 DB 구성
(추후 DB Docker 예정)

ERD 설계

DML/DDI 활용

함수 활용

종합 실습 - 환경 구성

PostgreSQL 설치 (터미널 기준)

Homebrew로 설치 (설치 되었다면 skip)

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

PostgreSQL 설치 (터미널 내 17버전)

```
brew install postgresql@17  
brew services start postgresql@17 -- 서비스 시작  
psql postgres -- psql 접속 (접속 후 postgres=# 확인)  
postgres=# ALTER USER (...) WITH PASSWORD '원하는비밀번호';  
-- DB 비번 지정 (필수!)  
postgres=# \q -- psql 접속 종료  
psql postgres -U (pc명 or 계정) -W (postgres 비번 통해서 접속)
```

* docker 내 DB 설치는 이후 과정에서 진행 예정

종합 실습 - 제출물

실습 제출물
(당일 제출 원칙)

ERD - 학사관리시스템 ERD

- ERD 내 범례 추가할 것 (설명문)
- ERD 표현 시 각 연결관계가 잘 보이도록 할 것 (선 겹침 금지)

리포트 - PDF 제출

- 내용
 - 학사관리시스템 요구사항 (설계 방향)
 - postgresSQL 접속 결과 화면
 - 문항별 실습 결과 (SQL문, 결과 화면 필수 - 항목 누락 당 감점)

스마트 데이터터의 이해 및 활용

9. Day 1 핵심 복습 & DBMS 생태계

10. SQL 중급 — 집계 & 그룹화

11. SQL 고급 — JOIN 알고리즘

12. JOIN 종류 상세

13. 서브쿼리 & 집합 연산자

14. CTE · View · Materialized View

15. Windows Function — 분석 함수

16. 종합 실습 — 2

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

9. Day 1 핵심 복습 & DBMS 생태계

- DDL/DML 정리 · DBMS vs DW vs Data Mining · MSA 연동

이전 내용 빠른 복습

DB 개요

- 파일 시스템의 한계 → DBMS 등장 / ACID 4원칙 (A·C·I·D)
- WAL(Write-Ahead Logging): 로그 먼저 → 데이터 파일 반영 → 장애 복구 보장
- Isolation Level: RC → RR → Serializable
- Dirty·Non-Repeatable·Phantom Read

관계형 모델

- 테이블·PK·FK·UK·관계 (1:1 / 1:N / N:M)
- 정규화 1NF ~ BCNF
 - 부분종속(2NF), 이행종속(3NF), 비후보키 결정자(BCNF)

ERD

- Entity·Attribute·Relationship / IE 표기법(까마귀발) / 개념→논리→물리 3단계

이전 내용 빠른 복습

DDL

- CREATE TABLE·제약조건·데이터 타입 / ALTER TABLE / DBMS별 차이
- PK: GENERATED ALWAYS AS IDENTITY (PG) / AUTO_INCREMENT (MySQL)

DML

- SELECT 실행 순서(FROM→WHERE→GROUP→HAVING→SELECT→ORDER) / INSERT·UPDATE·DELETE
- NULL 주의: = NULL(×) → IS NULL(○) / COALESCE() / CASE WHEN

DBMS vs Data Warehouse vs Data Mining

구분	DBMS	Data Warehouse	Data Mining
목적	일상 트랜잭션 처리(OLTP)	분석, 의사결정 지원(OLAP)	숨겨진 패턴, 지식 발견
데이터 특성	현재 데이터, 정규화, 실시간	이력 데이터, 비정규화, 주기적 적재	대용량 데이터, 통계/ML모델
쿼리 유형	단순 CRUD, 짧은 트랜잭션	복잡한 집계, 다차원 분석	분류, 군집, 연관, 예측 분석
예시 시스템	MySQL, PostgreSQL, Oracle	BigQuery, Snowflake, Redshift	Python ML, Spark MLlib
응답시간	ms ~ 초 단위	초~분 단위	분~시간 단위
갱신방식	실시간 INSERT, UPDATE, DELETE	배치 ETL(야간 주기적 적재)	배치/온라인 모델 학습

Kafka 연동 패턴	• 목적: 비동기 메시징, 이벤트 기반 아키텍처, 서비스 간 decoupling • DB 변경사항(Event Sourcing)을 Kafka Topic으로 전파 → 다른 마이크로서비스 구독 • 주의! - Kafka 메시지 전송 전 DB COMMIT 완료/Kafka Transaction 기능으로 atomic 처리 • Schema Registry : Producer/Consumer 간 스키마(Avro/Protobuf) 중앙 관리 → 호환성 보장
Elasticsearch 연동	• 목적: 빠른 검색, 필터링, 로그 분석, 비정형 텍스트 인덱싱 • DB → Logstash/Debezium → Elasticsearch 동기화 (CDC 패턴) • 주의! - ES는 Eventually Consistent → 실시간 정합성 보장 불가, RDBMS와 역할 분리 필요
Redis 연동	• 목적: 캐시(Cache-Aside/Write-Through), 세션 저장, Pub/Sub, Rate Limiting • 캐시 히트율 목표 90%+ / Cache Stampede 방지 (Lock 또는 Jitter) • 데이터 유실 주의: persistence 설정 (RDB/AOF) 확인 필수

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

10. SQL 중급 — 집계 & 그룹화

- 집계함수, GROUP BY, HAVING, ROLLUP, CUBE, FILTER

집계 함수 (Aggregate Functions) 핵심 5가지

COUNT()

- 행의 개수를 반환
- COUNT(*) : NULL 포함한 전체
- COUNT(col) : NULL 제외
- COUNT(DISTINCT col) : 고유값 개수

SUM() AVG()

- 합계, 평균
- NULL은 자동 제외
- AVG(NULL) = NULL (0은 아니라는 것!!!)
- 부서별 평균 급여, 카테고리별 총 매출

집계 함수 (Aggregate Functions) 핵심 5가지

MIN()
MAX()

- 최대값, 최소값
- 문자열에도 사용 가능
(알파벳, 유니코드 순)

FILTER
(PostgreSQL)

- 조건부 집계 (PostgreSQL 전용)
- COUNT(*) FILTER (WHERE score >=80)
- MySQL/Oracle
 - SUM(CASE WHEN score >=80 THEN 1 ELSE 0 END)

GROUP BY / HAVING 기본 패턴

【 SQL 예시 】

```
-- GROUP BY: 그룹별 집계
-- SELECT 절에는 그룹 키 또는 집계함수만 사용 가능
SELECT department_id,
COUNT(*) AS emp_count,
ROUND(AVG(salary), 0) AS avg_salary,
MAX(salary) AS max_salary,
MIN(salary) AS min_salary
FROM employees
WHERE hire_date >= '2020-01-01' -- 집계 전 행 필터 (WHERE)
GROUP BY department_id
HAVING COUNT(*) >= 3 -- 집계 후 그룹 필터 (HAVING)
AND AVG(salary) >= 5000
ORDER BY avg_salary DESC;

-- COUNT DISTINCT: 중복 제거 후 개수
SELECT COUNT(DISTINCT customer_id) AS unique_buyers
FROM orders WHERE amount >= 100;

-- PostgreSQL FILTER 절 (다른 DBMS: CASE WHEN으로 대체)
SELECT
COUNT(*) AS total,
COUNT(*) FILTER (WHERE true_label = pred_label) AS correct,
ROUND(COUNT(*) FILTER (WHERE true_label = pred_label)::NUMERIC /
COUNT(*), 3) AS accuracy
FROM prediction_results;
```

Checkpoint

- ✓ **WHERE** = 집계 전 행 필터
- ✓ **HAVING** = 집계 후 그룹 필터

ROLLUP / CUBE – 다차원 소계 집계

【 SQL 예시 】

-- 샘플 테이블

```
CREATE TABLE sales_summary (region TEXT, product TEXT, amount INT);
INSERT INTO sales_summary VALUES
('East','A',100),('East','B',150),('West','A',200),('West','B',50);
```

-- **ROLLUP**: 계층적 소계 (지역별 → 전체합)

```
SELECT region, product, SUM(amount) AS total
FROM sales_summary
GROUP BY ROLLUP(region, product);
```

-- 결과: (East,A,100),(East,B,150),(East,NULL,250),
-- (West,A,200),(West,B,50),(West,NULL,250),(NULL,NULL,500)

-- **CUBE**: 모든 조합 소계 (지역별 + 상품별 + 전체)

```
SELECT region, product, SUM(amount) AS total
FROM sales_summary
GROUP BY CUBE(region, product);
```

-- 결과: ROLLUP 결과 + (NULL,A,300),(NULL,B,200) 추가

-- **GROUPING** 함수: NULL이 집계용인지 실제 NULL인지 구분

```
SELECT CASE WHEN GROUPING(region)=1 THEN '전체' ELSE region END AS region_label,
CASE WHEN GROUPING(product)=1 THEN '소계' ELSE product END AS product_label,
SUM(amount) AS total
FROM sales_summary
GROUP BY ROLLUP(region, product);
```

Checkpoint

- ✓ **ROLLUP**
: 계층적 요약(빠름)
- ✓ **CUBE**
: 모든 조합(느림)
- ✓ **NULL**
= 해당 차원의 총합

ROLLUP / CUBE DBMS별 지원 현황

DBMS	ROLLUP	CUBE	GROUPING SETS	특이사항
PostgreSQL	O	O	O	GROUPING() 함수 제공 SQL 표준에 가장 근접
MySQL MariaDB	O(5.7+)	△ (일부)	O (8.0+)	구버전은 CUBE 미지원 버전 확인 필수
Oracle	O	O	O	GROUPING_ID() 추가 제공 계층 집계가 강력한 DB
SQL Server	O	O	O	GROUPING() , GROUPING_ID() 제공

집계 쿼리 성능 최적화 포인트

SELECT에 GROUP BY 키 이외 컬럼 포함 금지 (MySQL ONLY_FULL_GROUP_BY)

위반 시 Error 또는 임의값 반환 : GROUP BY 또는 집계 함수 명시

OFFSET 기반 페이지네이션의 함정

LIMIT 10 OFFSET 990 → DB가 990개를 읽고 버린 후 10개 반환 → O(N)

해결: Cursor/Keyset 방식 — WHERE id > :last_id ORDER BY id LIMIT 10

ORDER BY + 인덱스 활용

인덱스 컬럼을 ORDER BY 기준으로 사용 → Sort 노드 제거 → 성능↑

INDEX(dept_id, salary DESC) → GROUP BY dept_id ORDER BY salary DESC에 최적

집계 쿼리 성능 최적화 포인트

통계 테이블 전략 (대용량 집계)

자주 쓰는 집계값을 별도 테이블에 미리 계산·저장 (Materialized 집계)

예: `user_stats(user_id, total_orders, last_order_date)` — 트리거로 갱신

또는 Materialized View + 주기적 REFRESH

AI 서비스 연계

임베딩 클러스터별 평균 유사도, 태그 빈도 (GROUP BY로 분석 후 시각화)

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

11. SQL 고급 — JOIN 알고리즘

- NL·Hash·Merge·BNL/BKA · DBMS별 엔진 비교

JOIN 알고리즘

Nested Loop Join (NLJ)

- 외부 테이블 행마다 내부 테이블 탐색
- 인덱스가 있으면 빠름
- 외부 집합이 작을 때 / 내부 인덱스 있을 때 적합

Sort-Merge Join (SMJ)

- 양쪽을 정렬 후 병합하여 매칭
- 이미 정렬된 데이터면 유리
- 범위/비등가 조인도 가능
- 이미 정렬된 대용량 / 스트리밍 데이터 적합

Hash Join (HJ)

- 작은쪽(해시테이블), 큰쪽(프로브탐색)
- 동등 조인에 강력
- 메모리 부족 시 디스크 활용
- 대용량 등가 조인 / 인덱스 없을 때 적합

BNL/BKA (MySQL)

- Block Nested Loop
: 여러 행을 버퍼에 모아 내부 테이블 탐색
- Batched Key Access
: 모은 키를 배치로 인덱스 조회
- join_buffer_size로 튜닝

DBMS별 JOIN 엔진 비교

DBMS	지원 알고리즘	기본 전략	특이사항
PostgreSQL	NLJ·Hash·Merge + 병렬 Hash	통계 기반 옵티마이저 자동 선택	<code>enable_hashjoin/mergejoin/nestloop</code> 스위치
MySQL 8.0+	NLJ·BNL·BKA + Hash(8.0.18+)	인덱스 있으면 NLJ, 없으면 BNL	<code>optimizer_switch</code> 로 BNL/BKA/Hash 토글
SQL Server	NLJ·Hash·Merge + Adaptive Join	Adaptive Join: 실행 중 NL↔Hash 자동 전환	통계 부정확 시 <code>tempdb Spill</code> 주의
Oracle	NLJ·Sort-Merge·Hash	힌트: <code>USE_NL</code> / <code>USE_HASH</code> / <code>USE_MERGE</code>	Adaptive Plan/Stats Feedback

실행계획으로 JOIN 알고리즘 확인

【 SQL 예시 】

-- PostgreSQL: EXPLAIN ANALYZE로 실제 조인 알고리즘 확인

EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)

SELECT s.name, c.title, e.score

FROM students s

JOIN enrollments e ON e.student_id = s.id

JOIN courses c ON c.id = e.course_id

WHERE s.grade = 3;

-- 결과 해석:

-- Hash Join (cost=12.5..45.8 rows=200 width=32)

-- (actual time=0.5..2.3 rows=185 loops=1)

-- Hash Cond: (e.student_id = s.id)

-- Buffers: shared hit=12 read=5 ← 디스크 읽기 횟수

-- -> Seq Scan on enrollments ← Full Scan (인덱스 필요?)

-- -> Hash

-- -> Index Scan on students ← 인덱스 사용

-- Index Cond: (grade = 3)

-- MySQL 8.0.18+: EXPLAIN ANALYZE로 Hash Join 확인

EXPLAIN ANALYZE

SELECT * FROM customers c JOIN orders o ON c.id = o.customer_id;

-- "-> Inner hash join (o.customer_id = c.id)"

-- Hash Batches > 1 시 → work_mem 증가 검토 (메모리 Spill 발생)

Checkpoint

- ✓ 실행계획 핵심 체크
: 조인 알고리즘 타입
- ✓ actual rows vs
rows(예측) / Buffers
/ Spill 여부

JOIN 성능 튜닝 실전 체크리스트

공통 원칙

- FK 컬럼에 반드시 인덱스! (ON DELETE CASCADE 대상 컬럼 포함)
- 작은 테이블 × 큰 테이블 + 인덱스 → Nested Loop
- 대형 × 대형 → Hash
- ON 조건 누락 주의 → 카르테시안 곱(N×M행) 발생

PostgreSQL 튜닝

- 큰 동등 조인 → work_mem 늘리기 / hash_mem_multiplier 확인
- enable_parallel_hash=on, max_parallel_workers_per_gather 상향
- Hash Batches > 1 → Spill 발생 → work_mem 부족 신호

JOIN 성능 튜닝 실전 체크리스트

공통 원칙

- FK 컬럼에 반드시 인덱스! (ON DELETE CASCADE 대상 컬럼 포함)
- 작은 테이블 × 큰 테이블 + 인덱스 → Nested Loop
- 대형 × 대형 → Hash
- ON 조건 누락 주의 → 카르테시안 곱($N \times M$ 행) 발생

MySQL 튜닝

- 인덱스 부재 조인 → BNL/BKA → join_buffer_size 적절히 설정
- 8.0.18+ → Hash Join 후보 → EXPLAIN ANALYZE로 확인
- 과대 설정 주의: 스레드 × 조인 수만큼 메모리 할당됨

JOIN 성능 튜닝 실전 체크리스트

공통 원칙

- FK 컬럼에 반드시 인덱스! (ON DELETE CASCADE 대상 컬럼 포함)
- 작은 테이블 × 큰 테이블 + 인덱스 → Nested Loop
- 대형 × 대형 → Hash
- ON 조건 누락 주의 → 카르테시안 곱($N \times M$ 행) 발생

SQL Server 튜닝

- tempdb Spill (Hash/Sort Warning) 감지 → 통계 갱신, MAXDOP 조정
- Adaptive Join 활용 (2017+): 런타임에 NL ↔ Hash 자동 전환

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

12. JOIN 종류 상세

▪ INNER · OUTER · SELF · CROSS · Anti-Join · Semi-Join

JOIN 개념 및 종류 개요

JOIN

- 두 테이블의 카르테시안 곱 (Cartesian Product) + 조건 필터링
- **A JOIN B ON 조건**
→ 내부적으로 $A \times B$ 후 조건을 만족하는 행만 반환

INNER JOIN

- 두 테이블에 모두 매칭되는 행만 반환 (교집합)
- 가장 일반적인 조인
- **JOIN만 쓰면 INNER JOIN**

LEFT OUTER JOIN

- 왼쪽 테이블 전체 + 오른쪽 없으면 **NULL**
- "모든 A를 보여주고, B가 없으면 **NULL**로" → 미수강 학생도 표시

JOIN 개념 및 종류 개요

JOIN

- 두 테이블의 카르테시안 곱 (Cartesian Product) + 조건 필터링
- **A JOIN B ON 조건**
→ 내부적으로 **A × B** 후 조건을 만족하는 행만 반환

RIGHT OUTER JOIN

- 오른쪽 테이블 전체 + 왼쪽 없으면 **NULL**
- **LEFT JOIN**으로 방향 바꿔 쓰는 것이 실무 관례 (가독성)

FULL OUTER JOIN

- 양쪽 모두 전체 보존
- MySQL 미지원 → **LEFT JOIN UNION RIGHT JOIN**으로 우회

JOIN 개념 및 종류 개요

JOIN

- 두 테이블의 카르테시안 곱 (Cartesian Product) + 조건 필터링
- **A JOIN B ON 조건**
→ 내부적으로 **A × B** 후 조건을 만족하는 행만 반환

SELF JOIN

- 같은 테이블을 2개처럼 조인
→ 계층형 데이터 (직원-상사)

CROSS JOIN

- 카르테시안 곱 (**N × M**행)
→ 의도적으로만 사용, 실수 주의

INNER / LEFT / RIGHT / FULL OUTER JOIN 비교

【 SQL 예시 】

-- 샘플: student(id,name), enroll(student_id,course,grade)

-- **INNER JOIN:** 수강한 학생만

SELECT s.name, e.course, e.grade

FROM student s INNER JOIN enroll e ON s.id = e.student_id;

-- → 양쪽에 공통으로 있는 행만

-- **LEFT OUTER JOIN:** 모든 학생 (수강 없으면 NULL)

SELECT s.name, e.course, e.grade

FROM student s LEFT JOIN enroll e ON s.id = e.student_id;

-- → 수강 없는 학생도 포함 (course, grade = NULL)

-- **RIGHT OUTER JOIN:** 모든 수강 기록 (학생 없으면 NULL)

SELECT s.name, e.course

FROM student s RIGHT JOIN enroll e ON s.id = e.student_id;

-- → 고아 수강(학생 없는 enroll)도 포함

-- **FULL OUTER JOIN (MySQL 미지원 → UNION 우회)**

SELECT s.name, e.course FROM student s LEFT JOIN enroll e ON s.id = e.student_id
UNION

SELECT s.name, e.course FROM student s RIGHT JOIN enroll e ON s.id = e.student_id;

-- PostgreSQL/Oracle/SQL Server: FULL OUTER JOIN 직접 지원

Checkpoint

- ✓ 실행계획 핵심 체크
: 조인 알고리즘 타입
- ✓ actual rows vs
rows(예측) / Buffers
/ Spill 여부

SELF JOIN / CROSS JOIN / Anti-Join 패턴

【 SQL 예시 】

-- SELF JOIN: 직원-상사 계층 조회

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON m.id = e.manager_id
ORDER BY m.name NULLS FIRST, e.name;
```

-- Anti-Join: 한 번도 수강하지 않은 학생 (3가지 방법)

-- 방법 1: LEFT JOIN + IS NULL (직관적)

```
SELECT s.name FROM student s
LEFT JOIN enroll e ON e.student_id = s.id
WHERE e.student_id IS NULL;
```

-- 방법 2: NOT EXISTS (성능 우수, NULL 안전 - 권장)

```
SELECT s.name FROM student s
WHERE NOT EXISTS (SELECT 1 FROM enroll e WHERE e.student_id = s.id);
```

-- 방법 3: NOT IN (NULL 함정 주의!)

-- 서브쿼리에 NULL이 있으면 전체 결과가 빈 집합!

```
SELECT s.name FROM student s WHERE s.id NOT IN (SELECT student_id FROM enroll);
```

-- CROSS JOIN: 전체 학생 × 전체 강좌 조합 (추천 후보 생성)

```
SELECT s.id AS student_id, c.id AS course_id
FROM students s CROSS JOIN courses c
LIMIT 100; -- 실수 방지: 반드시 LIMIT 또는 의도 확인
```

Checkpoint

- ✓ **NOT IN vs NOT EXISTS**
: NULL 안전성·성능
모두 NOT EXISTS가
우수
- ✓ **NOT EXISTS 선호**

실무 집계 JOIN — 고객별 주문 통계

【 SQL 예시 】

-- 고객별 주문건수·총액 (LEFT JOIN: 주문 없는 고객도 포함)

```
SELECT
  c.customer_name,
  COUNT(o.order_id) AS order_count,
  COALESCE(SUM(o.amount),0) AS total_spent,
  MAX(o.order_date) AS last_order_date
FROM customers c
  LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_spent DESC;
```

-- 총액 상위 10명 (서브쿼리 또는 CTE 활용)

```
WITH customer_stats AS (
  SELECT c.customer_name,
         COUNT(o.order_id) AS cnt, SUM(o.amount) AS total
  FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
  GROUP BY c.customer_id, c.customer_name
)
SELECT customer_name, cnt, total
FROM customer_stats
ORDER BY total DESC
LIMIT 10;
```

-- 다중 테이블 JOIN (학생-수강-강좌-교수)

```
SELECT s.name, c.title, p.name AS professor, e.score
FROM students s
  JOIN enrollments e ON e.student_id = s.id
  JOIN courses c ON c.id = e.course_id
  JOIN professors p ON p.id = c.professor_id
WHERE e.score >= 80
ORDER BY e.score DESC;
```

JOIN 주의사항 및 성능 팁

항목	문제	해결책
ON 조건 누락	CROSS JOIN 발생 (N×M 행 폭발)	반드시 ON 조건 명시, EXPLAIN으로 확인
OUTER JOIN 후 WHERE 필터	사실상 INNER JOIN으로 변환됨	WHERE b.col IS NULL은 Anti-Join 패턴으로 의도적 사용
중복 행 폭발	1:N 조인 시 N개만큼 행 증가	서브쿼리에서 집계 후 조인, 또는 DISTINCT
NULL과 조인	FK가 NULL이면 INNER JOIN에서 제외	의도적 포함: LEFT JOIN + COALESCE
FK 인덱스 미생성	JOIN시 Full Scan 발생	FK 컬럼에 반드시 INDEX 생성
대용량 CROSS JOIN	메모리/시간 폭발	LIMIT 또는 LATERAL로 제한

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

13. 서브쿼리 & 집합 연산자

- 스칼라·인라인뷰·상관 서브쿼리 · EXISTS·IN·ANY·ALL · UNION·INTERSECT·EXCEPT

서브쿼리 유형

스칼라 서브쿼리 (SELECT 절)

- 단일 값 하나만 반환
- SELECT name,
(SELECT COUNT(*)
FROM orders o
WHERE o.cust_id=c.id)
AS order_count
FROM customers c
- 행마다 실행 → 성능 주의
- JOIN으로 대체 검토

인라인 뷰 (FROM 절)

- 임시 테이블처럼 사용
- SELECT *
FROM (
SELECT *,
ROW_NUMBER() OVER(...)
AS rn
FROM orders
) sub
WHERE rn <= 5
- 페이지네이션에 자주 사용

서브쿼리 유형

WHERE 절 서브쿼리

- 비교 조건으로 사용
- WHERE salary > (
 SELECT AVG(salary)
 FROM employees
)
- 단일값: =(같다)
- 다중행: IN/ANY/ALL/EXISTS
- 소량 데이터에 유리

상관 서브쿼리 (Correlated)

- 바깥 쿼리의 컬럼 참조 → 행마다 독립 실행
- WHERE salary > (
 SELECT AVG(salary)
 FROM employees
 WHERE dept_id=e.dept_id
 -- ↑ 바깥 쿼리 참조!
)
- JOIN/CTE로 대체 권장

EXISTS / IN / ANY / ALL 비교 및 선택 기준

【 SQL 예시 】

-- IN: 목록 또는 서브쿼리 결과에 포함 여부

```
SELECT name FROM students
WHERE major_id IN (SELECT id FROM majors WHERE region = 'Seoul');
```

-- EXISTS: 서브쿼리에 행이 하나라도 존재하면 TRUE (더 효율적)

```
SELECT name FROM students s
WHERE EXISTS (SELECT 1 FROM enrollments e WHERE e.student_id = s.id);
```

-- NOT EXISTS (권장): NOT IN보다 안전하고 성능 우수

```
SELECT name FROM students s
WHERE NOT EXISTS (SELECT 1 FROM enrollments e WHERE e.student_id = s.id);
```

-- NOT IN의 NULL 함정

-- 서브쿼리에 NULL 하나라도 있으면 전체 결과가 빈 집합!

```
WHERE id NOT IN (SELECT student_id FROM enrollments)
```

-- enrollments.student_id에 NULL이 있으면 항상 FALSE → 빈 결과

-- ANY: 하나 이상 조건 만족 (= > MIN과 동일)

```
WHERE salary > ANY (SELECT salary FROM employees WHERE dept = 'HR')
```

-- ALL: 모든 값에 대해 조건 만족 (= > MAX와 동일)

```
WHERE salary > ALL (SELECT salary FROM employees WHERE dept = 'HR')
```

Checkpoint

- ✓ **EXISTS vs IN**
: EXISTS는 행 존재만 확인
(SELECT 1)
→ 대용량에서 IN보다 빠름
- ✓ **NOT IN은 NULL 위험**

집합 연산자 — UNION / INTERSECT / EXCEPT

【 SQL 예시 】

```
-- UNION: 합집합 (중복 제거)
SELECT name FROM students_2024
UNION
SELECT name FROM students_2025; -- 같은 이름은 1번만 출력

-- UNION ALL: 합집합 (중복 포함, 성능 빠름 - 중복 불필요 시 권장)
SELECT 'sale' AS type, product_id, amount FROM sales
UNION ALL
SELECT 'refund'      , product_id, amount FROM refunds;

-- INTERSECT: 교집합 (두 결과에 공통으로 있는 행)
SELECT customer_id FROM orders WHERE EXTRACT(YEAR FROM order_date)=2024
INTERSECT
SELECT customer_id FROM orders WHERE EXTRACT(YEAR FROM order_date)=2025;
-- → 2024~2025년 모두 구매한 고객

-- EXCEPT (Oracle: MINUS): 차집합
SELECT student_id FROM enrollments WHERE course='DB'
EXCEPT
SELECT student_id FROM enrollments WHERE course='Algorithm';
-- → DB 수강 but 알고리즘 미수강 학생

-- MySQL INTERSECT/EXCEPT 미지원 → LEFT JOIN + IS NULL 우회
SELECT name FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_name = c.name);
```

Checkpoint

- ✓ UNION ALL이 UNION보다 빠름 (정렬/비교 불필요)
- ✓ 중복이 없거나 상관없을 때 UNION ALL 사용

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

14. CTE · View · Materialized View

- WITH · 재귀 CTE · 뷰 활용 전략

CTE (Common Table Expression) — WITH 절

【 SQL 예시 】

```
-- 기본 CTE: 복잡한 쿼리를 단계별로 분해
WITH dept_avg AS (
  SELECT dept_id, AVG(salary) AS avg_sal
  FROM employees
  GROUP BY dept_id
)
SELECT e.emp_name, e.salary, d.avg_sal,
       e.salary - d.avg_sal AS diff
FROM employees e
JOIN dept_avg d ON d.dept_id = e.dept_id
WHERE e.salary > d.avg_sal
ORDER BY diff DESC;
```

Checkpoint

- ✓ **CTE 장점**
 - : 가독성 향상
 - : 재사용 가능
 - : 재귀 쿼리 지원
- ✓ **서브쿼리가 복잡하면 CTE로 분해**

CTE (Common Table Expression) — WITH 절

【 SQL 예시 】

```
-- 다중 CTE (여러 단계 분리)
WITH
-- 1단계: 학과별 평균 성적
major_avg AS (
  SELECT s.major_id, AVG(e.score) AS avg_score
  FROM students s JOIN enrollments e ON e.student_id = s.id
  GROUP BY s.major_id
),
-- 2단계: 학과 평균보다 높은 학생 필터
top_students AS (
  SELECT s.id, s.name, s.major_id, AVG(e.score) AS my_avg
  FROM students s JOIN enrollments e ON e.student_id = s.id
  GROUP BY s.id, s.name, s.major_id
  HAVING AVG(e.score) > (SELECT avg_score FROM major_avg m WHERE
m.major_id = s.major_id)
)
SELECT t.name, m.avg_score AS major_avg, t.my_avg
FROM top_students t JOIN major_avg m ON m.major_id = t.major_id
ORDER BY t.my_avg DESC;
```

Checkpoint

- ✓ **CTE 장점**
 - : 가독성 향상
 - : 재사용 가능
 - : 재귀 쿼리 지원
- ✓ **서브쿼리가 복잡하면 CTE로 분해**

재귀 CTE — 계층형 데이터 탐색

【 SQL 예시 】

```
-- 조직도: 최상위 관리자부터 하위 직원까지 전체 탐색
WITH RECURSIVE org_tree AS (
  -- 기저 단계(Anchor): 최상위 관리자 (manager_id IS NULL)
  SELECT id, emp_name, manager_id,
         1 AS depth,
         emp_name::TEXT AS path
  FROM employees
  WHERE manager_id IS NULL

  UNION ALL

  -- 재귀 단계: 하위 직원 추가
  SELECT e.id, e.emp_name, e.manager_id,
         t.depth + 1,
         t.path || ' > ' || e.emp_name
  FROM employees e
  JOIN org_tree t ON t.id = e.manager_id
  WHERE t.depth < 10 -- ← 무한 재귀 방지! 항상 필요
)
SELECT depth,
       REPEAT(' ', depth - 1) || emp_name AS indented_name,
       path
FROM org_tree
ORDER BY path;
```

Checkpoint

- ✓ 카테고리 트리 탐색
(e-commerce 카테고리 계층)
- ✓ 같은 패턴
: 루트 카테고리 → 하위 카테고리 재귀 탐색
- ✓ 재귀 CTE 활용
: 조직도, 카테고리 트리, 경로 탐색 등

View / Materialized View 생성 및 활용

【 SQL 예시 】

```
-- 일반 View: 조회할 때마다 쿼리 실행 (항상 최신 데이터)
CREATE OR REPLACE VIEW v_student_dashboard AS
SELECT s.id, s.student_no, s.name, m.name AS major_name,
       COUNT(e.id) AS course_count,
       ROUND(AVG(e.score), 2) AS avg_score,
       MAX(e.score) AS best_score,
       SUM(c.credits) AS total_credits
FROM students s
LEFT JOIN majors m ON m.id = s.major_id
LEFT JOIN enrollments e ON e.student_id = s.id
LEFT JOIN courses c ON c.id = e.course_id
GROUP BY s.id, s.student_no, s.name, m.name;

-- View 조회 (일반 테이블처럼 사용)
SELECT * FROM v_student_dashboard WHERE avg_score >= 85;

-- Materialized View (PostgreSQL): 결과를 디스크에 저장 → 빠른 조회
CREATE MATERIALIZED VIEW mv_monthly_sales AS
SELECT DATE_TRUNC('month', order_date) AS month,
       SUM(amount) AS total_sales
FROM orders
GROUP BY 1;

CREATE INDEX ON mv_monthly_sales (month); -- 인덱스 생성 가능!
```

Checkpoint

- ✓ 데이터 갱신
(자동 아님 → 스케줄러 또는
pg_cron 필요)
: REFRESH MATERIALIZED
VIEW CONCURRENTLY
mv_monthly_sales;
-- 잠금 없이 갱신
- ✓ View 활용
: 권한 제어(민감 컬럼 숨기기) /
복잡한 쿼리 재사용 /
인터페이스 안정화

View vs Materialized View vs CTE 비교

구분	View	Materialized View	CTE
데이터 저장	저장 안 함	결과를 디스크에 저장	저장 안 함(쿼리 내 임시)
데이터 최신성	항상 최신	Refresh 시점까지만	쿼리 실행 시점
인덱스 생성	불가	가능(강력한 장점)	불가
재사용	여러 쿼리에서 재사용 가능	여러 쿼리에서 재사용 가능	해당 쿼리에서만 사용
사용예시	권한 제어, 인터페이스	대시보드, 집계 캐시 정보	복잡한 쿼리 분해
자동 갱신	자동(쿼리 실행할 때마다)	수동 Refresh 필요	자동(쿼리 실행시 마다 반영)

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

15. Windows Function – 분석 함수

- ROW_NUMBER
- RANK
- DENSE_RANK
- NTILE
- LAG
- LEAD
- 누적합
- 이동평균

Window Function — GROUP BY와 핵심 차이

【 GROUP BY 】

- 그룹으로 집계
→ 원래 행 수가 줄어듦
(개별 행 정보 소실)

【 Window Function 】

- 집계하면서도 원래 행을 유지
→ 행별 결과 + 집계값
동시 표현

기본 문법

- 함수명() OVER ([PARTITION BY 컬럼] [ORDER BY 컬럼] [ROWS/RANGE 프레임])
- PARTITION BY: 분석 단위 구분 (예: 부서별, 지역별) → GROUP BY와 유사하지만 행 유지
- ORDER BY: 윈도우 내 정렬 기준
- ROWS/RANGE BETWEEN: 집계 범위 프레임 지정

Window Function — GROUP BY와 핵심 차이

【 GROUP BY 】

- 그룹으로 집계
→ 원래 행 수가 줄어듦
(개별 행 정보 소실)

【 Window Function 】

- 집계하면서도 원래 행을 유지
→ 행별 결과 + 집계값
동시 표현

기본 문법

- 함수명() OVER ([PARTITION BY 컬럼] [ORDER BY 컬럼] [ROWS/RANGE 프레임])
- PARTITION BY: 분석 단위 구분 (예: 부서별, 지역별) → GROUP BY와 유사하지만 행 유지
- ORDER BY: 윈도우 내 정렬 기준
- ROWS/RANGE BETWEEN: 집계 범위 프레임 지정

Window Function — GROUP BY와 핵심 차이

【 GROUP BY 】

- 그룹으로 집계
→ 원래 행 수가 줄어듦
(개별 행 정보 소실)

【 Window Function 】

- 집계하면서도 원래 행을 유지
→ 행별 결과 + 집계값
동시 표현

순위 함수

- ROW_NUMBER(): 동점 상관없이 고유 번호 → 1, 2, 3, 4
- RANK(): 동점자 같은 순위, 다음 건너뛰 → 1, 2, 2, 4
- DENSE_RANK(): 동점자 같은 순위, 연속 → 1, 2, 2, 3
- NTILE(n): n개 그룹으로 균등 분할 (백분위 분석)

Window Function — GROUP BY와 핵심 차이

【 GROUP BY 】

- 그룹으로 집계
→ 원래 행 수가 줄어듦
(개별 행 정보 소실)

【 Window Function 】

- 집계하면서도 원래 행을 유지
→ 행별 결과 + 집계값
동시 표현

이동/비교 함수

- LAG(col, n) / LEAD(col, n) / FIRST_VALUE / LAST_VALUE

Window Function — GROUP BY와 핵심 차이

【 GROUP BY 】

- 그룹으로 집계
→ 원래 행 수가 줄어들음
(개별 행 정보 소실)

【 Window Function 】

- 집계하면서도 원래 행을 유지
→ 행별 결과 + 집계값
동시 표현

집계 Window

- SUM/AVG/COUNT/MIN/MAX OVER() — 누적합, 이동평균

ROW_NUMBER / RANK / DENSE_RANK 비교

【 SQL 예시 】

-- 쇼핑몰 매출 데이터 분석

```
CREATE TABLE sales (
  id SERIAL PRIMARY KEY, region VARCHAR(20), month DATE, revenue NUMERIC
);
INSERT INTO sales VALUES
  (DEFAULT,'Seoul','2025-01-01',10000),(DEFAULT,'Seoul','2025-02-01',13000),
  (DEFAULT,'Seoul','2025-03-01',12500),(DEFAULT,'Busan','2025-01-01',9000),
  (DEFAULT,'Busan','2025-02-01',9500);
```

-- 지역별 매출 순위 (3가지 함수 동시 비교)

```
SELECT region, month, revenue,
  ROW_NUMBER() OVER (PARTITION BY region ORDER BY revenue DESC) AS row_num,
  RANK() OVER (PARTITION BY region ORDER BY revenue DESC) AS rank_r,
  DENSE_RANK() OVER (PARTITION BY region ORDER BY revenue DESC) AS d_rank,
  NTILE(3) OVER (PARTITION BY region ORDER BY revenue DESC) AS tier,
  -- 전체 학교 순위 (PARTITION 없이 전체 기준)
  RANK() OVER (ORDER BY revenue DESC) AS overall_rank
FROM sales;
```

-- 동일 매출 시:

-- ROW_NUMBER: 1,2,3,4 (항상 고유)

-- RANK: 1,2,2,4 (건너뛰)

-- DENSE_RANK: 1,2,2,3 (연속)

LAG / LEAD — 전월 대비 증감 분석

【 SQL 예시 】

```
-- 지역별 전월 대비 매출 증감 계산
SELECT
  region,
  month,
  revenue,
  LAG(revenue, 1) OVER (PARTITION BY region ORDER BY month)
                        AS prev_month_sales,
  revenue - LAG(revenue, 1) OVER (PARTITION BY region ORDER BY month)
                        AS mom_diff,
  ROUND(
    (revenue - LAG(revenue, 1) OVER (PARTITION BY region ORDER BY month))
    / NULLIF(LAG(revenue, 1) OVER (PARTITION BY region ORDER BY month), 0) * 100, 1
  )
  AS mom_growth_pct,
-- LEAD: 다음 달 매출 미리 보기
  LEAD(revenue, 1) OVER (PARTITION BY region ORDER BY month)
                        AS next_month_sales
FROM sales
ORDER BY region, month;

-- 활용 사례:
-- MoM (Month-over-Month) 성장률
-- YoY (Year-over-Year): LAG(revenue, 12) - 12개월 전 대비
-- 연속 이벤트 감지 (로그인 연속일 수)
-- 대기 시간 계산 (이전 이벤트와의 간격)
```

Checkpoint

- ✓ **NULLIF(prev, 0)**
: 이전 값이 0일 때
0 나누기 방지
→ **NULL**
반환으로
오류 방지

누적합 / 이동평균 — ROWS/RANGE 프레임

【 SQL 예시 】

-- 누적합 (Running Total / Cumulative Sum)

```
SELECT region, month, revenue,
       SUM(revenue) OVER (
         PARTITION BY region
         ORDER BY month
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS cumulative_revenue
FROM sales;
```

-- 3개월 이동평균 (Moving Average)

```
SELECT region, month, revenue,
       ROUND(AVG(revenue) OVER (
         PARTITION BY region
         ORDER BY month
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW -- 현재 포함 최근 3개월
       ), 0) AS ma_3month
FROM sales;
```

-- 카테고리별 누적 점유율 (비율 분석)

```
SELECT product, sales,
       SUM(sales) OVER (ORDER BY sales DESC ROWS UNBOUNDED PRECEDING) AS cum_sales,
       SUM(sales) OVER () AS total_sales,
       ROUND(SUM(sales) OVER (ORDER BY sales DESC ROWS UNBOUNDED PRECEDING)
         / SUM(sales) OVER () * 100, 1) AS cum_pct
FROM product_sales;
```

Checkpoint

- ✓ **ROWS vs RANGE**
: ROWS
=물리적 행 기준
(정확)
- ✓ **RANGE**
=ORDER BY 값
기준
(같은 값은 모두 포함)

Window Function 고급 패턴 — 실무 활용

【 SQL 예시 】

-- 1. 코호트 분석: 가입 월별 재구매율

```
WITH monthly_orders AS (  
  SELECT customer_id, DATE_TRUNC('month', order_date) AS order_month  
  FROM orders  
)  
first_order AS (  
  SELECT customer_id, MIN(order_month) AS cohort_month  
  FROM monthly_orders GROUP BY customer_id  
)  
SELECT fo.cohort_month,  
       mo.order_month,  
       COUNT(DISTINCT fo.customer_id) AS customers,  
       EXTRACT(MONTH FROM AGE(mo.order_month, fo.cohort_month)) AS months_after  
FROM first_order fo JOIN monthly_orders mo USING(customer_id)  
GROUP BY 1, 2 ORDER BY 1, 2;
```

- 2. 백분위 분석 (성적 분포)

```
SELECT student_id, score,  
       PERCENT_RANK() OVER (ORDER BY score) AS pct_rank,  
       CUME_DIST() OVER (ORDER BY score) AS cume_dist,  
       NTILE(4) OVER (ORDER BY score) AS quartile  
FROM enrollments WHERE course_id = 1
```

Window Function 고급 패턴 — 실무 활용

【 SQL 예시 】

```
-- 3. Gap and Island: 연속 구간 찾기
-- 방법: date - ROW_NUMBER()로 그룹키 생성
SELECT MIN(login_date) AS streak_start, MAX(login_date) AS streak_end,
COUNT(*) AS streak_days
FROM (
SELECT login_date,
login_date - ROW_NUMBER() OVER (ORDER BY login_date) * INTERVAL '1 day' AS grp
FROM user_logins WHERE user_id = 42
) t GROUP BY grp ORDER BY streak_start;
```


Window Function DBMS 지원 비교

함수/기능	PostgreSQL	MySQL	Oracle	SQL Server
ROW_NUMBER	O	O(8.0+)	O	O(2005+)
RANK/DENSE_RANK	O	O(8.0+)	O	O
NTITLE(n)	O	O(8.0+)	O	O
LAG/LEAD	O	O(8.0+)	O	O
FIRST_VALUE / LAST_VALUE	O	O(8.0+)	O	O
ROWS /RANGE BETWEEN	O	O(8.0+)	O	O
FILTER 절	O	X (CASE 대체)	X (CASE 대체)	X (CASE 대체)
통계함수 (PERCENTILE 등)	O	제한적	O	O

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

[참고] 심화 & 최신 트렌드

- LATERAL JOIN · 집합 연산 DBMS 비교 · AI SQL 도구

[참고] LATERAL JOIN — 행별 독립 서브쿼리

【 SQL 예시 】

```
-- LATERAL: 외부 쿼리의 각 행마다 서브쿼리를 독립 실행
-- 각 학생의 최근 수강 과목 Top-3 조회
SELECT s.name, recent.title, recent.score
FROM students s
CROSS JOIN LATERAL (
  SELECT c.title, e.score
  FROM enrollments e JOIN courses c ON c.id = e.course_id
  WHERE e.student_id = s.id      -- ← 외부 행 참조!
  ORDER BY e.score DESC
  LIMIT 3
) recent;

-- 각 카테고리별 Best-2 상품
SELECT cat.name AS category, top.*
FROM categories cat
CROSS JOIN LATERAL (
  SELECT p.name, p.price
  FROM products p
  WHERE p.category_id = cat.id
  ORDER BY p.price DESC
  LIMIT 2
) top;
```

Checkpoint

- ✓ **LATERAL** 없이는 **ROW_NUMBER** + 서브쿼리 조합으로 복잡하게 구현
- ✓ **MySQL**
: **LATERAL** 미지원
→ **FROM** 서브쿼리 + 윈도우함수
- ✓ **PostgreSQL, Hive, BigQuery**
: **LATERAL** 지원

[참고] UPSERT — INSERT ON CONFLICT

【 SQL 예시 】

-- **UPSERT**: 없으면 **INSERT**, 있으면 **UPDATE** (PostgreSQL)

```
INSERT INTO user_stats (user_id, login_count, last_login)
```

```
VALUES (42, 1, now())
```

```
ON CONFLICT (user_id)          -- PK 또는 UNIQUE 충돌 시
```

```
DO UPDATE SET
```

```
  login_count = user_stats.login_count + EXCLUDED.login_count,
```

```
  last_login = EXCLUDED.last_login;
```

-- **EXCLUDED**: 충돌된 새 값을 참조하는 가상 테이블

-- 중복 무시 (**DO NOTHING**)

```
INSERT INTO log_events (event_id, payload)
```

```
VALUES ('ev-001', '{"type": "click"}')
```

```
ON CONFLICT (event_id) DO NOTHING;
```

-- **MySQL: ON DUPLICATE KEY UPDATE**

```
INSERT INTO user_stats (user_id, login_count)
```

```
VALUES (42, 1)
```

```
ON DUPLICATE KEY UPDATE login_count = login_count + 1;
```

-- **SQL Server: MERGE 문 (표준 SQL)**

```
MERGE INTO user_stats t
```

```
USING (SELECT 42 AS uid, 1 AS cnt) s ON t.user_id = s.uid
```

```
WHEN MATCHED THEN UPDATE SET login_count = t.login_count + s.cnt
```

```
WHEN NOT MATCHED THEN INSERT (user_id, login_count) VALUES (s.uid, s.cnt);
```

Checkpoint

✓ UPSERT 활용

: 실시간 집계 카운터

: 스트리밍 데이터 적재

필수 패턴

[참고] STRING_AGG / ARRAY_AGG / JSON_AGG

【 SQL 예시 】

```
-- STRING_AGG: 여러 행을 하나의 문자열로
SELECT s.name,
       STRING_AGG(c.title, ', ' ORDER BY c.title) AS courses_taken
FROM   students s JOIN enrollments e ON e.student_id = s.id
       JOIN courses   c ON c.id = e.course_id
GROUP BY s.id, s.name;

-- ARRAY_AGG: 배열로 집계 (PostgreSQL)
SELECT major_id,
       ARRAY_AGG(name ORDER BY grade DESC) AS students,
       ARRAY_AGG(DISTINCT grade)           AS grades
FROM   students GROUP BY major_id;
```

[참고] STRING_AGG / ARRAY_AGG / JSON_AGG

【 SQL 예시 】

```
-- JSON_AGG + JSON_BUILD_OBJECT: JSON 배열로 집계 (API 응답용)
SELECT s.name,
       JSON_AGG(
         JSON_BUILD_OBJECT(
           'course', c.title,
           'score', e.score,
           'grade', CASE WHEN e.score >= 90 THEN 'A'
                        WHEN e.score >= 80 THEN 'B' ELSE 'C' END
         ) ORDER BY e.score DESC
       ) AS enrollment_history
FROM   students s
       JOIN enrollments e ON e.student_id = s.id
       JOIN courses   c ON c.id = e.course_id
GROUP BY s.id, s.name;
```

[참고] AI 기반 SQL 도구 활용 현황

Chat2DB (chat2db.ai)

- AI 연동 SQL 작성 + ERD 기반 자동 SQL 생성
- ChatGPT/Claude 연동
→ 자연어로 쿼리 작성, 오류 자동 수정
- DBeaver처럼 멀티 DB 지원
: PostgreSQL, MySQL, Oracle
, SQL Server 등

GitHub Copilot for SQL / Tabnine

- IDE에서 SQL 자동완성
- 쿼리 패턴 학습 기반, 반복 작업 효율화

[참고] AI 기반 SQL 도구 활용 현황

Cloud AI 기반 도구

- **BigQuery with Gemini**
: 자연어 → SQL 변환, 쿼리 최적화 제안
- **Snowflake Cortex AI**
: 내장 AI로 분석 자동화
- **AWS QueryEditor + Bedrock**
: SQL 작성·실행·최적화 통합

pgvector + LLM

- DB 자체가 AI 서비스의 일부
- 임베딩 저장 → 유사도 검색
→ **RAG (Retrieval-Augmented Generation)**
- **SELECT * FROM**
documents
ORDER BY embedding <=> query_vector
LIMIT 5

[참고] AI 기반 SQL 도구 활용 현황

AI SQL 도구 주의사항

- 생성된 쿼리는 반드시 **EXPLAIN**으로 성능 검증
- 데이터 보안: 민감 스키마를 **AI**에 전달 시 정보 유출 위험

[참고] 집합 연산자 DBMS별 지원 비교

연산자	PostgreSQL	MySQL	Oracle	SQL Server	비고
UNION ALL	O	O	O	O	모든 DBMS 지원, 중복 포함
UNION	O	O	O	O	중복 제거 (정렬 비용)
INTERSECT	O	O (8.0.31+)	O	O	MySQL 이전 버전: EXISTS로 대체
EXCEPT	O	X	▲	O	MySQL: NOT EXISTS / LEFT JOIN+IS NULL으로 대체
INTERSECT ALL	O	X	X	X	빈도 기반 교집합 (희귀)
EXCEPT ALL	O	X	X	X	빈도 기반 교집합 (희귀)

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

[참고] 집계 & DML 심화

- 트랜잭션 제어 · DB별 AUTO COMMIT · 정렬 · 페이지네이션

[참고] 트랜잭션 제어 — BEGIN / COMMIT / ROLLBACK / SAVEPOINT

【 SQL 예시 】

-- 기본 트랜잭션 (PostgreSQL)

```
BEGIN;
UPDATE account SET balance = balance - 10000 WHERE id = 'A';
UPDATE account SET balance = balance + 10000 WHERE id = 'B';
COMMIT; -- 성공 시 확정
```

-- 오류 발생 시 롤백

```
BEGIN;
UPDATE account SET balance = balance - 10000 WHERE id = 'A';
-- 만약 여기서 오류 →
ROLLBACK; -- 전체 취소
```

-- SAVEPOINT: 중간 저장점 (부분 롤백)

```
BEGIN;
INSERT INTO orders(customer_id, amount) VALUES(1, 50000);
SAVEPOINT after_insert; -- 중간 저장

INSERT INTO order_items(order_id, product_id) VALUES(lastval(), 99);
-- 여기서 문제 발생 →
ROLLBACK TO SAVEPOINT after_insert; -- insert 이후만 취소
-- COMMIT; 또는 ROLLBACK;
```

Checkpoint

- ✓ **MySQL**
: START TRANSACTION , BEGIN
- ✓ **Oracle**
: DML 시작 시 자동 트랜잭션
(명시적 BEGIN 없음)
- ✓ **SQL Server**
: BEGIN TRANSACTION,
BEGIN TRAN
- ✓ **SAVEPOINT**
: 배치 처리 시 청크 단위 커밋,
오류시 마지막 체크포인트로 롤백

[참고] DB별 AUTO COMMIT 정책 비교

DBMS	기본 AUTO COMMIT	트랜잭션 시작	묵시적 COMMIT 발생 시
PostgreSQL	ON	BEGIN; START TRANSACTION;	DDL 실행 (대부분 트랜잭션 내 허용)
MySQL (InnoDB)	ON	START TRANSACTION; BEGIN;	DDL 실행 (ALTER/CREATE/DROP/TRUNC ATE)
Oracle	OFF(자동 시작)	DML 시작 시 자동으로 트랜잭션 시작	DDL 실행 전·후 자동 COMMIT
SQL Server	ONSkcc29	BEGIN TRANSACTION; BEGIN TRAN;	DDL은 트랜잭션 내 가능 (ROLLBACK 가능)

[참고] 정렬(ORDER BY)과 NULL 처리

구분	PostgreSQL	MySQL	Oracle	SQL Server
기본 정렬	ORDER BY col ASC (기본) / ORDER BY col DESC 여러 컬럼 정렬: ORDER BY salary DESC, name ASC 숫자·날짜·문자열 정렬 기준이 다름 — 문자 "10" < "9" (사전순)			
NULL 정렬 순서	ASC이면 NULL이 마지막 (NULLS LAST가 기본)	ASC이면 NULL 먼저, DESC이면 NULL 나중	ASC이면 NULL이 마지막 (NULLS LAST가 기본)	NULL은 ASC에서 가장 앞에
NULL 정렬 명시적 제어	ORDER BY col DESC NULLS LAST	ORDER BY ISNULL(col), col DESC — NULL을 마지막으로	ORDER BY col DESC NULLS LAST	ORDER BY CASE WHEN col IS NULL THEN 1 ELSE 0 END, col
결과 제한	LIMIT n OFFSET m	LIMIT n OFFSET m	FETCH FIRST n ROWS ONLY / OFFSET m ROWS FETCH NEXT n ROWS ONLY (12c+)	SELECT TOP(n) ... / OFFSET m ROWS FETCH NEXT n ROWS ONLY (표준)

[참고] 페이지네이션 — OFFSET 방식 vs Cursor/Keyset 방식

【 SQL 예시 】

```
-- OFFSET 방식 (간단하지만 대용량에서 느림)
SELECT * FROM orders ORDER BY id LIMIT 10 OFFSET 9990;
-- 문제: DB가 9990개를 읽고 버린 후 10개만 반환 → O(N)
-- OFFSET이 클수록 더 느려짐 (100만 페이지 = 100만 행 스캔)

-- Cursor/Keyset 방식 (권장 — 항상 O(log N))
-- 첫 페이지 (first page)
SELECT * FROM orders ORDER BY id LIMIT 10;
-- 반환된 마지막 row의 id = 1000 (예시)

-- 다음 페이지: 마지막 id 이후부터 바로 탐색
SELECT * FROM orders WHERE id > 1000 ORDER BY id LIMIT 10;
-- → 인덱스로 바로 1001부터 접근 → 항상 빠름

-- 정렬 기준이 고유하지 않을 때: 복합 조건
-- 마지막: (created_at='2024-03-15', id=500)
SELECT * FROM orders
WHERE (created_at, id) > ('2024-03-15', 500)
ORDER BY created_at, id LIMIT 10;
```

Checkpoint

- ✓ 단점
 - : 임의의 페이지 이동 불가
 - (1→3→2 순서 이동 어려움)
- ✓ 적합
 - : 무한 스크롤
 - : 다음 페이지 버튼만 있는 UI

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

[참고] JOIN 심화 — 실전 패턴 & 성능

- Multi-table JOIN · ON vs WHERE · 실행계획 해석

[참고] ON 절 vs WHERE 절 — OUTER JOIN에서의 차이

【 SQL 예시 】

-- OUTER JOIN에서 ON vs WHERE는 완전히 다른 결과!

-- 시나리오: 모든 학생과 수강 정보 (수강 없으면 NULL)
-- + CS101 과목 성적만 보고 싶음

-- 잘못된 방법: WHERE 절에서 필터 → 사실상 INNER JOIN

```
SELECT s.name, e.score
FROM students s LEFT JOIN enrollments e ON e.student_id = s.id
WHERE e.course_id = 1; -- LEFT JOIN 효과 사라짐! 수강 없는 학생
제외됨
```

-- 올바른 방법: ON 절에서 조건 추가 → LEFT JOIN 유지

```
SELECT s.name, e.score
FROM students s
LEFT JOIN enrollments e ON e.student_id = s.id AND e.course_id = 1;
-- → 수강 없는 학생: score = NULL로 표시됨
```

-- 핵심: LEFT JOIN에서 WHERE로 우측 테이블 필터링
→ Anti-Join 패턴(IS NULL) 제외하면 사실상 INNER JOIN

Checkpoint

- ✓ **ON 절 조건**
: 조인 과정에서 적용
(OUTER JOIN 결과 유지)
- ✓ **WHERE**
: 조인 완료 후 행 필터
(NULL 행 제거 → INNER JOIN)
- ✓ 의도적으로 INNER JOIN처럼 쓰고 싶으면 WHERE 사용
- ✓ OUTER JOIN 결과를 유지하면서 특정 조건 필요하면 ON 사용

[참고] Semi-Join — 존재 여부만 확인하는 조인

【 SQL 예시 】

```
-- Semi-Join: 오른쪽 테이블의 존재 여부만 확인, 오른쪽 컬럼은 반환 안 함
-- IN과 EXISTS는 내부적으로 Semi-Join 최적화됨

-- EXISTS (권장): 행 하나라도 있으면 TRUE
SELECT s.name
FROM students s
WHERE EXISTS (SELECT 1 FROM enrollments e WHERE e.student_id = s.id
              AND e.score >= 90);
-- → 성적 90점 이상 과목이 하나라도 있는 학생

-- IN (소량 목록에 적합)
SELECT name FROM students
WHERE major_id IN (SELECT id FROM majors WHERE region = 'Seoul');

-- JOIN으로 구현 (DISTINCT 필요 → 중복 발생 가능)
SELECT DISTINCT s.name
FROM students s JOIN enrollments e ON e.student_id = s.id
WHERE e.score >= 90;
-- DISTINCT 없으면 90점 이상 과목 수만큼 중복 행 발생

-- Anti Semi-Join: 존재하지 않는 경우 (NOT EXISTS)
SELECT s.name
FROM students s
WHERE NOT EXISTS (SELECT 1 FROM enrollments e WHERE e.student_id = s.id);
-- → 단 한 번도 수강하지 않은 학생
```

[참고] JOIN 종류별 결과 행 수 예측

JOIN 종류	A(5행) JOIN B(3행) 매칭2	결과 행 수	NULL 포함 컬럼
INNER JOIN	A.id=B.a_id 2건 매칭	2행	없음
LEFT JOIN	A전체+B.매칭	5행	B컬럼 (3행은 NULL)
RIGHT JOIN	A.매칭+B전체	3행 + 비매칭A	A컬럼 (일부 NULL)
FULL OUTER JOIN	A전체+B전체+겹침	5+3-2=6행	양쪽 비매칭 컬럼
CROSS JOIN	전체 조합	5×3=15행	없음
Anti-Join (NOT EXISTS)	A에서 B매칭 제외	5-2=3행	없음

[참고] 다중 테이블 JOIN — 실습 사용 쿼리

【 SQL 예시 】

```
-- 학생-수강-강좌-교수-학과 5개 테이블 JOIN
SELECT
  s.student_no      AS 학번,
  s.name            AS 학생,
  d.name            AS 학과,
  c.code            AS 강좌코드,
  c.title           AS 강좌명,
  p.name            AS 담당교수,
  e.score           AS 점수,
  CASE
    WHEN e.score >= 90 THEN 'A'
    WHEN e.score >= 80 THEN 'B'
    WHEN e.score >= 70 THEN 'C'
    WHEN e.score >= 60 THEN 'D'
    ELSE 'F'
  END              AS 학점
FROM students s
  JOIN departments d ON d.id = s.department_id
  JOIN enrollments e ON e.student_id = s.id
  JOIN courses      c ON c.id = e.course_id
  JOIN professors p  ON p.id = c.professor_id
WHERE s.enrolled = TRUE
  AND e.score IS NOT NULL
ORDER BY s.name, e.score DESC;
```

Checkpoint

- ✓ **ON** 조건 누락 시 **CROSS JOIN** 발생
주의
— 5개 테이블이면 4개의 **ON** 조건
필요

[참고] JOIN 실수 패턴 및 방지법

ON 조건 누락 → 카르테시안 곱 폭발

- **FROM a JOIN b**
→ 반드시 **ON a.id = b.a_id** 추가
- **EXPLAIN**으로 **actual rows**가
예상보다 훨씬 많으면 의심

OUTER JOIN 후 WHERE 절로 NULL 필터 → INNER JOIN으로 변환

- **LEFT JOIN + WHERE b.col IS NOT NULL**
→ Anti-Join이 아닌 **INNER JOIN**
- 해결
: **AND b.col IS NOT NULL** 조건을
WHERE 대신 **ON** 절에 추가

[참고] JOIN 실수 패턴 및 방지법

중복 행 폭발 — 1:N 조인 시 N배 증가

- **COUNT(*)** 결과가 이상하게 크면
다중 **JOIN**으로 인한 중복 의심
- 해결
: 서브쿼리에서 집계 후 **JOIN**, 또는 **DISTINCT**

NOT IN에서 **NULL** 함정

- 서브쿼리에 **NULL**이 하나라도 있으면
결과가 항상 빈 집합
- 해결
: **NOT IN** → **NOT EXISTS**로 변경

[참고] JOIN 실수 패턴 및 방지법

큰 테이블을 Small Table처럼 JOIN — 옵티마이저 오판

- **ANALYZE** 실행으로 통계 갱신 / **EXPLAIN**으로 rows 예측값 확인

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

[참고] 서브쿼리 심화 & CTE 고급

- Correlated Subquery · Materialized CTE · PIVOT

[참고] 상관 서브쿼리 (Correlated Subquery) 최적화

【 SQL 예시 】

```
-- 상관 서브쿼리: 바깥 쿼리의 행마다 내부 쿼리 반복 실행
-- → 행 수 × 내부 쿼리 비용 = 성능 문제

-- 느린 방법: 상관 서브쿼리 (N × M)
SELECT e.emp_name, e.salary, e.dept_id
FROM employees e
WHERE e.salary > (
  SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id
);

-- 빠른 방법 1: CTE로 한 번만 집계
WITH dept_avg AS (
  SELECT dept_id, AVG(salary) AS avg_sal FROM employees GROUP BY dept_id
)
SELECT e.emp_name, e.salary, d.avg_sal
FROM employees e JOIN dept_avg d ON d.dept_id = e.dept_id
WHERE e.salary > d.avg_sal;

-- 빠른 방법 2: Window Function (가장 우아)
SELECT emp_name, salary, dept_id,
  AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
FROM employees
WHERE salary > AVG(salary) OVER (PARTITION BY dept_id);
-- → 서브쿼리 필요 없이 한 번의 스캔으로 처리 가능
```

Checkpoint

- ✓ 상관 서브쿼리는 가독성이
좋지만 성능이 나쁨
→ CTE 또는
Window Function

[참고] CTE MATERIALIZED vs NOT MATERIALIZED (PostgreSQL 12+)

【 SQL 예시 】

```
-- NOT MATERIALIZED (기본): 인라인으로 펼쳐서 최적화 허용
WITH dept_stats AS NOT MATERIALIZED (
  SELECT department_id, COUNT(*) AS cnt, AVG(salary) AS avg_sal
  FROM employees
  GROUP BY department_id
)
SELECT e.name, ds.avg_sal
FROM employees e
JOIN dept_stats ds ON ds.department_id = e.department_id
WHERE e.salary > ds.avg_sal;
-- → 옵티마이저가 CTE를 JOIN 최적화에 포함 가능

-- MATERIALIZED: 결과를 임시 저장 후 재사용 (한 번만 실행)
WITH expensive_calc AS MATERIALIZED (
  SELECT DISTINCT customer_id,
    COUNT(DISTINCT product_id) AS variety
  FROM orders WHERE order_date >= '2024-01-01'
)
SELECT c.name, ec.variety
FROM customers c JOIN expensive_calc ec ON ec.customer_id = c.id
WHERE ec.variety >= 5;
-- 비용이 큰 집계를 한 번만 실행하고 여러 곳에서 재사용
```

Checkpoint

- ✓ 언제 **MATERIALIZED**?
 1. CTE가 여러 번 참조 시
 2. 내부에 랜덤 함수 포함
(random(), now())
 3. 옵티마이저가 잘못된
플랜을 선택할 때
강제 고정

[참고] PIVOT — 행을 열로 변환 (CASE WHEN 방식)

【 SQL 예시 】

```
-- 월별 카테고리별 매출을 행 → 열로 피벗
-- (PostgreSQL/MySQL은 crosstab/동적 피벗 없음 → CASE WHEN 사용)
SELECT
  product_category,
  SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 1 THEN amount ELSE 0 END) AS jan,
  SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 2 THEN amount ELSE 0 END) AS feb,
  SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 3 THEN amount ELSE 0 END) AS mar,
  SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 4 THEN amount ELSE 0 END) AS apr,
  SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 5 THEN amount ELSE 0 END) AS may,
  SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 6 THEN amount ELSE 0 END) AS jun,
  SUM(amount) AS h1_total
FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 2024
GROUP BY product_category
ORDER BY h1_total DESC;

-- SQL Server/Oracle: PIVOT 문법 직접 지원
-- PostgreSQL: tablefunc 확장의 crosstab() 함수 활용 가능
```

[참고] 서브쿼리 vs CTE vs Window Function

서브쿼리

- 단순하고 한 번만 사용되는 조건
- 소량 데이터의 IN 체크
- **EXISTS**로 존재 여부만 확인

CTE

- 복잡한 쿼리를 단계별로 분해 → 가독성
- 같은 집계 결과를 여러 곳에서 재사용
- 재귀 쿼리 (조직도, 카테고리 트리)
- 서브쿼리가 3단계 이상 중첩될 때

[참고] 서브쿼리 vs CTE vs Window Function

Window Function

- 원래 행을 유지하면서 집계값 필요
(순위, 누적, 비교)
- 상관 서브쿼리로 느린 경우
(부서 평균보다 높은 사원)
- **LAG/LEAD**로 이전/다음 행 값 필요
- **ROWS/RANGE** 프레임으로 이동 집계 필요

JOIN

- 두 테이블의 데이터를 함께 조회
→ 항상 **JOIN** 우선 고려
- **EXISTS/IN**으로 표현한 서브쿼리가
복잡해질 때

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

[참고] Window Function 심화

- ROWS/RANGE 프레임 · 실무 패턴 · DBMS별 차이

[참고] ROWS vs RANGE 프레임

【 SQL 예시 】

```
-- ROWS: 물리적 행 개수 기준
-- 현재 행 기준으로 앞/뒤 N개의 물리적 행 포함
SELECT date, revenue,
       SUM(revenue) OVER (
         ORDER BY date
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW -- 현재+앞2개 = 3개
       ) AS sum_3rows
FROM daily_sales;

-- RANGE: 논리적 값 기준 (ORDER BY 값이 같은 행은 모두 포함)
SELECT date, revenue,
       SUM(revenue) OVER (
         ORDER BY date
         RANGE BETWEEN INTERVAL '2 days' PRECEDING AND CURRENT ROW
       ) AS sum_3days -- 2일 전 날짜~오늘까지 모든 행
FROM daily_sales;

-- 프레임 경계 키워드:
-- UNBOUNDED PRECEDING: 파티션의 첫 행
-- N PRECEDING: N행/N값 이전
-- CURRENT ROW: 현재 행
-- N FOLLOWING: N행/N값 이후
-- UNBOUNDED FOLLOWING: 파티션의 마지막 행
```

Checkpoint

- ✓ 정확한 행 개수 기반 이동평균
→ **ROWS** 사용
- ✓ 날짜 범위나 값 범위 기반
집계 → **RANGE** 사용

[참고] NTILE / PERCENT_RANK / CUME_DIST

【 SQL 예시 】

```
-- NTILE(n): n개 구간으로 균등 분할
SELECT student_id, score,
       NTILE(4) OVER (ORDER BY score DESC) AS quartile, -- 4분위
       NTILE(10) OVER (ORDER BY score DESC) AS decile, -- 10분위
       NTILE(100) OVER (ORDER BY score DESC) AS percentile -- 백분위
FROM enrollments WHERE course_id = 1;

-- PERCENT_RANK: 0.0~1.0 백분위 순위
-- 공식: (rank - 1) / (total_rows - 1)
SELECT student_id, score,
       PERCENT_RANK() OVER (ORDER BY score DESC) AS pct_rank,
       CUME_DIST() OVER (ORDER BY score DESC) AS cume_dist
-- CUME_DIST: 현재 값 이하인 행의 비율 (0~1)
FROM enrollments WHERE course_id = 1;
```


[참고] NTILE / PERCENT_RANK / CUME_DIST

【 SQL 예시 】

-- 활용: 상위 20% 학생 추출

```
SELECT * FROM (  
  SELECT student_id, score,  
         PERCENT_RANK() OVER (ORDER BY score DESC) AS pct_rank  
  FROM enrollments WHERE course_id = 1  
) t  
WHERE pct_rank <= 0.20; -- 상위 20%
```

-- FIRST_VALUE / LAST_VALUE: 파티션의 첫/마지막 값

```
SELECT region, month, revenue,  
       FIRST_VALUE(revenue) OVER (PARTITION BY region ORDER BY month) AS first_rev,  
       LAST_VALUE(revenue) OVER (PARTITION BY region ORDER BY month  
                                ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS last_rev  
FROM sales;
```

[참고] Gap & Island — 연속 구간 패턴 심화

【 SQL 예시 】

```
-- 연속 로그인 날짜 구간 찾기 (Gap and Island 문제)
-- 원리: 날짜에서 ROW_NUMBER를 빼면 연속 구간은 같은 값을 가짐
WITH login_data AS (
  SELECT DISTINCT user_id, login_date
  FROM   user_sessions WHERE user_id = 42
),
grouped AS (
  SELECT user_id, login_date,
         login_date - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date)
                        * INTERVAL '1 day') AS grp_key
  FROM   login_data
)
SELECT
  user_id,
  MIN(login_date) AS streak_start,
  MAX(login_date) AS streak_end,
  COUNT(*)       AS consecutive_days
FROM   grouped
GROUP BY user_id, grp_key
ORDER BY streak_start;
```

[참고] Gap & Island — 연속 구간 패턴 심화

【 SQL 예시 】

```
-- 활용 사례:  
-- 연속 출석 보너스 계산  
-- 이상 탐지: 비정상적으로 긴 연속 이벤트  
-- 재고 연속 품질 기간  
-- 장애 연속 발생 구간
```

```
-- Gap 찾기: 활동 없는 날짜 (출석 공백)
```

```
SELECT login_date + INTERVAL '1 day' AS gap_start,  
       LEAD(login_date) OVER (ORDER BY login_date) - INTERVAL '1 day' AS gap_end  
FROM login_data  
WHERE LEAD(login_date) OVER (ORDER BY login_date) > login_date + INTERVAL '1 day';
```

【 SQL 예시 — 1 】

```
-- 코호트 분석: 가입 월별 이후 월 재구매 유지율
WITH user_cohorts AS (
  SELECT user_id,
         DATE_TRUNC('month', created_at) AS cohort_month
  FROM   users
),
monthly_active AS (
  SELECT DISTINCT user_id,
                 DATE_TRUNC('month', order_date) AS active_month
  FROM   orders
),
cohort_matrix AS (
  SELECT
    uc.cohort_month,
    ma.active_month,
    COUNT(DISTINCT uc.user_id) AS active_users,
    EXTRACT(MONTH FROM AGE(ma.active_month, uc.cohort_month)) AS months_after
  FROM   user_cohorts uc
        JOIN monthly_active ma ON ma.user_id = uc.user_id
  GROUP BY 1, 2
),
cohort_size AS (
  SELECT cohort_month, COUNT(*) AS total_users
  FROM   user_cohorts GROUP BY 1
)
```

[참고] 코호트 분석 — Window Function 활용

【 SQL 예시 — 2 】

```
SELECT
  cm.cohort_month,
  cm.months_after,
  cm.active_users,
  cs.total_users,
  ROUND(cm.active_users::NUMERIC / cs.total_users * 100, 1) AS retention_rate
FROM cohort_matrix cm
  JOIN cohort_size cs ON cs.cohort_month = cm.cohort_month
ORDER BY 1, 2;
```

SQL 파일 참조

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

[참고] DBMS별 집계·윈도우 성능 비교

- COUNT·GROUP BY·FILTER·집계 함수 DBMS별 동작

[참고] 집계 함수 DBMS별 상세 비교

함수	PostgreSQL	MySQL/MariaDB	Oracle	SQL Server
COUNT(*)	빠름 (MVCC 특성상 약간 느릴 수 있음)	MyISAM : 메타데이터로 빠름 InnoDB : 전체 스캔	인덱스 활용 적극적	통계 기반 빠름
COUNT(DISTINCT)	일반적으로 빠름	추가 정렬/해시 비용	Hash 집계로 효율적	Hash 집계
AVG(NULL)	NULL 제외 자동	NULL 제외 자동	NULL 제외 자동	NULL 제외 자동
FILTER 절	O	CASE WHEN으로 대체	CASE WHEN으로 대체	CASE WHEN으로 대체
STDDEV/VARIANCE	O	O	O	O
GROUP_CONCAT	STRING_AGG(col,sep)	GROUP_CONCAT(col)	LISTAGG(col,sep)	STRING_AGG(col,sep)

[참고] 집계 쿼리 AI 서비스 연계 — 벡터 임베딩 통계

【 SQL 예시 】

-- 1. 사용자별 임베딩 개수 및 평균 유사도

```
SELECT user_id,
       COUNT(*) AS total_embeddings,
       ROUND(AVG(similarity), 3) AS avg_similarity,
       MAX(similarity) AS best_match
FROM   embedding_store
GROUP BY user_id
ORDER BY avg_similarity DESC;
```

-- 2. 클러스터별 통계 (k-means 결과 분석)

```
SELECT cluster_id,
       COUNT(*) AS cluster_size,
       ROUND(AVG(similarity), 3) AS cohesion,
       COUNT(DISTINCT tag) AS tag_variety,
       STRING_AGG(DISTINCT tag, ' ') AS top_tags
FROM   embedding_store
GROUP BY cluster_id
HAVING COUNT(*) >= 5
ORDER BY cohesion DESC;
```

-- 3. 태그별 빈도 (워드클라우드용 데이터)

```
SELECT tag, COUNT(*) AS freq,
       ROUND(COUNT(*)::NUMERIC / SUM(COUNT(*)) OVER () * 100, 1) AS pct
FROM   embedding_store
GROUP BY tag ORDER BY freq DESC LIMIT 20;
```

Checkpoint

- ✓ AI 서비스에서 임베딩 데이터 분석 (Vector DB 연계)
- ✓ embedding_store (id, user_id, cluster_id, similarity, tag)
- ✓ AI 파이프라인에서 GROUP BY 집계는 임베딩 클러스터 품질 검증, 태그 빈도 분석에 핵심적으로 활용

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

Day 3 예고 — SQL 성능 최적화

- 인덱스 · 실행계획 · 튜닝 · 트랜잭션 · MVCC

쿼리 최적화의 기본 원칙 (미리 보기)

SELECT * 사용 금지

- 필요한 컬럼만 명시
- 불필요한 데이터 전송 방지
- 커버링 인덱스 활용 가능

WHERE 절 인덱스 컬럼에 함수 사용 금지

- **WHERE YEAR(order_date)=2024**
→ 인덱스 무력화!
- **WHERE order_date >= '2024-01-01'**
AND order_date < '2025-01-01'
→ 인덱스 사용

쿼리 최적화의 기본 원칙 (미리 보기)

N+1 문제

- **JOIN**으로 해결
- 루프 안에서 개별 쿼리 반복
→ 한 번의 **JOIN**으로 처리

EXPLAIN / EXPLAIN ANALYZE 습관화

- 실행계획으로 **Seq Scan vs Index Scan** 확인
- **cost, actual rows, Buffers** 체크

쿼리 최적화의 기본 원칙 (미리 보기)

통계 최신화

- **ANALYZE** 주기적 실행
- 통계가 오래되면 옵티마이저가 잘못된 실행계획 선택

미리 보기 — EXPLAIN ANALYZE 기초

【 SQL 예시 】

-- 실행계획 확인 (쿼리 실행 안 함)

EXPLAIN

```
SELECT s.name, AVG(e.score)
FROM students s JOIN enrollments e ON e.student_id = s.id
WHERE s.major_id = 1
GROUP BY s.id, s.name;
```

-- 실행계획 + 실제 시간 측정 (쿼리 실행함)

EXPLAIN (ANALYZE, BUFFERS)

```
SELECT s.name, AVG(e.score) FROM students s
JOIN enrollments e ON e.student_id = s.id
WHERE s.major_id = 1 GROUP BY s.id, s.name;
```

Checkpoint

✓ 결과 해석 방법

Seq Scan: 테이블 전체 스캔

→ 인덱스 필요 신호

Index Scan: 인덱스 사용

Index Only Scan: 인덱스만으로 처리

(가장 효율적)

Hash Join / Nested Loop / Merge Join

cost=0.00..45.3: 예상 비용

actual time=0.1..2.3: 실제 ms

rows=10: 예상 행 수

actual rows=8: 실제 행 수

Buffers: shared hit=5 read=2

→ 캐시 히트 vs 디스크 읽기

미리 보기 — EXPLAIN ANALYZE 기초

【 SQL 예시 】

-- 실행계획 확인 (쿼리 실행 안 함)

```
EXPLAIN
SELECT s.name, AVG(e.score)
FROM students s JOIN enrollments e ON e.student_id = s.id
WHERE s.major_id = 1
GROUP BY s.id, s.name;
```

-- 실행계획 + 실제 시간 측정 (쿼리 실행함)

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT s.name, AVG(e.score) FROM students s
JOIN enrollments e ON e.student_id = s.id
WHERE s.major_id = 1 GROUP BY s.id, s.name;
```

Checkpoint

✓ 인덱스 없을 때

"Seq Scan on students

(cost=0.00..45.00 rows=1000)"

✓ 인덱스 추가 후

"Index Scan using idx_students_major

on students (cost=0.29..8.30 rows=50)"

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

Day 2 전체 정리 & 다음 단계

집계 & 그룹화

- **GROUP BY/HAVING/ROLLUP/CUBE**
- **FILTER절·집계함수 DBMS비교**
- **페이지네이션(Cursor방식)**

JOIN 알고리즘

- **NL·Hash·Sort-Merge 3종**
- **DBMS별 엔진 비교**
- **ON절 vs WHERE절 차이**

JOIN 종류

- **INNER/OUTER/SELF/CROSS**
- **Anti-Join·Semi-Join**
- **FULL OUTER (MySQL 우회)**

서브쿼리·집합

- **스칼라·인라인·상관**
- **EXISTS/IN/ANY/ALL**
- **UNION·INTERSECT·EXCEPT**

CTE & View

- **WITH절·재귀CTE**
- **MATERIALIZED 힌트**
- **View·Materialized View**

Window Function

- **ROW_NUMBER/RANK/NTILE**
- **LAG·LEAD·누적합·이동평균**
- **Gap&Island·코호트 분석**

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

[참고] 실전 SQL 패턴 & 최신 트렌드

- 복잡 JOIN 패턴 · JSON 조작 · 최신 SQL 기능

실전 패턴 — TOP-N Per Group (3가지 방법)

【 SQL 예시 】

-- 각 학과별 성적 상위 3명 추출

-- 방법 1: Window Function (가장 추천)

```
SELECT * FROM (  
  SELECT s.name, m.name AS major, e.score,  
         ROW_NUMBER() OVER (PARTITION BY s.major_id ORDER BY e.score DESC) AS rn  
  FROM   students s  
        JOIN majors m ON m.id = s.major_id  
        JOIN enrollments e ON e.student_id = s.id  
  WHERE  e.course_id = 1  
) ranked  
WHERE rn <= 3;
```

실전 패턴 — TOP-N Per Group (3가지 방법)

【 SQL 예시 】

```
-- 각 학과별 성적 상위 3명 추출

-- 방법 2: LATERAL JOIN (PostgreSQL)
SELECT m.name AS major, top3.name, top3.score
FROM majors m
CROSS JOIN LATERAL (
  SELECT s.name, e.score
  FROM students s JOIN enrollments e ON e.student_id = s.id
  WHERE s.major_id = m.id AND e.course_id = 1
  ORDER BY e.score DESC LIMIT 3
) top3;
```

실전 패턴 — TOP-N Per Group (3가지 방법)

【 SQL 예시 】

-- 각 학과별 성적 상위 3명 추출

-- 방법 3: 상관 서브쿼리 (느림, 학습용)

```
SELECT s.name, e.score, s.major_id
FROM students s JOIN enrollments e ON e.student_id = s.id
WHERE e.course_id = 1
      AND (SELECT COUNT(*) FROM students s2 JOIN enrollments e2 ON e2.student_id = s2.id
           WHERE s2.major_id = s.major_id AND e2.course_id = 1 AND e2.score > e.score) < 3;
```

실전 패턴 — JSONB 조작 (PostgreSQL)

【 SQL 예시 】

```

-- JSONB 데이터 저장 및 조회
CREATE TABLE products (
  id BIGSERIAL PRIMARY KEY,
  name TEXT,
  attrs JSONB -- {"color":"red","size":"L","price":29900}
);

-- JSONB 조회 연산자
SELECT name,
  attrs->>'color' AS color, -- 텍스트로 추출
  attrs->'price' AS price_json, -- JSONB로 추출
  (attrs->>'price')::NUMERIC AS price_num, -- 숫자로 변환
  attrs @> '{"color":"red"}' AS is_red, -- 포함 여부
  attrs ? 'discount' AS has_discount -- 키 존재 여부
FROM products;

-- JSONB 업데이트
UPDATE products
SET attrs = attrs || '{"discount":0.1}' -- 키 추가/수정
WHERE id = 1;
UPDATE products
SET attrs = attrs - 'discount' -- 키 삭제
WHERE id = 1;

-- GIN 인덱스로 JSONB 전체 검색 가속
CREATE INDEX idx_products_attrs ON products USING GIN (attrs);
SELECT * FROM products WHERE attrs @> '{"color":"red","size":"L"}'; -- 인덱스 사용

```

실전 패턴 — 날짜 처리 완전 가이드

【 SQL 예시 】

-- 월별/주별/일별 집계

```
SELECT DATE_TRUNC('month', order_date) AS month,
       COUNT(*) AS orders,
       SUM(amount) AS revenue
FROM orders GROUP BY 1 ORDER BY 1;
```

-- 특정 기간 데이터 (최근 30일)

```
WHERE order_date >= CURRENT_DATE - INTERVAL '30 days'
WHERE order_date >= NOW() - INTERVAL '30 days' -- 시각까지 고려
```

-- 요일별 집계

```
SELECT TO_CHAR(order_date, 'Day') AS weekday,
       EXTRACT(DOW FROM order_date) AS dow_num, -- 0=일, 6=토
       COUNT(*) AS cnt
FROM orders GROUP BY 1, 2 ORDER BY 2;
```

-- 타임존 변환 (UTC → KST +9)

```
SELECT order_date AT TIME ZONE 'UTC' AT TIME ZONE 'Asia/Seoul' AS kst_time
FROM orders;
```

-- 월말일 계산

```
SELECT DATE_TRUNC('month', NOW()) + INTERVAL '1 month' - INTERVAL '1 day' AS last_day;
```

-- 두 날짜 사이 영업일 수

```
SELECT COUNT(*) FILTER (WHERE EXTRACT(DOW FROM d) NOT IN (0, 6)) AS workdays
FROM GENERATE_SERIES('2024-01-01'::DATE, '2024-12-31'::DATE, '1 day') d;
```


실전 패턴 — 문자열 처리 고급

【 SQL 예시 】

-- 이메일 도메인 추출

```
SELECT email,
       SPLIT_PART(email, '@', 2)           AS domain,
       SUBSTRING(email FROM '@(.+)$')      AS domain2
FROM   users;
```

-- 전화번호 정규화 (하이픈 제거)

```
SELECT REGEXP_REPLACE(phone, '[^0-9]', '', 'g') AS clean_phone
FROM   users;
```

-- 이름 마스킹 (홍길동 → 홍*동)

```
SELECT name,
       SUBSTRING(name, 1, 1)
       || REPEAT('*', GREATEST(LENGTH(name)-2, 0))
       || CASE WHEN LENGTH(name) >= 2 THEN SUBSTRING(name, LENGTH(name)) ELSE ''
END
       AS masked_name
FROM   users;
```

-- 텍스트 유사도 (pg_trgm 확장)

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
SELECT a.name, b.name, SIMILARITY(a.name, b.name) AS sim
FROM   products a CROSS JOIN products b
WHERE  a.id < b.id AND SIMILARITY(a.name, b.name) > 0.5; -- 유사 상품 쌍
```

-- 정규표현식 검색

```
SELECT name FROM customers WHERE name ~ '^[가-힣]{2,4}$'; -- 2~4자 한글 이름
```

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

[참고] 최신 SQL 기능 & AI 시대의 DB

- SQL:2016+ 신기능 · RAG 패턴 · 벡터 검색

SQL:2016 이후 주요 신기능

JSON 네이티브 지원 (SQL:2016)

- 표준 JSON_VALUE(), JSON_QUERY(), JSON_OBJECT() 함수
- PostgreSQL JSONB, MySQL JSON, SQL Server OPENJSON 등 각자 구현

Row Pattern Matching (SQL:2016 — MATCH_RECOGNIZE)

- 시계열 데이터에서
주가 패턴(상승→유지→하락) 등
복잡한 패턴 탐지
- Oracle, SQL Server에서 지원
- PostgreSQL은 pg_match_recognize 확장

MERGE (SQL:2003 표준)

- **INSERT + UPDATE + DELETE**를 하나의 문장으로 — **UPSERT**의 표준형
- **Oracle·SQL Server**: 표준 지원
- **PostgreSQL**: **ON CONFLICT**로 대체

FILTER 절 (SQL:2003)

- 집계함수에 조건 추가
: **COUNT(*) FILTER (WHERE status = 'A')**
- **PostgreSQL** 완전 지원
- 다른 **DBMS**: **CASE WHEN**으로 대체

Window Function 확장 (SQL:2011+)

- **ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**
- **EXCLUDE 절 (PG 14+)**: 특정 행 프레임에서 제외

AI 시대의 DB — pgvector 하이브리드 검색

【 SQL 예시 】

```
-- RAG (Retrieval-Augmented Generation) 패턴
-- 1. pgvector 설치
CREATE EXTENSION IF NOT EXISTS vector;

-- 2. 문서 임베딩 저장 테이블 (OpenAI text-embedding-3-small = 1536차원)
CREATE TABLE knowledge_base (
  id      BIGSERIAL PRIMARY KEY,
  title   TEXT NOT NULL,
  content TEXT,
  chunk_idx INT DEFAULT 0,      -- 긴 문서를 청크로 분할
  embedding VECTOR(1536),
  search_ts TSVECTOR,          -- 전문 검색 인덱스
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 3. 인덱스 생성
CREATE INDEX ON knowledge_base USING HNSW (embedding vector_cosine_ops);
CREATE INDEX ON knowledge_base USING GIN (search_ts);
```

Checkpoint

- ✓ 하이브리드 검색
- : 의미 기반(벡터 70%)
- + 정확한 키워드(FTS 30%)
- RAG 답변 품질 향상

AI 시대의 DB — pgvector 하이브리드 검색

【 SQL 예시 】

```
-- RAG (Retrieval-Augmented Generation) 패턴
-- 4. 하이브리드 검색 (벡터 유사도 + 키워드 결합)
WITH vector_search AS (
  SELECT id, 1 - (embedding <=> $1::VECTOR) AS vec_score
  FROM knowledge_base ORDER BY embedding <=> $1::VECTOR LIMIT 20
),
keyword_search AS (
  SELECT id, ts_rank(search_ts, plainto_tsquery('korean', $2)) AS kw_score
  FROM knowledge_base WHERE search_ts @@ plainto_tsquery('korean', $2)
)
SELECT kb.title, kb.content,
       COALESCE(vs.vec_score,0)*0.7 + COALESCE(ks.kw_score,0)*0.3 AS
hybrid_score
FROM knowledge_base kb
  LEFT JOIN vector_search vs ON vs.id = kb.id
  LEFT JOIN keyword_search ks ON ks.id = kb.id
WHERE vs.id IS NOT NULL OR ks.id IS NOT NULL
ORDER BY hybrid_score DESC LIMIT 5;
```

Checkpoint

- ✓ 하이브리드 검색
 - : 의미 기반(벡터 70%)
 - + 정확한 키워드(FTS 30%)
 - RAG 답변 품질 향상

SQL 튜닝 안티패턴 vs 해결책

안티패턴	문제	해결책
SELECT *	불필요한 데이터 전송, 커버링 인덱스 불가	필요한 컬럼만 명시 SELECT id, name, email
인덱스 컬럼에 함수 적용	WHERE YEAR(date)=2024 → 인덱스 무력화	WHERE date >= '2024-01-01' AND date < '2025-01-01'
N+1 문제	루프 내 N번 개별 쿼리 → DB 과부하	JOIN 또는 IN 절로 한 번에 처리
NOT IN (NULL 포함)	서브쿼리에 NULL 시 결과 항상 빈 집합	NOT EXISTS 또는 LEFT JOIN + IS NULL 으로 변경
%keyword% LIKE	인덱스 미사용, 전체 스캔	전문 검색 인덱스(GIN + pg_trgm) 활용
대용량 OFFSET	OFFSET 이 클수록 선형 증가 O(N)	Cursor/Keyset 방식으로 변경
암묵적 타입 변환	WHERE user_id='123' (INT컬럼) → 인덱스 무력화	같은 타입으로 비교: WHERE user_id=123
DISTINCT 남용	GROUP BY 없이 중복 제거만 사용	중복 원인 파악 후 JOIN/집계 구조 개선

GROUP BY 실수 — ONLY_FULL_GROUP_BY 모드

MySQL

ONLY_FULL_GROUP_BY (기본 활성화)

- **SELECT** 절에 **GROUP BY** 키 이외
비집계 컬럼 포함 시 오류 발생
- **SELECT name, dept, COUNT(*)**
FROM emp GROUP BY dept → **ERROR**
- **SELECT dept, COUNT(*)**
혹은
GROUP BY dept, name

PostgreSQL

- **GROUP BY** 키에 없는 컬럼을
SELECT에 쓰면 항상 오류 (엄격)
- 예외: 기본키를 **GROUP BY**에 포함하면
해당 테이블의 모든 컬럼 가능

GROUP BY 실수 — ONLY_FULL_GROUP_BY 모드

Oracle

- **SELECT** 절 비집계 컬럼이 **GROUP BY** 키에 있어야 함 (표준 준수)
- **SELECT ANY_VALUE(col)**로 임의값 선택 가능 (비표준)

실무

- 집계 쿼리 작성 시 "**SELECT**에 쓸 컬럼 → **GROUP BY**에도 포함" 원칙
- **Window Function**은 **GROUP BY** 없이도 집계값과 개별 행을 함께 출력 가능
- 복잡한 집계는 **CTE**로 단계별 분리 → 실수 방지

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

16. 종합실습 - 2

- 종합실습 1
+ CampusHub — 복합 쿼리 실습 (JOIN + 서브쿼리 + 윈도우함수)

종합 실습 - 방향

학습 목표

- 스키마 생성
 - 샘플데이터
 - JOIN 실습
(Inner/Left/Right/Full/Workaround)
 - 확장(Self/Cross) → 실무 집계
 - ON vs WHERE 차이
- 심화실습

사전 준비

- PostgreSQL 스크립트 실행
postgres_join_lab_large

종합실습 1 환경 활용

신규 DB 개설

INNER/OUTER/FULL
/SELF/CROSS/ANTI
/SEMI JOIN과 집계 실습

심화실습
(GROUP BY, Window
function, CTE)

종합 실습 - 배경 및 실습 과제

배경

- 학사 시스템: **student, enroll**
 - 규칙상 수강은 학생당 0~2건
 - $\text{student_id} \% 3 = 0 \rightarrow 0\text{건}$ (약 333명 수강 없음)
 - $= 1 \rightarrow 1\text{건}$ (약 334명)
 - $= 2 \rightarrow 2\text{건}$ (약 333명)
 - 또한 고아 수강(enroll에만 존재)
2건(학생 1001, 1010) 포함
- 캠퍼스 스토어: **customers, orders**
 - 고객 1명당 정확히 6건의 주문이 존재
- 조직도: **emp**
 - CEO 1명, 매니저 10명, 직원 300명
(각 직원은 매니저 10명 중 1명에게 배정)

【 데이터 확인 및 실습과제 】

• Postgres/MySQL: LIMIT 5 / SQL Server: TOP(5)

1. 학생과 수강을 **INNER JOIN**하여
수강 존재 학생의 과목/성적을 조회
2. 모든 학생 기준으로 수강을 붙이고,
과목(없으면 **NULL**)까지 보이기
3. 수강이 기준. 학생이 없으면 학생 정보가 **NULL**
4. 학생/수강 모두 포함
5. 한 번도 수강하지 않은 학생 목록
6. 한 과목 이상 수강한 학생 목록(중복 제거)
7. 고객별 주문건수/총액
8. 총액 상위 10명과 금액
9. 모든 직원과 그 매니저 이름
10. “모든 학생 기준”으로 과목 분포를 보고 싶다
→ **LEFT JOIN + 집계**

종합 실습 - 배경 및 실습 과제(심화)

배경

- **학사 시스템: student, enroll**
 - 규칙상 수강은 학생당 0~2건
 - $\text{student_id} \% 3 = 0 \rightarrow 0\text{건}$ (약 333명 수강 없음)
 - $= 1 \rightarrow 1\text{건}$ (약 334명)
 - $= 2 \rightarrow 2\text{건}$ (약 333명)
 - 또한 고아 수강(enroll에만 존재)
2건(학생 1001, 1010) 포함
- **캠퍼스 스토어: customers, orders**
 - 고객 1명당 정확히 6건의 주문이 존재
- **조직도: emp**
 - CEO 1명, 매니저 10명, 직원 300명
(각 직원은 매니저 10명 중 1명에게 배정)

【 데이터 확인 및 실습과제(심화) 】

- **Postgres/MySQL: LIMIT 5 / SQL Server: TOP(5)**

- 11. DB 과목을 듣지 않은 모든 학생을 나열**
- 12. (가정) 과목별로 매니저가 운영 책임을 갖는다고 가정하고, emp의 매니저(이름 'Mgr_'로 시작)와 과목을 임의로 매핑한 테이블 `course_owner(course, manager_id)`를 만든 뒤, 과목별 수강 인원 + 책임 매니저 이름 리포트를 작성하세요**
- 13. 학생 × 과목 전체 조합을 만들어 "학생별 과목 추천 후보"를 만들되, 샘플 100건만 본다**

종합 실습 - 배경 및 실습 과제(심화)

배경

- **학사 시스템: student, enroll**
 - 규칙상 수강은 학생당 0~2건
 - $\text{student_id} \% 3 = 0 \rightarrow 0\text{건}$ (약 333명 수강 없음)
 - $= 1 \rightarrow 1\text{건}$ (약 334명)
 - $= 2 \rightarrow 2\text{건}$ (약 333명)
 - 또한 고아 수강(enroll에만 존재)
2건(학생 1001, 1010) 포함
- **캠퍼스 스토어: customers, orders**
 - 고객 1명당 정확히 6건의 주문이 존재
- **조직도: emp**
 - CEO 1명, 매니저 10명, 직원 300명
(각 직원은 매니저 10명 중 1명에게 배정)

【 데이터 확인 및 실습과제(심화) 】

- **Postgres/MySQL: LIMIT 5 / SQL Server: TOP(5)**

- 14. 스칼라 서브쿼리 (SELECT 절) 사용
학생 + 소속 학과명 붙이기**
- 15. 평균 GPA 보다 높은 학생 (WHERE 서브쿼리)**
- 16. 자신의 학과 평균 GPA보다 높은 학생
(Correlated subquery)**
- 17. 수강(enroll) 기록이 있는 학생만**
- 18. 한 번도 수강하지 않은 학생**
- 19. HR 학과 학생 일부와의 비교 데모**
- 20. CS 학과 학생 또는 DB 과목을 수강한 학생 목록**

종합 실습 - 배경 및 실습 과제(심화)

배경

- **학사 시스템: student, enroll**
 - 규칙상 수강은 학생당 0~2건
 - $\text{student_id} \% 3 = 0 \rightarrow 0\text{건}$ (약 333명 수강 없음)
 - $= 1 \rightarrow 1\text{건}$ (약 334명)
 - $= 2 \rightarrow 2\text{건}$ (약 333명)
 - 또한 고아 수강(enroll에만 존재)
2건(학생 1001, 1010) 포함
- **캠퍼스 스토어: customers, orders**
 - 고객 1명당 정확히 6건의 주문이 존재
- **조직도: emp**
 - CEO 1명, 매니저 10명, 직원 300명
(각 직원은 매니저 10명 중 1명에게 배정)

【 데이터 확인 및 실습과제(심화) 】

- **Postgres/MySQL: LIMIT 5 / SQL Server: TOP(5)**

- 21. student 테이블에는 학과(major)와 GPA가 있습니다.**
학과별·GPA 구간별 인원을 집계하되, 소계(소계 행)와 총계 행까지 한 번의 쿼리로 아래의 내용이 반영되도록 출력
- **GPA를 구간 (3.0 미만 / 3.0~3.5 / 3.5 초과)으로 분류하는 파생 컬럼을 추가**
 - **GROUP BY ROLLUP(major, gpa_tier)로 학과별, 전체 소계를 동시에 조회**
 - **GROUPING(major) 함수로 소계 행에 '전체' 라벨을 붙일 것**
 - **결과를 major, gpa_tier 순으로 정렬하되 소계 행은 하단에 표시**

종합 실습 - 배경 및 실습 과제(심화)

배경

- **학사 시스템: student, enroll**
 - 규칙상 수강은 학생당 0~2건
 - student_id % 3 = 0 → 0건 (약 333명 수강 없음)
 - = 1 → 1건 (약 334명)
 - = 2 → 2건 (약 333명)
 - 또한 고아 수강(enroll에만 존재)
2건(학생 1001, 1010) 포함
- **캠퍼스 스토어: customers, orders**
 - 고객 1명당 정확히 6건의 주문이 존재
- **조직도: emp**
 - CEO 1명, 매니저 10명, 직원 300명
(각 직원은 매니저 10명 중 1명에게 배정)

【 데이터 확인 및 실습과제(심화) 】

- **Postgres/MySQL: LIMIT 5 / SQL Server: TOP(5)**

22. emp 테이블은 CEO(manager_id = NULL) → 매니저(10명) → 개발자(300명)의 3단계 계층 구조입니다.

WITH RECURSIVE로 모든 직원의 계층 경로와 깊이를 출력

- **WITH RECURSIVE로 CEO에서 시작하는 조직 트리를 탐색**

- 각 행에 **depth(0=CEO), path** 컬럼을 포함
(예: 'CEO > Mgr_2 > Dev_15')

- 매니저별 직속 부하 직원 수를 집계하는 쿼리
별도로 작성 (컬럼명 : **direct_reports**)

종합 실습 - 배경 및 실습 과제(심화)

배경

- **학사 시스템: student, enroll**
 - 규칙상 수강은 학생당 0~2건
 - $\text{student_id} \% 3 = 0 \rightarrow 0\text{건}$ (약 333명 수강 없음)
 - $= 1 \rightarrow 1\text{건}$ (약 334명)
 - $= 2 \rightarrow 2\text{건}$ (약 333명)
 - 또한 고아 수강(enroll에만 존재)
2건(학생 1001, 1010) 포함
- **캠퍼스 스토어: customers, orders**
 - 고객 1명당 정확히 6건의 주문이 존재
- **조직도: emp**
 - CEO 1명, 매니저 10명, 직원 300명
(각 직원은 매니저 10명 중 1명에게 배정)

【 데이터 확인 및 실습과제(심화) 】

- **Postgres/MySQL: LIMIT 5 / SQL Server: TOP(5)**

23. 각 학과(major)별로 GPA가 높은 순서로 순위를 매기고, 상위 3명씩만 추출하는 쿼리를 Window Function으로 작성 (서브쿼리 방식과 CTE 방식 모두 작성)

- 학과 내 순위를 계산
ROW_NUMBER() OVER (PARTITION BY major
ORDER BY gpa DESC)
- GPA가 동일한 경우 student_id 오름차순을
2차 기준으로 사용
- RANK()와 DENSE_RANK()를 함께 계산해
동점 처리 방식 차이를 비교
- 결과에 전체 학과별 학생
수(total_in_major)도 COUNT()
OVER(PARTITION BY major)로 추가

종합 실습 - 배경 및 실습 과제(심화)

배경

- 학사 시스템: **student, enroll**
 - 규칙상 수강은 학생당 0~2건
 - $\text{student_id} \% 3 = 0 \rightarrow 0\text{건}$ (약 333명 수강 없음)
 - $= 1 \rightarrow 1\text{건}$ (약 334명)
 - $= 2 \rightarrow 2\text{건}$ (약 333명)
 - 또한 고아 수강(enroll에만 존재)
2건(학생 1001, 1010) 포함
- 캠퍼스 스토어: **customers, orders**
 - 고객 1명당 정확히 6건의 주문이 존재
- 조직도: **emp**
 - CEO 1명, 매니저 10명, 직원 300명
(각 직원은 매니저 10명 중 1명에게 배정)

【 데이터 확인 및 실습과제(심화) 】

- Postgres/MySQL: LIMIT 5 / SQL Server: TOP(5)

24. enroll 테이블을 **student_id** 기준으로 정렬하고, 각 학생의 이전 수강 과목 대비 성적 변화를 **LAG()**로 계산
(성적은 A=4, B=3, C=2, D=1 점수로 변환)
- **grade**를 숫자 점수(A=4, B=3, C=2, D=1)로 변환하는 **CASE** 식을 작성
 - **LAG(score) OVER (PARTITION BY student_id ORDER BY course)**로 이전 과목 점수를 가져올 것
 - 현재 점수 - 이전 점수를 **diff** 컬럼으로 추가하고, 상승/유지/하락을 텍스트로 표시
 - 학생별 최고점과 최저점의 차이(**score_range**)를 **Window Function**으로 계산

종합 실습 - 배경 및 실습 과제(심화)

배경

- **학사 시스템: student, enroll**
 - 규칙상 수강은 학생당 0~2건
 - $\text{student_id} \% 3 = 0 \rightarrow 0\text{건}$ (약 333명 수강 없음)
 - $= 1 \rightarrow 1\text{건}$ (약 334명)
 - $= 2 \rightarrow 2\text{건}$ (약 333명)
 - 또한 고아 수강(enroll에만 존재)
2건(학생 1001, 1010) 포함
- **캠퍼스 스토어: customers, orders**
 - 고객 1명당 정확히 6건의 주문이 존재
- **조직도: emp**
 - CEO 1명, 매니저 10명, 직원 300명
(각 직원은 매니저 10명 중 1명에게 배정)

【 데이터 확인 및 실습과제(심화) 】

- **Postgres/MySQL: LIMIT 5 / SQL Server: TOP(5)**

25. orders 테이블의 주문을 order_id 순으로 정렬하여 누적 주문금액과 3개 주문 이동평균 계산 (ROWS BETWEEN 사용)

- $\text{SUM}(\text{amount}) \text{ OVER } (\text{ORDER BY order_id} \text{ ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW})$ 로 누적합 계산
- $\text{AVG}(\text{amount}) \text{ OVER } (\text{ORDER BY order_id} \text{ ROWS BETWEEN 2 PRECEDING AND CURRENT ROW})$ 로 3개 이동평균 계산
- customer_id별로 PARTITION을 나눠 고객 별 누적 구매금액도 함께 계산
- 누적합이 전체 합의 50%를 초음하는 첫 번째 order_id를 찾는 쿼리를 작성

종합 실습 - 제출물

실습 제출물
(당일 제출 원칙)

SQL Query 및 결과

- 각 문항별 쿼리와 실행결과 화면
(쿼리와 실행결과가 나온 화면 Screenshot)

제출 방법 및 배점

- Slack 내 댓글 제출 (제출기한 - 3일차 과정 시작 전)
- 문항 당 4점
- 채점 기준
 - 요구사항 반영 여부 + Query 내 컬럼 선택 (불필요한 컬럼 선택 확인)

스마트 데이터터의 이해 및 활용



17.인덱스 설계 및 최적화

18.실행계획 분석

19.SQL 튜닝 & 느린 쿼리 식별

20.MVCC & 트랜잭션 격리 수준

21.Lock & Deadlock 관리

22.고급 DB 설계

23.종합 실습 - 3

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

17. 인덱스 설계 및 최적화

▪ B-Tree · Hash · GIN · GiST · BRIN · 복합 인덱스 · 커버링 인덱스

인덱스 개념 및 트레이드오프

인덱스 없음

- **Table Full Scan**
→ 전체 행 순차 검사 → **$O(N)$**

인덱스 생성

- **B-Tree 탐색** → **$O(\log N)$**
: 수백만 행에서 수십ms 차이
- **핵심 트레이드 오프**
 - 읽기 성능 향상 (자주 쓰는 필드)
WHERE/JOIN/ORDER BY
 - 쓰기 비용 증가
INSERT/UPDATE/DELETE 시
인덱스 재정렬 비용
 - 저장 공간 증가
인덱스 자체도 디스크 공간 소모

인덱스 개념 및 트레이드오프



인덱스 설계 원칙

- **WHERE** 조건에 자주 등장하는 컬럼 우선
- 선택도(**Selectivity**) 높은 컬럼: 고유값이 많을수록 좋은 인덱스
- **FK** 컬럼에 반드시 인덱스 (**JOIN** 성능, **FOREIGN KEY** 체크)
- 쓰기 중심(**OLTP**) vs 읽기 중심(**OLAP**): 인덱스 전략이 다름
- 미사용 인덱스는 제거 (쓰기 비용만 증가, 조회 이점 없음)

인덱스 종류 비교

인덱스	구조	최적용도	PostgreSQL 지원
B-Tree	균형 이진 트리	=, BETWEEN, LIKE 'prefix%', ORDER BY, 범위	기본 인덱스 (모든 DBMS)
Hash	해시 테이블	등호(=) 비교만 — 범위 불가	PostgreSQL, MySQL(제한적)
GIN	역인덱스 (Inverted)	JSONB, 배열, 전문검색(FTS) 등	PostgreSQL, MySQL(FTS 한정)
GiST	일반화 검색 트리	지리정보(PostGIS), 범위 타입, 벡터 유사도	PostgreSQL(확장)
BRIN	블록 범위 최소/최대	타임시리즈, 물리적 연속 저장 대용량	PostgreSQL
Bitmap	비트 배열	저 선택도 컬럼 조합 (AND/OR)	Oracle (DW용)
Clustered	데이터 물리 정렬	범위 스캔, PK 기반 접근	SQL Server/MySQL InnoDB

B-Tree 인덱스 — 생성 및 성능 비교 실습

【 SQL 예시 】

```
-- AI 연계 테이블: 사용자 쿼리 로그
CREATE TABLE query_logs (
  id SERIAL PRIMARY KEY,
  user_id TEXT,
  user_input TEXT,
  similarity FLOAT,
  response_quality TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

Checkpoint

- ✓ 결과: Seq Scan 1850ms
→ Index Scan 0.8ms
(약 2,300배 향상)

B-Tree 인덱스 — 생성 및 성능 비교 실습

【 SQL 예시 】

```
-- 대량 데이터 생성 (100만 건)
INSERT INTO query_logs (user_id, user_input, similarity, response_quality)
SELECT 'u' || (i % 1000),
'query_' || i,
RANDOM(),
CASE (i%4) WHEN 0 THEN 'excellent' WHEN 1 THEN 'good'
WHEN 2 THEN 'fair' ELSE 'poor' END
FROM GENERATE_SERIES(1, 1000000) i;

-- Before: 인덱스 없이 EXPLAIN ANALYZE
EXPLAIN ANALYZE SELECT * FROM query_logs WHERE user_id = 'u123';
-- Seq Scan on query_logs (cost=0.00..20000 rows=1000 actual
time=120..1850)

-- 인덱스 생성
CREATE INDEX idx_query_logs_user ON query_logs(user_id);
CREATE INDEX idx_query_logs_created ON query_logs(created_at DESC);

-- After: 인덱스 적용 후
EXPLAIN ANALYZE SELECT * FROM query_logs WHERE user_id = 'u123';
-- Index Scan using idx_query_logs_user (actual time=0.05..0.8 rows=1000)
```

Checkpoint

✓ 결과: Seq Scan 1850ms
→ Index Scan 0.8ms
(약 2,300배 향상)

인덱스 생성 명령어

【 SQL 예시 】

```
-- 복합 인덱스: 선두 컬럼 원칙 (leftmost prefix rule)
CREATE INDEX idx_orders_cust_date ON orders (customer_id, order_date DESC);

-- 커버링 인덱스: SELECT 컬럼이 모두 인덱스에 있으면 테이블 접근 불필요
CREATE INDEX idx_customer_cover ON customers (customer_id) INCLUDE (name, email);

-- Index Only Scan: 테이블 읽기 없이 인덱스만으로 결과 반환 → 가장 빠름
SELECT customer_id, name, email FROM customers WHERE customer_id = 42;

-- 부분 인덱스 (Partial Index): 특정 조건 행만 인덱싱
CREATE INDEX idx_orders_pending ON orders (created_at)
WHERE status = 'PENDING'; -- PENDING 상태 5%만 인덱싱 → 크기↓, 속도↑

-- 함수 기반 인덱스
CREATE INDEX idx_users_email_lower ON users (LOWER(email));
-- WHERE LOWER(email) = 'user@example.com' → 인덱스 사용

-- CONCURRENTLY: 운영 중 잠금 없이 인덱스 생성
CREATE INDEX CONCURRENTLY idx_products_category ON products (category_id);
```

Checkpoint

- ✓ 선두 컬럼 원칙
 - 복합 인덱스는 앞 컬럼부터
순서대로 조건이 있어야
인덱스 사용 가능
- ✓ 사용 가능 (선두
customer_id 포함)
 - WHERE customer_id = 5
 - WHERE customer_id = 5
AND order_date > '...'
 - WHERE customer_id = 5
ORDER BY order_date
DESC

인덱스 생성 명령어

【 SQL 예시 】

```
-- 복합 인덱스: 선두 컬럼 원칙 (leftmost prefix rule)
CREATE INDEX idx_orders_cust_date ON orders (customer_id, order_date DESC);

-- 커버링 인덱스: SELECT 컬럼이 모두 인덱스에 있으면 테이블 접근 불필요
CREATE INDEX idx_customer_cover ON customers (customer_id) INCLUDE (name, email);

-- Index Only Scan: 테이블 읽기 없이 인덱스만으로 결과 반환 → 가장 빠름
SELECT customer_id, name, email FROM customers WHERE customer_id = 42;

-- 부분 인덱스 (Partial Index): 특정 조건 행만 인덱싱
CREATE INDEX idx_orders_pending ON orders (created_at)
WHERE status = 'PENDING'; -- PENDING 상태 5%만 인덱싱 → 크기↓, 속도↑

-- 함수 기반 인덱스
CREATE INDEX idx_users_email_lower ON users (LOWER(email));
-- WHERE LOWER(email) = 'user@example.com' → 인덱스 사용

-- CONCURRENTLY: 운영 중 잠금 없이 인덱스 생성
CREATE INDEX CONCURRENTLY idx_products_category ON products (category_id);
```

Checkpoint

- ✓ 사용 불가
 - WHERE order_date > '2024-01-01'
 - (customer_id 없음)

인덱스 생성 명령어

【 SQL 예시 】

```
-- 복합 인덱스: 선두 컬럼 원칙 (leftmost prefix rule)
CREATE INDEX idx_orders_cust_date ON orders (customer_id, order_date DESC);

-- 커버링 인덱스: SELECT 컬럼이 모두 인덱스에 있으면 테이블 접근 불필요
CREATE INDEX idx_customer_cover ON customers (customer_id) INCLUDE (name, email);

-- Index Only Scan: 테이블 읽기 없이 인덱스만으로 결과 반환 → 가장 빠름
SELECT customer_id, name, email FROM customers WHERE customer_id = 42;

-- 부분 인덱스 (Partial Index): 특정 조건 행만 인덱싱
CREATE INDEX idx_orders_pending ON orders (created_at)
WHERE status = 'PENDING'; -- PENDING 상태 5%만 인덱싱 → 크기↓, 속도↑

-- 함수 기반 인덱스
CREATE INDEX idx_users_email_lower ON users (LOWER(email));
-- WHERE LOWER(email) = 'user@example.com' → 인덱스 사용

-- CONCURRENTLY: 운영 중 잠금 없이 인덱스 생성
CREATE INDEX CONCURRENTLY idx_products_category ON products (category_id);
```

Checkpoint

- ✓ 복합 인덱스 설계 원칙
 - 등호(=) 조건 컬럼 먼저
 - 범위(<, >) 조건 나중
 - 선택도 높은 컬럼 우선

인덱스 DBMS별 상세 차이

MySQL/MariaDB

- **InnoDB**: 클러스터형 인덱스 기본
(PK 기준으로 데이터 물리 정렬)
- PK 없으면 내부 Row ID 자동 생성
→ 대체 PK 권장
- **GIN/GiST** 미지원
→ **FULLTEXT** 인덱스로 전문검색 (제한적)
- **Hash** 인덱스: InnoDB는 어댑티브 해시 인덱스
(자동, 수동 생성 불가)

PostgreSQL

- 비클러스터형 기본
(힙 저장, 인덱스는 별도 구조)
- **GIN** : **JSONB**, 배열, **FTS** / **GiST**
- 지리정보, 범위, 벡터 유사도
- **BRIN** : 타임시리즈 대용량
(작은 크기, 빠른 빌드)
- **Hash** : 등호만
- **deduplication** : 동일 페이지 내 중복값
공간 절약 (**PG 13+**)

인덱스 DBMS별 상세 차이

Oracle

- **B-Tree(기본) + Bitmap(저선택도 DW용)**
+ **Cluster** 인덱스

SQL Server

- 클러스터형(테이블당 1개)
 - 데이터 물리 정렬
- 비클러스터형 : 여러 개 가능
- **Columnstore** 인덱스
 - **OLAP** 집계에 최적화

인덱스 설계 전략 — 선택도와 컬럼 순서

선택도(Selectivity)

- 컬럼의 고유값 비율
(1에 가까울수록 인덱스 효과 큼)
 - 선택도
= 고유값 수 / 전체 행 수
 - gender(M/F)
→ 선택도 0.5 나쁨
 - email
→ 선택도 ≈ 1 → 좋음

복합 인덱스 컬럼 순서 전략

- 등호(=) 조건 컬럼을 앞에
범위(<, >, BETWEEN)
조건 컬럼을 뒤에
- WHERE status='A'
AND created_at
>'2024-01-01'
→ INDEX(status, created_at)
[OK]
(status로 좁히고
created_at 범위)
→ INDEX(created_at, status)
[비효율]
(범위 넓게 잡고 status 필터)

미사용 인덱스 감지 및 제거

- SELECT indexrelname,
idx_scan
FROM pg_stat_user_indexes
WHERE idx_scan = 0
- idx_scan = 0
→ 쿼리에서 한 번도 사용 안 된
인덱스
→ 삭제 검토
- REINDEX INDEX
CONCURRENTLY idx_name
- 인덱스 재구성

인덱스 관리 — 사용률 모니터링 및 제거

【 SQL 예시 】

-- PostgreSQL: 인덱스 사용률 확인

```
SELECT
indexrelname AS index_name,
relname AS table_name,
idx_scan AS scan_count, -- 인덱스 사용 횟수
idx_tup_read AS tuples_read,
idx_tup_fetch AS tuples_fetched,
pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
ORDER BY idx_scan ASC -- 사용 횟수가 적은 것 = 제거 후보
LIMIT 20;
```

-- 미사용 인덱스 (idx_scan = 0)

```
SELECT indexrelname, relname, pg_size_pretty(pg_relation_size(indexrelid))
FROM pg_stat_user_indexes
WHERE idx_scan = 0
AND indexrelname NOT LIKE 'pk_%' -- PK 제외
ORDER BY pg_relation_size(indexrelid) DESC;
```

인덱스 관리 — 사용률 모니터링 및 제거

【 SQL 예시 】

```
-- PostgreSQL: 인덱스 사용률 확인
-- 인덱스 재구성 (블로트 제거)
REINDEX INDEX CONCURRENTLY idx_orders_cust_date; -- 운영 중 잠금 없이
REINDEX TABLE CONCURRENTLY orders; -- 테이블 전체

-- MySQL: 인덱스 사용 통계
SELECT TABLE_NAME, INDEX_NAME, CARDINALITY
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = 'mydb'
ORDER BY CARDINALITY ASC;
```

인덱스 심화 — B-Tree 내부 구조

B-Tree (Balanced Tree): 모든 리프 노드가 같은 깊이 → $O(\log N)$ 탐색 보장

구성 요소

- **Root Node**
 - B-Tree 시작점
- **Branch Node**
 - 탐색 경로를 안내하는 중간 노드
- **Leaf Node**
 - 실제 인덱스 키 값
 - + 테이블 행의 물리 위치
(ctid/ROWID) 저장
- 리프 노드는 이중 연결 리스트
→ 범위 탐색 효율적

인덱스 키 크기 영향

- 키가 작을수록 노드당 더 많은 항목
→ 트리 깊이 감소
→ 빠름
- **UUID(16바이트)**
vs **BIGINT(8바이트)**
 - **UUID** 인덱스는 더 큰 트리

Fill Factor (페이지 채움 비율)

- 기본 **90%**
 - 페이지를 **90%**만 채우고
10% 여유 공간 남김
- **UPDATE** 많은 테이블
 - **Fill Factor** 낮추면
페이지 분할(Split) 감소
 - **CREATE INDEX ... WITH
(fillfactor = 70);**

커버링 인덱스 심화 — Index Only Scan 달성

【 SQL 예시 】

```
-- 커버링 인덱스: SELECT에 필요한 모든 컬럼이 인덱스에 포함
-- → 테이블 Heap 접근 없이 인덱스만으로 응답 = Index Only Scan

-- 예시: 사용자 이메일로 검색 → id, name 반환
-- 일반 인덱스 (Index Scan: 테이블 접근 필요)
CREATE INDEX idx_email ON users (email);
SELECT id, name FROM users WHERE email = 'a@b.com';
-- → Index Scan using idx_email + Heap Fetch (추가 I/O 발생)

-- 커버링 인덱스 (Index Only Scan: 테이블 접근 없음!)
CREATE INDEX idx_email_cover ON users (email) INCLUDE (id, name);
SELECT id, name FROM users WHERE email = 'a@b.com';
-- → Index Only Scan (I/O 최소화, 훨씬 빠름)

-- 커버링 인덱스 실행계획 확인
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, name FROM users WHERE email = 'a@b.com';
-- "Index Only Scan using idx_email_cover on users"
-- "Heap Fetches: 0" ← 테이블 접근 0번!

-- PostgreSQL: INCLUDE 절 (인덱스 탐색에는 미사용, 반환에만 포함)
-- SQL Server: INCLUDE 절 지원 동일
-- MySQL: 커버링 인덱스 = WHERE + SELECT 컬럼 모두 인덱스에 포함
```

Checkpoint

- ✓ Index Only Scan
 - Heap Fetches=0이 목표
 - Visibility Map이 최신 (VACUUM으로 유지)

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

18. 실행계획 분석

- EXPLAIN ANALYZE · 플랜 읽기 · DBMS별 도구

실행계획이란?

SQL이 느린 이유를 눈으로 보는 창

- DB 옵티마이저가 SQL을 실행하기 전에 결정하는 실행 방법
- "어떤 순서로 테이블을 읽고, 어떤 인덱스를 쓰며, 어떤 조인 방식을 택했는가"

EXPLAIN vs EXPLAIN ANALYZE

- **EXPLAIN**
 - 추정 실행 계획만 보여줌 (쿼리 실행 안 함)
- **EXPLAIN ANALYZE**
 - 실제 실행 + 측정값
(실제 실행됨 — DML시 ROLLBACK 권장)
- **EXPLAIN (ANALYZE, BUFFERS)**
 - 버퍼 히트/디스크 읽기까지 측정

실행계획이란?

실행계획을 읽어야 하는 상황

- 쿼리가 갑자기 느려졌을 때
- 인덱스를 만들었는데 안 쓰이는 것 같을 때
- **JOIN** 방식이 이상하게 보일 때 (**Hash vs Nested Loop**)
- **rows** 예측값과 **actual rows**가 크게 다를 때 (통계 갱신 필요)

시각화 도구: explain.dalibo.com (PostgreSQL JSON 플랜 붙여넣기)

PostgreSQL EXPLAIN ANALYZE — 플랜 읽기

【 SQL 예시 】

```
-- PostgreSQL: 전체 실행계획 측정
BEGIN;
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT s.name, c.title, e.score
FROM students s
JOIN enrollments e ON e.student_id = s.id
JOIN courses c ON c.id = e.course_id
WHERE s.major_id = 1;
ROLLBACK; -- DML이면 롤백

-- 결과 예시:
-- Hash Join (cost=12.5..45.8 rows=200 width=32)
-- (actual time=0.5..2.3 rows=185 loops=1)
-- Hash Cond: (e.student_id = s.id)
-- Buffers: shared hit=12 read=5
-- → shared hit: 메모리에서 읽음 (빠름)
-- → read: 디스크에서 읽음 (느림) → 캐시 히트율 확인
-- → Seq Scan on enrollments (cost=0..15 rows=1000)
-- → 인덱스 없음! → 인덱스 추가 검토
-- → Hash
-- → Index Scan on students (cost=0.29..8 rows=50)
-- Index Cond: (major_id = 1)
--
-- Planning Time: 0.2 ms
-- Execution Time: 2.5 ms ← 전체 실행 시간
```

Checkpoint

- ✓ Hash Batches > 1
 - 메모리 Spill 발생
 - work_mem 증가 검토
- ✓ 예측 rows와 실제 rows 차이
 - ANALYZE 필요

EXPLAIN ANALYZE 상세 활용

【 SQL 예시 】

```
-- DML 포함 시 ROLLBACK으로 실제 변경 방지  
BEGIN;  
EXPLAIN ANALYZE UPDATE orders SET status = 'DONE' WHERE id = 1;  
ROLLBACK;
```

```
-- JSON 포맷 (dalibo.com 붙여넣기용)  
EXPLAIN (ANALYZE, FORMAT JSON) SELECT ...;
```

Checkpoint

- ✓ **Hash Batches > 1**
 - 메모리 **Spill** 발생
 - **work_mem** 증가 검토
- ✓ **예측 rows와 실제 rows 차이**
 - **ANALYZE** 필요

MySQL EXPLAIN / SQL Server 실행계획

【 SQL 예시 】

-- MySQL: EXPLAIN (기본), EXPLAIN ANALYZE (8.0.18+)

EXPLAIN FORMAT=JSON

SELECT * FROM orders o

JOIN customers c ON o.customer_id = c.id

WHERE o.order_date >= '2024-01-01';

-- 결과 해석:

-- "access_type": "range" → 범위 인덱스 스캔 (좋음)

-- "access_type": "ALL" → Full Table Scan (나쁨!)

-- "rows_examined_per_scan": 45 → 예상 스캔 행 수

-- "key": "idx_orders_date" → 사용된 인덱스

-- MySQL: EXPLAIN ANALYZE (실제 측정)

EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 1;

-- "-> Index lookup on orders using idx_cust (customer_id=1)

-- (actual time=0.1..0.5 rows=10 loops=1)"

MySQL EXPLAIN / SQL Server 실행계획

【 SQL 예시 】

-- SQL Server: 텍스트 실행 계획

SET SHOWPLAN_ALL ON;

SELECT * FROM Orders o JOIN Customers c ON o.CustomerID = c.CustomerID
WHERE o.OrderDate >= '2024-01-01';

SET SHOWPLAN_ALL OFF;

-- Oracle

SET AUTOTRACE ON;

SELECT * FROM orders WHERE order_date >= DATE '2024-01-01';

-- Execution Plan: NESTED LOOPS, INDEX RANGE SCAN 등

실행계획 주요 노드 타입 해석

노드타입	의미	Good/Bad	조치
Seq Scan	전체 테이블 순차 스캔	보통 Bad	인덱스 추가 (전체 10% 이상 읽을 때는 의도적으로)
Index Scan	인덱스 탐색 + 테이블 랜덤 접근	Good	FK/WHERE 컬럼 인덱스 확인
Index Only Scan	인덱스만으로 처리(테이블 접근 없음)	Best	커버링 인덱스 설계
Bitmap Heap Scan	비트맵 빌드 후 정렬 접근	양호	여러 인덱스 조합 시
Hash Join	해시 테이블 빌드 후	대용량 등가 조인	Work_mem 부족 시 Spill 주의
Nested Loop	중첩 반복 조인	소규모 + 인덱스	외부 집합이 커지면 느려짐
Sort	결과 정렬	확인 필요!	ORDER BY 컬럼 인덱스 추가
Materialize	임시 결과	확인 필요!	서브 쿼리를 CTE/JOIN dmfh rotjs

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

19. SQL 튜닝 & 느린 쿼리 식별

- 안티패턴 · Slow Query · pg_stat_statements · 파티셔닝

SQL 튜닝 안티패턴 vs 해결책 (핵심)

안티패턴	문제	올바른 방법	효과
SELECT *	불필요한 데이터 전송, 커버링 인덱스 사용 불가	SELECT id, email	- 트래픽 감소 Index Only Scan 가능
WHERE YEAR(date)=2026	인덱스 컬럼에 함수 쓰면 인덱스 무력화	WHERE date >='2026-01-01' AND date <'2026-12-31'	인덱스 Range Scan 활용
WHERE LOWER(email)=x	함수로 인덱스 무력화	CREATE INDEX ON user(LOWER(email))	함수 기반 인덱스 사용
서브쿼리 과다 사용	N*M 반복 실행으로 성능 저하	JOIN or CTE 변경	한 번의 스캔으로 해결
NOT IN(NULL 포함)	NULL 있으면 항상 빈 결과	NOT EXIST 로 변경	NULL 안전 + 빠름
LIKE '%keyword%'	인덱스 미사용, 테이블 Full Scan	pg_trgm GIN 또는 FTS	텍스트 검색 인덱스
암묵적 타입 변환	WHERE int_col='123' (인덱스 무력화)	같은 타입으로 변환 WHERE int_col=123	인덱스 사용
COUNT(*) 과다 사용	집계 전체 스캔	필요 시 집계 테이블 별도 관리	조회 성능 고려

SQL 튜닝 안티패턴 1~4 (인덱스 관련)

【 SQL 예시 】

```
-- 안티패턴 1: 인덱스 컬럼에 함수 적용 → 인덱스 무력화
-- X
WHERE YEAR(order_date) = 2024 -- 인덱스 미사용!
WHERE UPPER(email) = 'A@B.COM' -- 인덱스 미사용!
-- O
WHERE order_date >= '2024-01-01' AND order_date < '2025-01-01'
CREATE INDEX idx_email_lower ON users (LOWER(email)); -- 함수 기반 인덱스

-- 안티패턴 2: SELECT * → 불필요한 데이터 + 커버링 인덱스 불가
-- X
SELECT * FROM orders WHERE customer_id = 1;
-- O
SELECT id, order_date, amount FROM orders WHERE customer_id = 1;
```

SQL 튜닝 안티패턴 1~4 (인덱스 관련)

【 SQL 예시 】

```
-- 안티패턴 3: 암묵적 타입 변환 → 인덱스 무력화
-- O user_id가 INT인데 문자열 비교
WHERE user_id = '12345' -- 전체 타입 변환 발생 → Full Scan
-- O
WHERE user_id = 12345

-- 안티패턴 4: LIKE 앞 와일드카드 → 전체 스캔
-- X
WHERE name LIKE '%길동%' -- B-Tree 인덱스 사용 불가
-- O prefix는 인덱스 사용 가능
WHERE name LIKE '홍%'
-- 또는 GIN + pg_trgm으로 중간 검색 가속
CREATE EXTENSION pg_trgm;
CREATE INDEX idx_name_trgm ON users USING GIN (name gin_trgm_ops);
SELECT * FROM users WHERE name LIKE '%길동%'; -- 인덱스 사용!
```

SQL 튜닝 안티패턴 5~8 (쿼리 구조 관련)

【 SQL 예시 】

```
-- 안티패턴 5: N+1 문제
-- X 루프 안에서 개별 쿼리 (1 + N번)
for student in students:
    score = db.query("SELECT score FROM e WHERE student_id=%s", student.id)
-- O 한 번의 JOIN으로 처리
SELECT s.id, s.name, e.score
FROM students s LEFT JOIN enrollments e ON e.student_id = s.id
WHERE s.id = ANY($1); -- 배열로 한 번에

-- 안티패턴 6: NOT IN + NULL 함정
-- X 서브쿼리에 NULL 있으면 전체 결과 빈 집합!
WHERE id NOT IN (SELECT student_id FROM enrollments)
-- O NOT EXISTS로 변경
WHERE NOT EXISTS (SELECT 1 FROM enrollments e WHERE e.student_id = s.id)
```

Checkpoint

- ✓ 8 안티패턴 암기
 - 함수적용
 - SELECT*
 - 타입불일치
 - LIKE%앞
 - N+1
 - NOT IN
 - DISTINCT남용
 - OFFSET

SQL 튜닝 안티패턴 5~8 (쿼리 구조 관련)

【 SQL 예시 】

```
-- 안티패턴 7: DISTINCT 남용 → 중복 원인 파악 없이 제거
-- X 중복이 왜 발생하는지 파악 안 하고
SELECT DISTINCT s.name FROM students s JOIN enrollments e ON e.student_id = s.id
-- O EXISTS로 존재 여부 확인 (중복 없음)
SELECT name FROM students s WHERE EXISTS (SELECT 1 FROM enrollments e
WHERE e.student_id = s.id)

-- 안티패턴 8: 대용량 OFFSET → 선형 증가
-- X 1만 페이지 = 10만 행 스캔 후 버림
SELECT * FROM orders ORDER BY id LIMIT 10 OFFSET 100000;
-- O Cursor 방식 (O(log N))
SELECT * FROM orders WHERE id > :last_id ORDER BY id LIMIT 10;
```

Checkpoint

- ✓ 8 안티패턴 암기
 - 함수적용
 - **SELECT***
 - 타입불일치
 - **LIKE%앞**
 - **N+1**
 - **NOT IN**
 - **DISTINCT남용**
 - **OFFSET**

느린 쿼리 식별 — MySQL & PostgreSQL

【 SQL 예시 】

-- MySQL: Slow Query Log 활성화

SET GLOBAL slow_query_log = 'ON';

SET GLOBAL long_query_time = 1; -- 1초 이상 쿼리 기록

SET GLOBAL slow_query_log_file = '/var/log/mysql/slow.log';

-- 느린 쿼리 분석 (상위 10개, 실행 시간 기준)

mysqldumpslow -s t -n 10 /var/log/mysql/slow.log;

-- MySQL 8.0: Performance Schema

SELECT DIGEST_TEXT, COUNT_STAR, AVG_TIMER_WAIT/1e12 AS avg_sec

FROM performance_schema.events_statements_summary_by_digest

ORDER BY AVG_TIMER_WAIT DESC LIMIT 10;

-- PostgreSQL: pg_stat_statements 활성화

-- postgresql.conf: shared_preload_libraries = 'pg_stat_statements'

CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

느린 쿼리 식별 — MySQL & PostgreSQL

【 SQL 예시 】

```
-- 느린 쿼리 TOP-10 (평균 실행 시간 기준)
SELECT
ROUND(mean_exec_time::NUMERIC, 2) AS avg_ms,
calls,
ROUND(total_exec_time::NUMERIC, 0) AS total_ms,
ROUND(shared_blks_hit::NUMERIC /
NULLIF(shared_blks_hit+shared_blks_read,0)*100,1) AS cache_hit_pct,
LEFT(query, 100) AS query_snippet
FROM pg_stat_statements
ORDER BY mean_exec_time DESC
LIMIT 10;

-- 통계 초기화 (튜닝 전후 비교)
SELECT pg_stat_statements_reset();
```

Slow Query 수집 및 분석 방법론

1단계

Slow Query 수집 및 분석 방법론

- PostgreSQL
 - log_min_duration_statement = 1000 (1초 이상 로깅)
- MySQL
 - slow_query_log = ON
 - long_query_time = 1
- pg_stat_statements
 - 누적 호출 수, 총 시간, 평균 시간 분석

2단계

병목 쿼리 특정 및 EXPLAIN 분석

- Seq Scan on 대형 테이블 → 인덱스 추가
- actual rows와 rows 예측값 차이 크면
→ ANALYZE 실행
- Hash Batches > 1
→ work_mem 증가 (per-session)

Slow Query 수집 및 분석 방법론

3단계 개선 적용 및 검증

- 인덱스 추가 (**CONCURRENTLY**)
→ 실행계획 재확인
- 쿼리 구조 변경
(서브쿼리→**CTE/JOIN, DISTINCT** 제거 등)
- **Before/After** 실행 시간 비교 및 기록

4단계 지속 모니터링

- **pg_stat_statements RESET** 후
일정 기간 모니터링
- **Cloud**
 - **AWS Performance Insights**
 - **GCP Query Insights**

파티셔닝 전략 — Range / Hash / List

【 SQL 예시 】

-- Range 파티셔닝 (PostgreSQL): 날짜 기준 분할

```
CREATE TABLE sales (  
  id BIGSERIAL,  
  sale_date DATE NOT NULL,  
  amount NUMERIC  
) PARTITION BY RANGE (sale_date);
```

```
CREATE TABLE sales_2023 PARTITION OF sales  
  FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');  
CREATE TABLE sales_2024 PARTITION OF sales  
  FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');  
CREATE TABLE sales_2025 PARTITION OF sales  
  FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
```

Checkpoint

- ✓ 파티셔닝 효과
 - 쿼리 성능(Pruning)
 - + 유지보수
 - (파티션 DROP/ATTACH)
 - + 아카이빙

파티셔닝 전략 — Range / Hash / List

【 SQL 예시 】

```
-- Partition Pruning: WHERE 절에 파티션 키 포함 시 해당 파티션만 스캔
EXPLAIN SELECT * FROM sales WHERE sale_date BETWEEN '2024-06-01' AND '2024-06-30';
```

```
-- → Seq Scan on sales_2024 (sales_2023, sales_2025는 자동 제외)
```

```
-- Hash 파티셔닝 (MySQL): 균등 분산
```

```
CREATE TABLE orders (order_id INT NOT NULL, customer_id INT NOT NULL, ...)
PARTITION BY HASH(customer_id) PARTITIONS 4;
```

```
-- List 파티셔닝 (PostgreSQL): 지역별
```

```
CREATE TABLE orders PARTITION BY LIST (region);
CREATE TABLE orders_kr PARTITION OF orders FOR VALUES IN ('KR', 'KO');
CREATE TABLE orders_us PARTITION OF orders FOR VALUES IN ('US', 'CA');
```

Checkpoint

- ✓ 파티셔닝 효과
 - 쿼리 성능(Pruning)
 - + 유지보수
 - (파티션 DROP/ATTACH)
 - + 아카이빙

파티셔닝 전략 DB별 지원 현황

파티셔닝 타입	PostgreSQL	MySQL/MariaDB	Oracle	SQL Server
Range	O (10+)	O	O	O (Ent.+)
List	O	O	O	O (Ent.+)
Hash	O	O	O	O (Ent.+)
Composite	O (Range+Hash 등 조합)	O	O	제한적
Partition Pruning	O (자동)	O (자동)	O (자동)	O (자동)
파티션 인덱스	O (Local/Global)	O	O (Local/Global)	O

파티셔닝 심화 — 운영 관리 및 성능

파티션 프루닝 (Partition Pruning)

- **WHERE** 조건에 파티션 키 포함 시 해당 파티션만 스캔
- **EXPLAIN**으로 실행 파티션 확인
- 'Seq Scan on orders_2024q1'
- 파티션 키가 **WHERE**에 없으면 모든 파티션 스캔 → 효과 없음

파티션 인덱스

- 각 파티션에 개별 인덱스 자동 생성 (부모 테이블 인덱스 생성 시)
- **CREATE INDEX ON orders (customer_id)**
→ 모든 파티션에 자동 적용

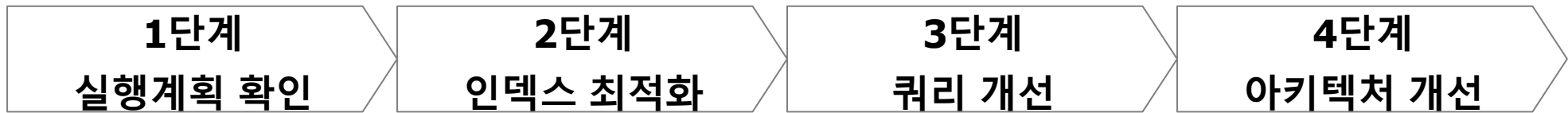
파티셔닝 심화 — 운영 관리 및 성능

파티션 유지관리

- 신규 파티션 미리 생성
 - 범위 파티션은 다음 기간 파티션 사전 준비
- 오래된 파티션 삭제
 - **DROP TABLE orders_2022q1**
→ 매우 빠름 (개별 파일 삭제)
- **pg_partman** 확장
 - **PostgreSQL** 파티션 자동 관리
(생성·삭제·유지보수)

파티셔닝 vs 샤딩 선택

- 파티셔닝
 - 단일 **DB** 인스턴스 내 논리 분할
→ 간단, **JOIN** 가능
- 샤딩
 - 여러 **DB** 인스턴스에 물리 분산
→ 수평 확장, **JOIN** 어려움



SQL 튜닝 실전 체크리스트

1단계 — 실행계획 확인

- **EXPLAIN ANALYZE**로 Seq Scan 여부 확인
- **actual rows vs rows** 예측 차이 큰 경우
→ **ANALYZE** 실행
- **Hash Batches > 1**
→ **work_mem** 부족, **Spill** 발생

2단계 — 인덱스 최적화

- **WHERE/JOIN/ORDER BY** 컬럼에
인덱스 추가
- 복합 인덱스 컬럼 순서
 - 등호(=) 먼저
 - 범위(<>) 나중
- 커버링 인덱스로 **Index Only Scan** 유도

SQL 튜닝 실전 체크리스트

3단계 — 쿼리 개선

- 함수 적용된 컬럼
→ 범위 조건 또는 함수 기반 인덱스로 변경
- 서브쿼리
→ **JOIN** 또는 **CTE**로 변경
- **N+1** 문제
→ 한 번의 **JOIN**으로 해결

4단계 — 아키텍처 개선

- 대용량 집계
→ **Materialized View** 또는 집계 테이블
- 전체 스캔 불가피
→ 파티셔닝으로 스캔 범위 축소
- 읽기 부하
→ **Replica** (읽기 전용 복제본) 분산

SQL 튜닝 Before/After 성능 비교 예시

쿼리 패턴	Before(예시)	After(예시)	성능 향상 (예시)
인덱스 없는 조회	Seq Scan (1000ms)	Index Scan (5ms)	200배 ↑
함수 적용으로 인덱스 무력화	WHERE YEAR(date) = 2024 (Seq Scan)	WHERE date >= '2026-01-01' (Index)	100배 ↑
SELECT * 전체 칼럼	100컬럼 * 100만행 전송	5컬럼만 (필요한 것만, 커버링 인덱스)	20배 ↑
N+1 쿼리	for 1000명 → 1000번 DB 쿼리	JOIN 1번	1000배 ↑
대용량 OFFSET	OFFSET 100만 (1000ms)	Cursor 방식 (3ms)	333배 ↑
Hash Join Spill	work_mem 부족 → 디스크	work_mem 256MB 증가	10배 ↑
미사용 인덱스 유지	INSERT 20ms (5개 인덱스 재정렬처리)	인덱스 2개 삭제 → 8ms	2.5배 ↑

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

20. MVCC & 트랜잭션 격리 수준

- 다중버전 동시성 제어 · Isolation Level · DBMS별 실습

MVCC (Multi-Version Concurrency Control) 심화

MVCC 핵심 원리

- 데이터 수정 시
In-place 덮어쓰기 대신
새 버전 레코드 생성
- 각 트랜잭션은 시작 시점의
스냅샷(**Snapshot**)을 가짐
- 읽기 작업이 쓰기 작업을
차단 안 함
→ **Reader-Writer Lock**
경합 감소

PostgreSQL MVCC 구현

- 각 행에 **xmin**(생성 TX ID),
xmax(삭제/수정 TX ID)
시스템 컬럼 저장
- **UPDATE** = 새 버전 **INSERT**
+ 구 버전 **xmax**
업데이트
(**In-place** 없음)
- **Dead tuple**
- 더 이상 필요 없는 구버전
→ **VACUUM**이 주기적으로 회수
- 장수 트랜잭션(**Long-running**)
- **VACUUM** 차단
→ **Table Bloat** 주의

MySQL InnoDB MVCC

- **Undo Log**
- 변경 전 데이터를
별도 **undo** 공간에 저장
- **Read View**
- 트랜잭션 시작 시 스냅샷 생성
- **Purge Thread**
- 더 이상 필요 없는 **undo log**
자동 정리

MVCC (Multi-Version Concurrency Control) 심화

MVCC 핵심 원리

- 데이터 수정 시
In-place 덮어쓰기 대신
새 버전 레코드 생성
- 각 트랜잭션은 시작 시점의
스냅샷 생성
- 읽기 작업이 쓰기 작업을
차단 안 함
→ **Reader-Writer Lock**
경합 감소

PostgreSQL MVCC 구현

- 각 행에 **xmin**(생성 TX ID),
xmax(삭제/수정 TX ID)
시스템 컬럼 저장

UPDATE → 새 버전 INSERT

업데이트
(In-place 없음)

- **Dead tuple**
 - 더 이상 필요 없는 구버전
 - **VACUUM**이 주기적으로 회수
- 장수 트랜잭션(Long-running)
 - **VACUUM** 차단
 - **Table Bloat** 주의

MySQL InnoDB MVCC

- **Undo Log**
 - 변경 전 데이터를
별도 **undo** 공간에 저장

Read View

스냅샷 생성

- **Purge Thread**
 - 더 이상 필요 없는 **undo log**
자동 정리

비용: 버전 저장 공간 증가 / 장수 TX 시 가비지 증가 / 인덱스 유지 복잡성

Isolation Level 실습 — PostgreSQL

【 SQL 예시 】

```
-- Read Committed (기본값): 커밋된 최신 데이터 읽음
CREATE TABLE bank (id SERIAL, name TEXT, balance INT);
INSERT INTO bank(name, balance) VALUES ('Alice', 1000), ('Bob', 1000);

-- Session 1
BEGIN ISOLATION LEVEL READ COMMITTED;
SELECT balance FROM bank WHERE name='Alice'; -- 1000

-- Session 2 (동시)
BEGIN; UPDATE bank SET balance=800 WHERE name='Alice'; COMMIT;

-- Session 1 재조회
SELECT balance FROM bank WHERE name='Alice'; -- 800 (바뀐 값!)
COMMIT;
```

Isolation Level 실습 — PostgreSQL

【 SQL 예시 】

```
-- Repeatable Read: 트랜잭션 내 항상 같은 값
-- Session 1
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT balance FROM bank WHERE name='Alice'; -- 1000

-- Session 2: UPDATE + COMMIT 후에도
SELECT balance FROM bank WHERE name='Alice'; -- 여전히 1000 (스냅샷 유지)
COMMIT;

-- Serializable: 가장 엄격 — 충돌 시 강제 롤백
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT SUM(balance) FROM bank;
-- Session 2에서 UPDATE 커밋 후 Session 1이 UPDATE 시도 →
-- ERROR: could not serialize access due to concurrent update
```

격리 수준 실습 — RC vs RR

【 SQL 예시 】

```
-- 1. Read Committed (PostgreSQL/Oracle 기본)
-- Session 1
BEGIN ISOLATION LEVEL READ COMMITTED;
SELECT balance FROM bank WHERE name = 'Alice'; -- 결과: 1000

-- Session 2 (동시에 실행)
BEGIN;
UPDATE bank SET balance = 800 WHERE name = 'Alice';
COMMIT;

-- Session 1 재조회 → 800 (바뀜!)
SELECT balance FROM bank WHERE name = 'Alice';
-- → 커밋된 변경사항 즉시 반영 = Non-Repeatable Read 발생!
```

격리 수준 실습 — RC vs RR

【 SQL 예시 】

```
-- 2. Repeatable Read
-- Session 1
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT balance FROM bank WHERE name = 'Alice'; -- 결과: 1000

-- Session 2 (동시에 실행)
BEGIN;
UPDATE bank SET balance = 700 WHERE name = 'Alice';
COMMIT;

-- Session 1 재조회 → 1000 (불변!)
SELECT balance FROM bank WHERE name = 'Alice';
-- → 트랜잭션 시작 시점 스냅샷 유지 = Non-Repeatable Read 방지

-- 격리 수준 설정
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE; -- 현재 트랜잭션
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED; -- MySQL 세션
ALTER SYSTEM SET default_transaction_isolation = 'read committed'; -- PG 전역
```


Serializable — 가장 엄격한 격리

【 SQL 예시 】

-- **Serializable**: 모든 트랜잭션이 순서대로 실행된 것처럼 보장

-- **Session 1**

```
BEGIN ISOLATION LEVEL SERIALIZABLE;  
SELECT SUM(balance) FROM bank; -- 2000
```

-- **Session 2 (동시에)**

```
BEGIN ISOLATION LEVEL SERIALIZABLE;  
UPDATE bank SET balance = balance - 100 WHERE name = 'Alice';  
COMMIT; -- 성공
```

-- **Session 1 계속**

```
UPDATE bank SET balance = balance + 100 WHERE name = 'Bob';  
COMMIT;
```

-- **ERROR: could not serialize access due to concurrent update**

-- → PostgreSQL SSI(Serializable Snapshot Isolation)가 순환 충돌 감지 → 강제 실패

-- 에러 코드: 40001 (Serialization Failure) → 앱에서 재시도 로직 필요

Isolation Level DBMS별 기본값 및 지원

격리수준	Dirty Read	Non-Rep Read	Phantom Read
Read Uncommitted	허용	허용	허용
Read Committed	방지	허용	허용
Repeatable Read	방지	방지	허용(표준) InnoDB
Serializable	방지	방지	방지
Snapshot	방지	방지	방지

Isolation Level DBMS별 기본값 및 지원

격리수준	PostgreSQL	MySQL/MariaDB	Oracle	SQL Server
Read Uncommitted	RC로 처리 (Read Committed)	지원(비권장)	미지원	지원
Read Committed	기본값	지원	기본값	기본값
Repeatable Read	지원	기본값	미지원	지원
Serializable	지원	지원	지원	지원
Snapshot	해당없음	해당없음	해당없음	지원

격리 수준별 실무 적용 가이드

작업 유형	권장 격리 수준	이유	주의사항
단순 분석 SELECT	Read Committed	빠르고 안정적 최신 데이터 읽기	Non-Repeatable Read 허용 확인
보고서, 집계(반복조회)	Repeatable Read	같은 트랜잭션 내 일관성 보장	Phantom Read 가능성
금융 이체, 재고 차감	Serializable	완벽한 무결성 보장	성능 저하, 충돌 재시도 필요
캐시 워밍업, 읽기 전용	Read Committed	성능 우선	읽기 불일치 허용 여부 확인
장바구니, 좌석예약	Serializable 혹은 SELECT FOR UPDATE	동시 접근 충돌 방지	Deadlock 가능성 설계 주의

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

21. Lock & Deadlock 관리

- Row Lock · Advisory Lock · 감지 · 해소 · 모니터링

Lock 종류 및 PostgreSQL Lock 구조

왜 Lock이 필요한가: "여러 트랜잭션이 동일 데이터 동시 변경 → 무결성 위반 위험"

PostgreSQL Lock 계층

- **Row-Level Lock (Tuple Lock)**
 - 행 단위 충돌 방지
(UPDATE, DELETE)
- **Table-Level Lock**
 - ACCESS SHARE
 - ROW EXCLUSIVE
 - ACCESS EXCLUSIVE 등
- **Advisory Lock**
 - 사용자 정의 논리 락
(pg_advisory_xact_lock())

Row-Level Lock 옵션

- **FOR UPDATE**
 - 해당 행 배타적 Lock
→ 다른 UPDATE/DELETE 차단 대기
- **FOR UPDATE NOWAIT**
 - 잠겨있으면 즉시 ERROR 반환 (대기 없음)
- **FOR UPDATE SKIP LOCKED**
 - 잠긴 행 건너뛰고 진행
→ 작업 큐 처리에 유용

Lock 유지 시간

- **COMMIT 또는 ROLLBACK** 시점에 자동 해제
- 트랜잭션이 길수록 Lock 유지 시간 증가
→ 충돌 확률 증가
- 트랜잭션을 짧게 유지하는 것이 성능의 핵심

낙관적 Lock vs 비관적 Lock

낙관적 Lock (Optimistic)

- 충돌이 드물다고 가정
- 버전 컬럼으로 충돌 감지
- 저장 시 충돌 → 재시도
- 구현
 - **version INT** 또는 **updated_at**
 - **UPDATE ... WHERE version = :old**
 - **0행** = 충돌 → 재시도
- 적합
 - 읽기 많고 갱신 드문 시스템
 - 블로그, 설정 변경
 - 긴 사용자 세션

비관적 Lock (Pessimistic)

- 충돌이 빈번하다고 가정
- 즉시 행 잠금 획득
- 다른 트랜잭션은 대기
- 구현
 - **SELECT ... FOR UPDATE**
 - 잠금 보유 중 다른 TX 대기
 - **COMMIT** 시 잠금 해제
- 적합
 - 은행 이체, 좌석 예매
 - 재고 차감
 - 짧은 트랜잭션 (ms)

Row Lock 시나리오 및 SKIP LOCKED

【 SQL 예시 】

```
-- Row Lock 시나리오
-- Session 1
BEGIN;
UPDATE items SET stock = stock - 1 WHERE id = 1;
-- id=1 행에 Lock 획득, COMMIT 안 함

-- Session 2 (동시)
BEGIN;
UPDATE items SET stock = stock - 1 WHERE id = 1;
-- → 대기(blocking): Session 1이 COMMIT/ROLLBACK 할 때까지 기다림

-- Session 1이 COMMIT → Session 2 실행 재개
```

SELECT FOR UPDATE / NOWAIT / SKIP LOCKED

- ✓ **SKIP LOCKED** 패턴
 - 분산 작업 큐
 - 이메일 발송 Queue
 - 배치 처리 등에 필수 패턴

Row Lock 시나리오 및 SKIP LOCKED

【 SQL 예시 】

```
-- FOR UPDATE NOWAIT: 잠겨있으면 즉시 실패
BEGIN;
SELECT * FROM items WHERE id = 1 FOR UPDATE NOWAIT;
-- Session 1이 Lock 보유 중이면:
-- ERROR: could not obtain lock on row in relation "items"

-- SKIP LOCKED: 작업 큐 처리 (경쟁 소비자 패턴)
-- Worker 여러 개가 중복 없이 작업 분배
WITH picked AS (
  SELECT id FROM job_queue
  WHERE status = 'READY'
  ORDER BY id
  FOR UPDATE SKIP LOCKED -- 다른 Worker가 처리 중인 행은 건너뛴다
  LIMIT 10
)
UPDATE job_queue SET status = 'RUNNING', started_at = now()
FROM picked WHERE job_queue.id = picked.id
RETURNING job_queue.*;
```

SELECT FOR UPDATE / NOWAIT / SKIP LOCKED

- ✓ **SKIP LOCKED 패턴**
 - 분산 작업 큐
 - 이메일 발송 Queue
 - 배치 처리 등에 필수 패턴

Deadlock 발생 구조 및 자동 감지

【 SQL 예시 】

```
-- Deadlock 발생 시나리오 (교차 대기)
-- Session 1
BEGIN;
UPDATE items SET stock = stock - 1 WHERE id = 1; -- row 1 Lock 획득

-- Session 2 (동시)
BEGIN;
UPDATE items SET stock = stock - 1 WHERE id = 2; -- row 2 Lock 획득

-- Session 1 계속
UPDATE items SET stock = stock - 1 WHERE id = 2; -- row 2 요청 → Session 2 대기 중

-- Session 2 계속
UPDATE items SET stock = stock - 1 WHERE id = 1; -- row 1 요청 → Session 1 대기 중
```

Deadlock 발생 구조 및 자동 감지

【 SQL 예시 】

```
-- → 순환 대기! PostgreSQL이 자동으로 감지
-- ERROR: deadlock detected
-- DETAIL: Process 123 waits for ShareLock on transaction 456;
-- blocked by process 456.
-- Process 456 waits for ShareLock on transaction 123;
-- blocked by process 123.
-- HINT: See server log for query details.

-- 한 쪽이 자동으로 ROLLBACK됨 → 다른 쪽이 계속 진행
-- deadlock_timeout = 1s (기본 감지 대기 시간)
```

Deadlock 해소 — 순서 통일 + Advisory Lock

【 SQL 예시 】

```
-- 해결 1: 항상 동일 순서로 Lock 획득 (id ASC)
BEGIN;
WITH targets AS (
SELECT id FROM items
WHERE id = ANY (ARRAY[1, 2])
ORDER BY id -- ← 핵심! 항상 id 오름차순으로 잠금
FOR UPDATE
)
UPDATE items SET stock = stock - 1
FROM targets WHERE items.id = targets.id;
COMMIT;

-- 해결 2: 트랜잭션 짧게 유지
-- - 비즈니스 로직, HTTP 호출, 파일 I/O는 트랜잭션 밖에서 수행
-- - 꼭 필요한 테이블/행만 접근
-- - 격리 수준 READ COMMITTED 유지
```

Deadlock 해소 — 순서 통일 + Advisory Lock

【 SQL 예시 】

```
-- 해결 3: Advisory Lock (업무 개념 단위 직렬화)
BEGIN;
SELECT pg_advisory_xact_lock(42); -- user_id=42에 대한 논리 락
UPDATE users SET balance = balance - 1000 WHERE id = 42;
INSERT INTO payments (user_id, amount) VALUES (42, 1000);
COMMIT; -- 트랜잭션 종료 시 Advisory Lock 자동 해제

-- 해결 4: 재시도 전략 (애플리케이션 레벨)
-- Deadlock 에러 코드: 40P01 (PostgreSQL)
-- → 짧은 백오프(지수 증가) 후 재시도
```

Deadlock 모니터링 — 대기 중인 세션 확인

【 SQL 예시 】

-- 현재 Lock 대기 중인 세션 전체 보기

```
SELECT  
bl.pid AS waiting_pid,  
wl.pid AS blocking_pid,  
bl.query AS waiting_query,  
wl.query AS blocking_query,  
now() - bl.query_start AS waiting_for  
FROM pg_catalog.pg_locks l1  
JOIN pg_catalog.pg_stat_activity bl ON bl.pid = l1.pid  
JOIN pg_catalog.pg_locks l2  
ON l1.locktype = l2.locktype  
AND l1.database IS NOT DISTINCT FROM l2.database  
AND l1.relation IS NOT DISTINCT FROM l2.relation  
AND l1.pid <> l2.pid  
JOIN pg_catalog.pg_stat_activity wl ON wl.pid = l2.pid  
WHERE NOT l1.granted AND l2.granted  
ORDER BY waiting_for DESC;
```

-- 핵심 파라미터 설정 (postgresql.conf)

-- deadlock_timeout = 200ms (감지 전 대기, 기본 1s)

-- log_lock_waits = on (락 대기 로그 기록)

-- log_min_error_statement = error (에러 SQL 기록)

-- 강제 종료

SELECT pg_cancel_backend(pid); -- 현재 쿼리만 취소 (graceful)

SELECT pg_terminate_backend(pid); -- 연결 자체 종료 (강제)

데드락 모니터링 & 재시도 전략

```
-- 잠금 대기 세션 확인
SELECT bl.pid AS waiting_pid, wl.pid AS blocking_pid,
bl.query AS waiting_query, wl.query AS blocking_query,
now() - bl.query_start AS waiting_for
FROM pg_catalog.pg_locks l1
JOIN pg_catalog.pg_stat_activity bl ON bl.pid = l1.pid
JOIN pg_catalog.pg_locks l2
ON l1.locktype = l2.locktype AND l1.relation IS NOT DISTINCT FROM l2.relation
AND l1.pid <> l2.pid
JOIN pg_catalog.pg_stat_activity wl ON wl.pid = l2.pid
WHERE NOT l1.granted AND l2.granted
ORDER BY waiting_for DESC;

-- 30초 이상 대기 중인 세션
SELECT pid, state, wait_event, now()-query_start AS duration, query
FROM pg_stat_activity
WHERE state != 'idle' AND now()-query_start > INTERVAL '30 seconds';

-- 세션 강제 종료
SELECT pg_cancel_backend(pid); -- 현재 쿼리만 취소
SELECT pg_terminate_backend(pid); -- 연결 완전 종료

-- postgresql.conf 권장 설정
-- log_lock_waits = on (락 대기 로그 기록)
-- deadlock_timeout = 500ms (감지 주기 단축)
-- 앱 레벨 재시도 (Python 예시)
-- try: execute_tx()
-- except psycopg2.errors.DeadlockDetected: # 에러코드 40P01
-- time.sleep(random.uniform(0.1, 0.5)) # Jitter로 충돌 방지
-- retry()
```

Deadlock 제어 — DBMS별 방법 비교

DBMS	자동 감지	예방 방법	해소 명령	로그/모니터링
PostgreSQL	O 자동 (deadlock_timeout)	• 순서 통일 • 짧은 트랜잭션 • Advisory Lock	pg_terminate_backend(pid)	pg_locks pg_stat_activity
MySQL/InnoDB	O 자동	• 순서 통일 • Row-Level Lock	KILL <process_id>	SHOW ENGINE INNODB STATUS
Oracle	O 자동	• 순서 통일 • 짧은 트랜잭션	ALTER SYSTEM KILL SESSION	V\$LOCK, V\$SESSION
SQL Server	O 자동 (Deadlock Monitor)	• DEADLOCK_PRIORITY 설정	KILL <spid>	sys.dm_tran_lo cks XML Deadlock
Cloud/분산	O 분산 그래프 탐지	• 중앙 Lock 서버	Cloud UI/API	분산 Lock Graph

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

22. 고급 DB 설계

- BCNF·4NF·5NF · 반정규화 · SCD · 샤딩

BCNF (Boyce-Codd Normal Form) 상세

함수 종속성 분석 → 후보키 파악 → 결정자 검사

조건

- 모든 결정자(Determinant)가 후보키(Candidate Key)
- 어떤 함수 종속 $X \rightarrow Y$ 가 있을 때 X 가 후보키가 아니면 BCNF 위반

BCNF 위반 예시

- 학습지원(교수, 과목, 강의실)
— 교수→강의실
(교수는 항상 같은 강의실 사용)
- 교수는 후보키가 아닌데
강의실을 결정 → BCNF 위반

해결

- 교수강의실(교수, 강의실)
+ 강의배정(교수, 과목)
두 테이블로 분리

BCNF (Boyce-Codd Normal Form) 상세

3NF vs BCNF 차이

- **3NF** : 함수 종속 보존 우선 (비후보 결정자 허용 가능)
- **BCNF** : 정합성 우선 (모든 결정자 = 후보키 엄격 적용)
- 실무 : 3NF까지 적용 → JOIN 비용·관리 복잡도 고려

정규화 심화 — 4NF / 5NF 비교

정규형	해결 문제	핵심 개념	예시	실무 적용
BCNF	비후보키 결정자	모든 결정자 = 후보키	교수 > 강의실 (교수가 후보키가 아님)	3NF 이후 추가 검토
4NF	다치 종속	독립적인 다중값 분리	직원>자격증, 직원>외국어(독립적)	직원_자격증 +직원_외국어로 분리
5NF	조인 종속	분해 후 조인 시 원본을 정확하게 복원	프로젝트-직원-역할 3개의 테이블 조합	이론적으로는 중요하나 실무에서는 많이...

정규화 vs 반정규화 선택

정규화 유지 (OLTP)

- 쓰기(INSERT/UPDATE) 중심
- 데이터 무결성 최우선
- 이상(Anomaly) 현상 방지
- 적합 시나리오
 - 은행 계좌 관리
 - 주문 처리 시스템
 - 사용자 등록·변경
- 3NF ~ BCNF까지 적용
- 성능 문제 → 인덱스로 해결

반정규화 허용 (OLAP)

- 읽기(집계 SELECT) 중심
- JOIN 비용 감소 우선
- 중복 허용 (동기화 필요)
- 적합 시나리오
 - 대시보드, 보고서
 - 분석 쿼리 최적화
 - AI/ML 피처 테이블
- 기법: 중복 컬럼·집계 컬럼 추가
- 안전 대안: **Materialized View**

SCD (Slowly Changing Dimension) — 이력 데이터 관리

【 SQL 예시 】

-- 이력 데이터 관리 전략

-- SCD Type 1: 덮어쓰기 (이력 없음)

```
UPDATE customers SET city = '부산' WHERE id = 42;
```

-- 이전 도시 정보 사라짐 → 이력 추적 불가

-- SCD Type 2: 이력 보존 (새 행 추가, 기간 관리)

```
CREATE TABLE customer_history (  
  id BIGSERIAL PRIMARY KEY,  
  customer_id INT NOT NULL,  
  name TEXT NOT NULL,  
  city TEXT NOT NULL,  
  valid_from DATE NOT NULL,  
  valid_to DATE, -- NULL = 현재 유효  
  is_current BOOLEAN DEFAULT TRUE  
);
```

Checkpoint

- ✓ SCD Type 3
 - 현재+이전값
별도 컬럼으로 보존
- ✓ SCD Type 6
 - Type 1+2+3 혼합

SCD (Slowly Changing Dimension) — 이력 데이터 관리

【 SQL 예시 】

-- 이력 데이터 관리 전략

-- 변경 발생 시: 기존 행 종료 + 새 행 삽입

```
UPDATE customer_history  
SET valid_to = CURRENT_DATE - 1, is_current = FALSE  
WHERE customer_id = 42 AND is_current = TRUE;
```

```
INSERT INTO customer_history (customer_id, name, city, valid_from)  
VALUES (42, '홍길동', '부산', CURRENT_DATE);
```

-- 현재 데이터 조회

```
SELECT * FROM customer_history WHERE is_current = TRUE AND customer_id = 42;
```

-- 특정 시점 데이터 조회 (2023-06-01 기준)

```
SELECT * FROM customer_history  
WHERE customer_id = 42  
AND valid_from <= '2023-06-01' AND (valid_to IS NULL OR valid_to >= '2023-06-01');
```

Checkpoint

- ✓ **SCD Type 3**
 - 현재+이전값
별도 컬럼으로 보존
- ✓ **SCD Type 6**
 - Type 1+2+3 혼합

샤딩(Sharding) — 수평 분할 전략

샤딩 : 데이터를 여러 DB 인스턴스에 수평으로 분산 저장 (파티셔닝의 확장)

샤딩 vs 파티셔닝

- 파티셔닝
 - 단일 DB 인스턴스 내에서 논리/물리 분할
- 샤딩
 - 여러 DB 인스턴스에 걸쳐 데이터 분산
 - 수평 확장(Scale-out)

주요 샤딩 전략

- Range 샤딩
 - 사용자 ID 범위 기준
 - : 1~1000만 → Shard1
 - : 1000만+ → Shard2
- Hash 샤딩
 - $\text{user_id} \% N$ 으로 균등 분산
 - (균형적이지만 이관 어려움)
- Directory 샤딩
 - 샤드 맵 테이블로 관리
 - (유연하지만 단일 장애점)

샤딩(Sharding) — 수평 분할 전략

샤딩 : 데이터를 여러 DB 인스턴스에 수평으로 분산 저장 (파티셔닝의 확장)

샤딩 구현 도구

- **Citus (PostgreSQL)**
 - 분산 쿼리, 자동 샤딩, SQL 그대로 사용
- **Vitess (MySQL)**
 - YouTube 출신, 오픈소스 MySQL 샤딩
- **TiDB**
 - MySQL 호환 분산 NewSQL (Google Spanner 영감)

샤딩 주의사항

- **Cross-shard JOIN 불가**
(앱 레벨에서 처리 필요)
- 분산 트랜잭션 복잡
(2PC 필요)
- 샤딩 전 파티셔닝·Read Replica로 충분한지 먼저 검토

샤딩 & CDC & MSA DB 패턴

샤딩 (Sharding)

- Hash 샤딩
 - `user_id % n`
 - 샤드 번호 결정 (균등 분산)
- Range 샤딩
 - ID 범위별 분산
 - (hot spot 가능성)
- Geo 샤딩
 - 지역별 분산 (KR → KR 샤드)
- 어려움
 - Cross-shard JOIN 불가
 - 분산 트랜잭션 복잡
 - Re-sharding 어려움
- 대안
 - 파티셔닝(단일 DB 내 분할)
 - Read Replica
 - Cloud DB 자동 확장

CDC (Change Data Capture)

- DB 변경을 실시간 이벤트로 스트리밍
- PostgreSQL Logical Decoding + wal2json
 - Kafka
- MySQL Binlog
 - Debezium
 - Kafka
- 활용
 - 실시간 동기화
 - DW 적재
 - 마이크로서비스 이벤트 전파

MSA Database per Service 패턴

- 각 서비스가 독립적인 DB 소유
 - 서비스별 독립 배포 가능
- Saga 패턴
 - 분산 트랜잭션을 로컬 TX 체인으로 처리
 - + 보상 트랜잭션
- CQRS
 - 읽기(Query) DB와 쓰기(Command) DB 분리

MSA DB 패턴 — Saga / CQRS / Outbox

Database per Service 원칙

- 각 마이크로서비스가 독립적인 DB 소유
- 장점
 - 서비스별 독립 배포
 - 기술 선택 자유
 - 장애 격리
- 단점
 - 분산 트랜잭션 복잡
 - 데이터 일관성 유지 어려움

MSA DB 패턴 — Saga / CQRS / Outbox

Saga 패턴

- 분산 트랜잭션을 로컬 TX 체인으로 처리
- 주문→결제→재고→배송 각 서비스
 - 로컬 트랜잭션 + 이벤트 발행
- 실패 시 보상 트랜잭션 (Compensating TX) 실행
- **Choreography**
 - 이벤트 기반 자율 조정
- **Orchestration**
 - 중앙 조정자

CQRS (Command Query Responsibility Segregation)

- 읽기(Query)와 쓰기(Command) 모델 분리
- 쓰기
 - RDBMS (정규화, ACID)
- 읽기
 - 비정규화 캐시
 - Read Replica

Outbox 패턴

- 트랜잭션 내 이벤트 발행 보장
- 메인 로직 + 아웃박스 테이블 **INSERT**
→ 같은 트랜잭션
- **CDC(Debezium)**
→ **Kafka**
→ 소비자 서비스
→ 최소 1회 전달 보장

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

[참고] 심화 & 운영 실무

- 백업/복구 · VACUUM · 모니터링 · 연결 풀링

[참고] PostgreSQL 백업 및 복구

【 SQL 예시 】

```
-- 논리 백업 (SQL 형태)
pg_dump -U postgres -d mydb -f backup.sql # 단순 SQL
pg_dump -U postgres -d mydb -Fc -f backup.dump # 압축 바이너리 (권장)
pg_dump -U postgres -d mydb -Fd -j 4 -f backup_dir/ # 병렬 (4 코어)

-- 전체 클러스터 백업 (전역 객체 포함)
pg_dumpall -U postgres > full_cluster_backup.sql

-- 복구
psql -U postgres -d mydb < backup.sql
pg_restore -U postgres -d mydb -j 4 backup.dump # 병렬 복구
```

Checkpoint

- ✓ 백업 원칙
 - 3-2-1
 - 3 copies / 2 media / 1 offsite) / 정기 복원
- ✓ 테스트 필수!

[참고] PostgreSQL 백업 및 복구

【 SQL 예시 】

```
-- 물리 백업 (PITR용)
pg_basebackup -U replicator -D /backup/base -Ft -z -P --wal-method=stream

-- PITR 설정 (postgresql.conf)
wal_level = replica
archive_mode = on
archive_command = 'cp %p /backup/wal/%f'

-- 복구 목표 시점 지정 (postgresql.conf)
restore_command = 'cp /backup/wal/%f %p'
recovery_target_time = '2024-12-15 14:30:00'

-- 자동 백업 스크립트 (crontab)
# 매일 새벽 3시
0 3 * * * pg_dump -U postgres -Fc -d mydb > /backup/daily-$(date +%F).dump
```

Checkpoint

- ✓ 백업 원칙
 - 3-2-1
 - 3 copies / 2 media / 1 offsite)
 - / 정기 복원
- ✓ 테스트 필수!

[참고] VACUUM & Table Bloat 관리

Table Bloat

- **Dead tuple**이 쌓여 테이블이 실제보다 커지는 현상
- **UPDATE**
= 새 버전 삽입 + 구 버전을 **Dead tuple**로 표시
- **Dead tuple**은 공간을 차지하고 순차 스캔 속도 저하

VACUUM 종류

- **VACUUM**
 - **Dead tuple** 정리
 - 공간 재사용 표시 (테이블 크기는 안 줄어듦)
- **VACUUM FULL**
 - 테이블 완전 재작성
→ 실제 크기 감소 (배타적 잠금 주의)
- **VACUUM FREEZE**
 - Transaction ID Wraparound 방지
- **VACUUM ANALYZE**
 - **Dead tuple** 정리 + 통계 갱신 동시에

[참고] VACUUM & Table Bloat 관리

autovacuum 설정 최적화

- **autovacuum_vacuum_scale_factor = 0.05 (5% dead → VACUUM 실행)**
- **autovacuum_analyze_scale_factor = 0.02 (2% 변경 → ANALYZE 실행)**
- 테이블별 오버라이드
 - **ALTER TABLE orders**
SET (
 autovacuum_vacuum_scale_factor
 = 0.01)

Bloat 확인

- **SELECT n_dead_tup, n_live_tup**
FROM pg_stat_user_tables
WHERE relname='orders'

[참고] VACUUM — Table Bloat 관리

【 SQL 예시 】

```
-- Table Bloat 확인 (PostgreSQL)
SELECT
  relname AS table_name,
  pg_size_pretty(pg_total_relation_size(relname::regclass)) AS total_size,
  n_dead_tup AS dead_tuples,
  n_live_tup AS live_tuples,
  ROUND(n_dead_tup::NUMERIC / NULLIF(n_live_tup + n_dead_tup, 0) * 100, 1)
  AS dead_pct,
  last_autovacuum,
  last_autoanalyze
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC LIMIT 10;
```

Checkpoint

- ✓ **Table Bloat 주의**
 - 장수 트랜잭션
(Long-running TX)은
VACUUM 차단
→ **pg_stat_activity**
모니터링

[참고] VACUUM — Table Bloat 관리

【 SQL 예시 】

```
-- 수동 VACUUM (운영 중에도 실행 가능, 잠금 없음)
VACUUM orders; -- Dead tuple 회수 (잠금 없음, 공간 미반환)
VACUUM ANALYZE orders; -- Dead tuple + 통계 갱신
VACUUM FULL orders; -- 완전 재작성 (배타적 잠금, 공간 반환)
VACUUM FREEZE orders; -- TX ID Wraparound 방지

-- 테이블별 autovacuum 설정 오버라이드
ALTER TABLE orders SET (
  autovacuum_vacuum_scale_factor = 0.01, -- 1% dead tuple 시 실행
  autovacuum_analyze_scale_factor = 0.005 -- 0.5% 변경 시 통계 갱신
);

-- Dead tuple 없이 인덱스 공간만 회수
REINDEX INDEX CONCURRENTLY idx_orders_customer;

-- Transaction ID Wraparound 모니터링 (2^31 = 약 21억 트랜잭션)
SELECT relname, age(relfrozenxid) AS xid_age,
       pg_size_pretty(pg_total_relation_size(oid)) AS size
FROM   pg_class WHERE relkind = 'r' AND age(relfrozenxid) > 100000000
ORDER BY age(relfrozenxid) DESC;
```

Checkpoint

- ✓ **Table Bloat 주의**
 - 장수 트랜잭션
(Long-running TX)은
VACUUM 차단
→ **pg_stat_activity**
모니터링

[참고] Connection Pool — pgBouncer 설정

DB 연결 비용

- 연결당 ~5~10MB 메모리 소모
 - 연결 핸드셰이크 비용
- 1000개 연결 × 5MB = 5GB 메모리
→ max_connections 한계

pgBouncer 모드

- **Session Mode**
 - 클라이언트 연결 동안 DB 연결 유지
(안전, 효율 낮음)
- **Transaction Mode**
 - 트랜잭션 단위로 DB 연결 할당/반환
(권장, 가장 효율적)
- **Statement Mode**
 - 문장 단위
(PREPARE 불가, 거의 미사용)

[참고] Connection Pool — pgBouncer 설정

ProxySQL (MySQL), RDS Proxy (AWS), Cloud SQL Proxy (GCP)

pgBouncer 모드

- **Session Mode**
 - 클라이언트 연결 동안 **DB** 연결 유지
(안전, 효율 낮음)
- **Transaction Mode**
 - 트랜잭션 단위로 **DB** 연결 할당/반환
(권장, 가장 효율적)
- **Statement Mode**
 - 문장 단위
(**PREPARE** 불가, 거의 미사용)

주요 설정 파라미터

- **max_client_conn = 2000**
(앱에서 pgBouncer에 연결되는 최대 수)
- **default_pool_size = 20**
(pgBouncer → PostgreSQL 실제 연결 수)
- **pool_mode = transaction**
- **server_idle_timeout = 600**
(유휴 DB 연결 유지 시간 초)

[참고] Connection Pool — pgBouncer 설정

모니터링: pgBouncer **SHOW POOLS; SHOW CLIENTS; SHOW SERVERS;**

pgBouncer 모드

- **Session Mode**
 - 클라이언트 연결 동안 **DB** 연결 유지
(안전, 효율 낮음)
- **Transaction Mode**
 - 트랜잭션 단위로 **DB** 연결 할당/반환
(권장, 가장 효율적)
- **Statement Mode**
 - 문장 단위
(**PREPARE** 불가, 거의 미사용)

주요 설정 파라미터

- **max_client_conn = 2000**
(앱에서 pgBouncer에 연결되는 최대 수)
- **default_pool_size = 20**
(pgBouncer → PostgreSQL 실제 연결 수)
- **pool_mode = transaction**
- **server_idle_timeout = 600**
(유휴 DB 연결 유지 시간 초)

[참고] Connection Pool — pgBouncer 설정 가이드

【 SQL 예시 】

```
-- DB 연결 비용과 Connection Pool 필요성
-- PostgreSQL: 연결당 약 5MB 메모리 + 핸드셰이크 비용
-- 1000개 연결 = 5GB 메모리 소모 → max_connections 초과 시 연결 거부

-- pgBouncer 주요 설정 (pgbouncer.ini)
[databases]
skala_db = host=127.0.0.1 port=5432 dbname=skala_db

[pgbouncer]
pool_mode = transaction -- 권장: 트랜잭션 단위 연결 할당
max_client_conn = 2000 -- 앱 측 최대 연결 수
default_pool_size = 20 -- DB에 실제 연결되는 수 (20개)
min_pool_size = 5
reserve_pool_size = 5
server_idle_timeout = 600
```

Checkpoint

- ✓ **pgBouncer Transaction Mode**
 - 20개 DB 연결로
 - 2000개 앱 연결 처리
 - 연결 오버헤드 감소

[참고] Connection Pool — pgBouncer 설정 가이드

【 SQL 예시 】

```
-- DB 연결 비용과 Connection Pool 필요성
-- PostgreSQL: 연결당 약 5MB 메모리 + 핸드셰이크 비용
-- 1000개 연결 = 5GB 메모리 소모 → max_connections 초과 시 연결 거부

-- 3가지 Pool 모드 비교:
-- Session Mode: 클라이언트 세션 동안 DB 연결 유지 (가장 안전)
-- Transaction Mode: 트랜잭션 단위로 연결 할당/반환 (권장 — 효율 최대)
-- Statement Mode: 쿼리 단위 (PREPARE 사용 불가 — 거의 미사용)

-- pgBouncer 상태 확인
-- psql -p 6432 pgbouncer -c "SHOW POOLS;"
-- psql -p 6432 pgbouncer -c "SHOW STATS;"
```

Checkpoint

- ✓ **pgBouncer Transaction Mode**
 - 20개 DB 연결로
 - 2000개 앱 연결 처리
 - 연결 오버헤드 감소

[참고] DB 모니터링 핵심 쿼리 모음

【 SQL 예시 】

-- 현재 실행 중인 쿼리 (30초 이상)

```
SELECT pid, now() - query_start AS duration,  
state, wait_event, LEFT(query, 100) AS query_snippet  
FROM pg_stat_activity  
WHERE state != 'idle'  
AND now() - query_start > INTERVAL '30 seconds'  
ORDER BY duration DESC;
```

-- 연결 수 현황

```
SELECT state, COUNT(*) AS cnt FROM pg_stat_activity GROUP BY state;
```

[참고] DB 모니터링 핵심 쿼리 모음

【 SQL 예시 】

-- 캐시 히트율 (권장: 99% 이상)

```
SELECT  
SUM(blks_hit)::FLOAT / NULLIF(SUM(blks_hit) + SUM(blks_read), 0) * 100  
AS cache_hit_pct  
FROM pg_stat_database WHERE datname = current_database();
```

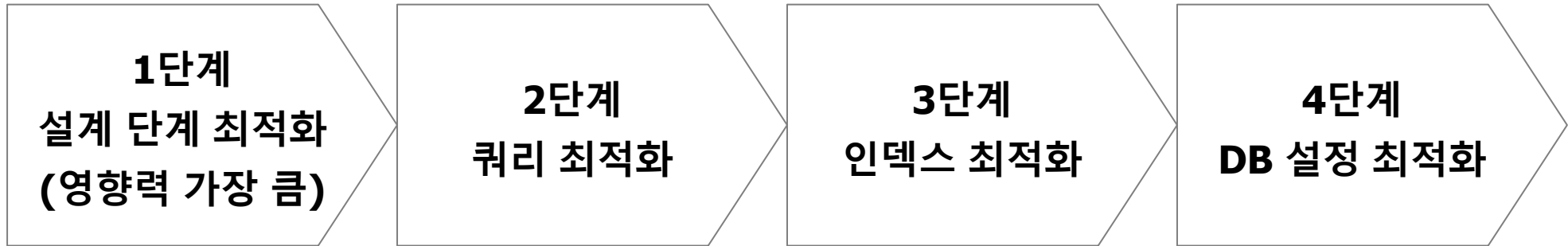
-- 테이블 크기 순위

```
SELECT relname, pg_size_pretty(pg_total_relation_size(relname::regclass)) AS size  
FROM pg_stat_user_tables ORDER BY pg_total_relation_size(relname::regclass) DESC  
LIMIT 10;
```

-- 인덱스 미사용 (idx_scan = 0)

```
SELECT indexrelname, relname, pg_size_pretty(pg_relation_size(indexrelid))  
FROM pg_stat_user_indexes WHERE idx_scan = 0 AND relname NOT LIKE 'pg_%'  
ORDER BY pg_relation_size(indexrelid) DESC;
```

DB 성능 최적화 4단계 접근법



DB 성능 최적화 4단계 접근법

1단계 — 설계 단계 최적화 (영향력 가장 큼)

- 적절한 정규화 수준 결정
(OLTP: 3NF / OLAP: 반정규화)
- PK 선택
 - 자연키 vs 대체키(BIGINT/UUID)
 - UUID는 인덱스 효율↓
- FK 컬럼에 반드시 인덱스 생성
- 데이터 타입 최적화
 - NUMERIC(금액)
 - TIMESTAMPTZ(시각)

2단계 — 쿼리 최적화

- **SELECT *** 제거
- 서브쿼리→**CTE/JOIN** 변환
- **N+1** 문제 해결
- **EXPLAIN ANALYZE**로 병목 파악

DB 성능 최적화 4단계 접근법

3단계 — 인덱스 최적화

- 선택도 높은 컬럼 우선, 복합 인덱스 컬럼 순서
- 커버링 인덱스, 부분 인덱스, 함수 기반 인덱스
- 미사용 인덱스 제거 (`idx_scan = 0`)

4단계 — DB 설정 최적화

- 메모리
 - `shared_buffers`(DB 캐시)
 - `work_mem`(정렬/해시 `per-query`)
- Connection Pool (pgBouncer)
- `VACUUM/ANALYZE` 스케줄

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

23. 종합실습 - 3

- HR DB 느린 쿼리 최적화 — EXPLAIN Before/After 비교

종합 실습 - HR DB 느린 쿼리 최적화 방향

학습 목표

- 인덱스 없는 **Seq Scan**
→ 인덱스 추가 후 **Index Scan** 전환 체험
- **pg_stat_statements**로
실제 느린 쿼리 식별
- **EXPLAIN ANALYZE Before/After**
결과 비교 리포트
- **Deadlock** 시나리오 실습 및 해소

사전 준비

- **PostgreSQL** 스크립트 실행
종합실습3_HR DB_PostgreSQL 실습스크립트_환경설정.sql

종합실습 1 환경 활용

HR 스키마 개설

실행계획 확인 및 수정

종합 실습 - HR DB 느린 쿼리 최적화 배경 및 실습 과제

배경

- HR 데이터베이스를 사용하는 인사 관리 시스템
 - 최근 직원 수가 5만 명 이상으로 늘면서
검색 속도가 느려지고 보고서 생성시간이
지연됨

【 데이터 확인 및 실습과제 】

- `SELECT COUNT(*) FROM employees;`

【 제출 자료 】

- ✓ 리포트로 느린 이유, 그에 따른 개선 방법 도출,
개선 후 주요 효과에 대해 실행계획 이전/후
결과 정리를 자세히 할 것
- ✓ 최소 2개 이상 쿼리를 만들고 쿼리에 대한 차이점
을 분석 결과를 작성할 것

종합 실습 - HR DB 느린 쿼리 최적화 방향 실습 내용

1. 실행 계획 맞보기 (사번이 100인 사람 검색)

2. 인덱스 없는 느린 쿼리 작성 후 쿼리 튜닝
(이메일 주소가 user1234@corp.com인 사람, lower(email) 사용)

3. LIKE 검색 쿼리 작성 후 쿼리 튜닝
(대상 : '%gmail.com' 같은 접미사 검색)

종합 실습 - HR DB 느린 쿼리 최적화 방향 실습 내용

4. 정렬(ORDER BY)과 필터가 결합된 쿼리 작성 후 쿼리 튜닝
(hire_date >= CURRENT_DATE - INTERVAL '365 days' 이면서
재직중인 사람을 연봉순으로 상위 100명만 출력)

5. OR 조건 쿼리 작성 후 쿼리 튜닝
(조건 : 부서코드가 10 또는 직무가 3, 4, 5 안에 있는 사람 검색)

종합 실습 - 제출물

실습 제출물
(당일 제출 원칙)

SQL Query 및 결과

- 각 문항별 쿼리와 실행결과 화면
(쿼리와 실행결과가 나온 화면 Screenshot)

제출 방법 및 배점

- Slack 내 댓글 제출 (제출기한 - 4일차 과정 시작 전)
- 문항 당 20점
- 채점 기준
 - 요구사항 반영 여부 + Query 내 컬럼 선택 (불필요한 컬럼 선택 확인)

스마트 데이터의 이해 및 활용

24. Stored Procedure & 함수

25. Trigger & 이벤트 처리

26. Cloud DB 개요

27. 서버리스 & 분산 Cloud DB

28. 데이터 웨어하우스 & 분석 DB

29. 현재의 트렌드

30. 보안 및 권한 관리

31. 백업·복구 & 고가용성

32. 모니터링 및 운영

33. 종합실습 - 4

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

24. Stored Procedure & 함수

- 4개 DBMS 문법 · DETERMINISTIC/IMMUTABLE · 보안 모델 · 실무 포인트

Stored Procedure vs 함수 비교

구분	Stored Procedure	함수 (Function, UDF)
역할	여러 SQL문을 묶어 한 번에 실행 (작업 절차)	계산해서 결과값을 돌려주는 함수
리턴값	있어도 되고 없어도 되고	반드시 1개 이상 결과 반환
호출 방법	CALL or EXEC	SELECT fn_함수() 형태
주 사용처	데이터 변경, 트랜잭션, 배치 작업	SELECT 안에서 계산, 데이터 변환
트랜잭션 제어	가능 (BEGIN/COMMIT/ROLLBACK)	불가능 (DML 금지)
부작용(DML)	INSERT, UPDATE, DELETE 가능	대부분 읽기 전용 (변경불가)
핵심 비유	작업 단위 (batch 작업 중심)	계산 도구 (연산기)

함수 예시 — 4개 DBMS (부가세 10% 계산)

【 SQL 예시 】

-- MySQL / MariaDB

```
CREATE FUNCTION fn_vat(amount DECIMAL(10,2))
RETURNS DECIMAL(10,2) DETERMINISTIC -- 같은 입력 = 항상 같은 결과
BEGIN RETURN amount * 0.1; END;
SELECT order_id, fn_vat(total_amount) AS vat FROM orders;
```

-- PostgreSQL

```
CREATE OR REPLACE FUNCTION fn_vat(amount NUMERIC)
RETURNS NUMERIC LANGUAGE sql IMMUTABLE -- IMMUTABLE: 인덱스 표현식에도 사용 가능
AS $$ SELECT amount * 0.1 $$;
CREATE INDEX idx_tax ON orders ((fn_vat(total_amount))); -- 함수 기반 인덱스 가능!
```

-- SQL Server (인라인 테이블 함수 — 가장 빠름)

```
CREATE FUNCTION dbo.fn_vat(@amount DECIMAL(10,2))
RETURNS TABLE AS RETURN (SELECT @amount * 0.1 AS tax);
```

-- Oracle

```
CREATE OR REPLACE FUNCTION fn_vat(p_amount NUMBER)
RETURN NUMBER RESULT_CACHE DETERMINISTIC -- 결과 메모리 캐시
IS BEGIN RETURN p_amount * 0.1; END;
/
```

Checkpoint

- ✓ 키워드 요약
- ✓ **MySQL**
 - DETERMINISTIC
- ✓ **PostgreSQL**
 - IMMUTABLE
 - STABLE
 - VOLATILE
- ✓ **Oracle**
 - RESULT_CACHE

Stored Procedure 예시 — 4개 DBMS (연도별 매출 상위 5명)

【 SQL 예시 】

-- MySQL

```
DELIMITER $$  
CREATE PROCEDURE sp_top_customers(IN p_year INT)  
BEGIN  
SELECT c.customer_id, SUM(o.total_amount) AS total_sales  
FROM orders o JOIN customers c ON o.customer_id = c.customer_id  
WHERE YEAR(o.order_date) = p_year  
GROUP BY c.customer_id ORDER BY total_sales DESC LIMIT 5;  
END$$  
DELIMITER ;  
CALL sp_top_customers(2024);
```

-- PostgreSQL

```
CREATE OR REPLACE PROCEDURE sp_top_customers(p_year INT) LANGUAGE plpgsql AS $$  
BEGIN  
SELECT c.customer_id, SUM(o.total_amount)  
FROM orders o JOIN customers c ON c.customer_id = o.customer_id  
WHERE EXTRACT(YEAR FROM o.order_date) = p_year  
GROUP BY c.customer_id ORDER BY 2 DESC LIMIT 5;  
END; $$;  
CALL sp_top_customers(2024);
```


Stored Procedure 예시 — 4개 DBMS (연도별 매출 상위 5명)

【 SQL 예시 】

```
-- SQL Server
CREATE PROCEDURE sp_top_customers @year INT
AS
BEGIN
SELECT TOP 5 c.customer_id, SUM(o.total_amount) AS total_sales
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
WHERE YEAR(o.order_date) = @year
GROUP BY c.customer_id
ORDER BY total_sales DESC;
END;
GO
```

Stored Procedure 예시 — 4개 DBMS (연도별 매출 상위 5명)

【 SQL 예시 】

-- Oracle

```
CREATE OR REPLACE PROCEDURE sp_top_customers(p_year NUMBER) AS
BEGIN
  FOR r IN (
    SELECT customer_id, SUM(total_amount) total_sales
    FROM orders
    WHERE EXTRACT(YEAR FROM order_date) = p_year
    GROUP BY customer_id
    ORDER BY total_sales DESC
    FETCH FIRST 5 ROWS ONLY
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(r.customer_id || ': ' || r.total_sales);
  END LOOP;
END;
/
```

보안 모델 — DEFINER vs INVOKER

DBMS	키워드	의미	주의사항
MySQL/MariaDB	SQL SECURITY DEFINER/INVOKER	- DEFINER : 작성자 권한 - INVOKER : 호출자 권한	- SQL Injection+DEFINER=권한 상승 위험 - search_path 고정 필수
PostgreSQL	SECURITY DEFINER/INVOKER	함수 소유자 권한으로 실행	DEFINER 함수는 Search_path 에 대한 하이재킹 주의
SQL Server	WITH EXECUTE AS OWNER/CALLER	- OWNER : 소유자 권한 - CALLER : 호출자	소유권 체이닝(dbo 통일)으로 단순화 권장
Oracle	AUTHID DEFINER/CURRENT_USER	기본 : 정의자 권한	PL/SQL에서 ROLE 아닌 직접 GRANT 권한만 유효

함수 성능 키워드 — DB별 최적화 힌트

자주 바뀌는 로직은 앱 코드, 공통 계산은 DB 함수, Cloud 환경에서 실행 시간 제한 주의

MySQL/MariaDB

- **DETERMINISTIC**
- 동일 입력 = 항상 같은 결과 선언
→ 복제(replication)/캐싱 시 안전
- 반대
 - **NOT DETERMINISTIC** (기본값)
 - **random(), NOW()** 포함 시

PostgreSQL

- 함수 안정성 3단계
 - **IMMUTABLE** : 같은 입력=항상 같은 결과
→ 인덱스 표현식·상수 폴딩 가능
 - **STABLE** : 같은 쿼리 안에서는 변하지 않음
(예: **SELECT COUNT(*) FROM ...**)
 - **VOLATILE**: 언제나 변할 수 있음 (기본값)
(예: **random(), now()**)

함수 성능 키워드 — DB별 최적화 힌트

자주 바뀌는 로직은 앱 코드, 공통 계산은 DB 함수, Cloud 환경에서 실행 시간 제한 주의

SQL Server

- 인라인 함수 vs 스칼라 함수
- 인라인 테이블 함수(iTVF)
 - 내부 SQL 그대로 펼쳐서 실행 → 빠름
- 스칼라 함수
 - 행마다 호출
 - 느림
 - (2019+부터 자동 인라이닝 일부 지원)

Oracle

- RESULT_CACHE
- 같은 입력이 여러 번 들어오면
결과를 메모리에 저장, 재사용

Stored Procedure 심화 — 에러 처리 패턴

예외 처리 (Exception Handling)

- PostgreSQL
 - EXCEPTION 블록으로 오류 포착
- MySQL
 - DECLARE ...
HANDLER FOR SQLEXCEPTION
- Oracle
 - EXCEPTION WHEN OTHERS THEN ...
- SQL Server
 - TRY...CATCH 블록

트랜잭션 롤백 패턴 (주문 생성 SP)

- 재고 확인 → 주문 헤더 **INSERT**
→ 주문 아이템 **INSERT** → 재고 **UPDATE**
- 재고 부족 시 전체 **ROLLBACK** + 에러 발생
- 모든 단계 성공 시 **COMMIT**

Stored Procedure 심화 — 에러 처리 패턴

커서(Cursor) 활용

- 결과 집합을 행 단위로 순회 처리
- 대용량 데이터는 **Set-based** 처리 권장
(커서는 느림)

SP vs 앱 코드 선택 기준

- **DB**
 - 공통 계산 로직
 - 트리거로 호출
 - 배치 작업
- **앱**
 - 자주 바뀌는 비즈니스 로직
 - 외부 **API** 연동
 - 복잡한 조건 분기

PostgreSQL SP — 주문 생성 (재고 부족 시 ROLLBACK)

```
CREATE OR REPLACE PROCEDURE sp_create_order(
  p_customer_id BIGINT,
  p_items JSONB -- [{"product_id":1,"qty":2}, ...]
) LANGUAGE plpgsql AS $$
DECLARE
  v_order_id BIGINT;
  v_item JSONB;
  v_stock INT;
  v_price NUMERIC;
BEGIN
  -- 1. 주문 헤더 생성
  INSERT INTO orders (customer_id, status) VALUES (p_customer_id, 'created')
  RETURNING id INTO v_order_id;

  -- 2. 각 아이템 처리
  FOR v_item IN SELECT * FROM jsonb_array_elements(p_items) LOOP
    -- 재고 확인 (FOR UPDATE로 동시성 처리)
    SELECT stock_qty, price INTO v_stock, v_price
    FROM products WHERE id = (v_item->>'product_id')::INT FOR UPDATE;

    IF v_stock < (v_item->>'qty')::INT THEN
      RAISE EXCEPTION '재고 부족: product_id=%', (v_item->>'product_id');
    END IF;
  END LOOP;
END;
```

```
-- 주문 아이템 삽입 + 재고 차감
INSERT INTO order_items (order_id, product_id, qty, unit_price)
VALUES (v_order_id, (v_item->>'product_id')::INT, (v_item->>'qty')::INT, v_price);

UPDATE products SET stock_qty = stock_qty - (v_item->>'qty')::INT
WHERE id = (v_item->>'product_id')::INT;
END LOOP;

UPDATE orders SET status = 'paid' WHERE id = v_order_id;
-- EXCEPTION 시 자동 ROLLBACK (PG의 기본 동작)
END;$$;

CALL sp_create_order(42,
'[{"product_id":1,"qty":2},{"product_id":3,"qty":1}]');
```


AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

25. Trigger & 이벤트 처리

- 감사 로그 · CDC · 배치 · DBMS별 비교 · 실행 순서 제어

Trigger 기본 개념 및 사용 시점

Trigger

- 특정 **DML/DDI** 이벤트 발생 시 **DB**가 자동 실행하는 코드
- 타이밍
 - **BEFORE / AFTER**
/ **INSTEAD OF**(뷰 전용)
- 범위
 - 행 단위
(**row-level, FOR EACH ROW**)
 - 문장 단위(**statement-level**)
- 가상 테이블
 - **OLD/NEW**
(**MySQL/PG/Oracle**)
 - **inserted/deleted**
(**SQL Server**)

Trigger가 잘 맞는 경우

- 감사 로그(**Audit**)
 - 누가 언제 무엇을 바꿨는지
자동 기록
- 기본값/유효성 보강
 - **created_at** 자동 세팅
 - 파생 컬럼 계산
- 비즈니스 규칙 보강
 - 재고 수량 음수 방지
- 소프트 삭제/버전 관리
 - 삭제 시 아카이브 테이블로 이동

피해야 할 경우

- 외부 연동
(**API 호출, 장시간 작업**)
→ 트랜잭션 연장, 장애 전파
- 순서 의존/상호참조 트리거
→ 유지보수 지옥
- 대량 **DML**에 **row** 트리거 남발
→ **statement** 트리거
또는 배치로 전환

Trigger 예시 — PostgreSQL (감사 로그 row-level)

【 SQL 예시 】

```
-- 감사 로그 테이블
CREATE TABLE audit_log(
  id BIGSERIAL PRIMARY KEY, table_name TEXT, op TEXT,
  row_json JSONB, at TIMESTAMPTZ DEFAULT now()
);

-- Row-level Trigger 함수
CREATE OR REPLACE FUNCTION trg_sales_ai() RETURNS trigger AS $$
BEGIN
  INSERT INTO audit_log(table_name, op, row_json)
  VALUES ('sales', TG_OP, to_jsonb(NEW));
  RETURN NEW;
END$$ LANGUAGE plpgsql;

CREATE TRIGGER sales_ai
AFTER INSERT ON sales
FOR EACH ROW EXECUTE FUNCTION trg_sales_ai();
```

Trigger 예시 — PostgreSQL (감사 로그 row-level)

【 SQL 예시 】

```
-- 감사 로그 테이블
CREATE TABLE audit_log(
  id BIGSERIAL PRIMARY KEY, table_name TEXT, op TEXT,
  row_json JSONB, at TIMESTAMPTZ DEFAULT now()
);

-- Statement-level + Transition table (대량 DML에 효율적)
CREATE OR REPLACE FUNCTION trg_sales_stmt_ai() RETURNS trigger AS $$
BEGIN
  INSERT INTO audit_log(table_name, op, row_json)
  SELECT 'sales', TG_OP, to_jsonb(n) FROM new_table AS n; -- Transition table
  RETURN NULL;
END$$ LANGUAGE plpgsql;

CREATE TRIGGER sales_stmt_ai AFTER INSERT ON sales
REFERENCING NEW TABLE AS new_table
FOR EACH STATEMENT EXECUTE FUNCTION trg_sales_stmt_ai();
```

Trigger 예시 — MySQL / SQL Server / Oracle 비교

【 SQL 예시 】

```
-- MySQL (row-level only, BEFORE/AFTER)
DELIMITER //
CREATE TRIGGER sales_bi BEFORE INSERT ON sales FOR EACH ROW
BEGIN SET NEW.created_at = COALESCE(NEW.created_at, NOW()); END//

CREATE TRIGGER sales_ai AFTER INSERT ON sales FOR EACH ROW
BEGIN
INSERT INTO audit_log(table_name, op, row_json)
VALUES ('sales','INSERT',JSON_OBJECT('id',NEW.id,'amount',NEW.amount));
END//
DELIMITER ;
-- 같은 테이블(sales) 수정 시 오류(1442)
```

Trigger 예시 — MySQL / SQL Server / Oracle 비교

【 SQL 예시 】

```
-- SQL Server (AFTER/INSTEAD OF, inserted/deleted 가상 테이블)
CREATE TRIGGER dbo.trg_sales_ai ON dbo.Sales AFTER INSERT AS BEGIN
SET NOCOUNT ON;
INSERT INTO dbo.AuditLog(table_name,op,row_json,at)
SELECT 'Sales','INSERT',(SELECT * FROM inserted FOR JSON PATH),SYSUTCDATETIME();
END;

-- Oracle (BEFORE row 기본값 + Compound Trigger로 Mutating Table 회피)
CREATE OR REPLACE TRIGGER sales_bi BEFORE INSERT ON sales FOR EACH ROW
BEGIN :NEW.created_at := NVL(:NEW.created_at, SYSTIMESTAMP); END;
/
```

Trigger 예시 — PostgreSQL

【 SQL 예시 】

```
CREATE TABLE audit_log(  
  id bigserial PRIMARY KEY, table_name text, op text,  
  row_json jsonb, at timestamptz default now()  
);  
  
-- Row-level  
CREATE OR REPLACE FUNCTION trg_sales_ai() RETURNS trigger AS $$  
BEGIN  
  INSERT INTO audit_log(table_name, op, row_json)  
  VALUES ('sales', TG_OP, to_jsonb(NEW));  
  RETURN NEW;  
END$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER sales_ai  
AFTER INSERT ON sales  
FOR EACH ROW EXECUTE FUNCTION trg_sales_ai();
```

Trigger 예시 — PostgreSQL

【 SQL 예시 】

-- Statement-level + Transition table

```
CREATE OR REPLACE FUNCTION trg_sales_stmt_ai() RETURNS trigger AS $$  
BEGIN  
  INSERT INTO audit_log(table_name, op, row_json)  
  SELECT 'sales', TG_OP, to_jsonb(n) FROM new_table AS n;  
  RETURN NULL;  
END$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER sales_stmt_ai  
AFTER INSERT ON sales  
REFERENCING NEW TABLE AS new_table  
FOR EACH STATEMENT EXECUTE FUNCTION trg_sales_stmt_ai();
```

-- 함수 + 트리거 (row/statement, transition table)

DBMS별 Trigger 기능 비교

항목	PostgreSQL	MySQL	MariaDB	SQL Server	Oracle
Row/Statement 트리거	둘 다	Row만	Row만	Statement만 (다중행은 inserted/deleted에 반영)	둘 다
INSTEAD OF (뷰)	지원	미지원	지원	지원	지원
Transition table (문장 단위 변경집합 참조)	지원	미지원	지원	inserted /deleted로 대체	미지원
DDL/Event 트리거	Event Trigger 지원	미지원	미지원	DDL Trigger 지원	DDL Trigger 지원
트리거 실행 순서 제어	명시적 지정 어려움(의존 금지)	FOLLOWS /PRECEDES	FOLLOWS	sp_settriggerorder	FOLLOWS
같은 테이블 재수정	가능(주의)	금지(오류 1442)	유사	가능(주의)	가능하나 Mutating table 제약 존재
Autonomous Tx	미지원	미지원	미지원	미지원	지원(PRAGMA AUTONOMOUS_TRA NSACTION)

DBMS별 Trigger 기능 비교

항목	PostgreSQL	MySQL	MariaDB	SQL Server	Oracle
보안 컨텍스트	함수에 SECURITY DEFINER 가능	트리거는 DEFINER 계정 권한	트리거는 DEFINER	EXECUTE AS로 전환 가능	디파이너 권한

**MySQL은 row 트리거만,
SQL Server는 statement 트리거 중심이라는 관점이
설계에 큰 영향**

이벤트 처리 아키텍처 — 배치·알림·CDC

원칙: 트리거에서 외부 **API** 호출 금지 → 트리거→로그 테이블/토픽 적재 → 비동기 소비자 패턴

스케줄 작업(배치)

- MySQL/MariaDB
 - EVENT Scheduler
(SET GLOBAL event_scheduler = ON;)
- PostgreSQL
 - 내장 스케줄러 없음
→ pg_cron 확장 또는 외부 스케줄러
- SQL Server
 - SQL Server Agent (작업/스케줄)
- Oracle
 - DBMS_SCHEDULER(권장)
 - DBMS_JOB(구버전)

알림/메시징

- PostgreSQL
 - LISTEN/NOTIFY
: PERFORM pg_notify('channel', payload)
→ 앱 실시간 수신
- SQL Server
 - Service Broker
 - Event Notifications
- Oracle
 - Advanced Queuing (AQ)

CDC (Change Data Capture)

- DB 변경을 실시간 이벤트 스트림
- MySQL/MariaDB
 - Binlog 기반 CDC
→ Debezium → Kafka
- PostgreSQL
 - Logical Decoding (wal2json, pgoutput)
→ Debezium → Kafka
- SQL Server
 - CDC 내장
 - Change Tracking 내장
- Oracle : GoldenGate (표준솔루션)

Trigger 심화 — CDC 패턴과 pg_notify

CDC (Change Data Capture) with Trigger

- Debezium 없이 트리거로 간단한 CDC 구현 가능
- 변경 내용을 **outbox** 테이블에 기록
→ 별도 프로세스가 읽어서 처리

PostgreSQL NOTIFY/LISTEN 패턴

- 트리거에서 **pg_notify()** 호출
→ 연결된 앱에 실시간 알림
- 앱
 - **LISTEN channel_name**
→ 이벤트 수신 후 처리
- 용도
 - 실시간 대시보드 갱신
 - 웹소켓 푸시 알림

Trigger 심화 — CDC 패턴과 pg_notify

DDL Trigger (PostgreSQL Event Trigger)

- **CREATE EVENT TRIGGER**
... **WHEN TAG IN**
(**'CREATE TABLE', 'ALTER TABLE'**)
- 스키마 변경 감시·차단
- 네이밍 규칙 자동 검사
- **DDL 변경 이력을**
ddl_history 테이블에 자동 기록

Trigger vs Application 선택

- 트리거
 - 어떤 경로로 데이터가 변경되어도 반드시 실행 (감사·무결성)
- 앱
 - 비즈니스 로직
 - 외부 시스템 호출
 - 복잡한 흐름 제어
- 일반 원칙
 - 트리거는 짧고 결정적으로, 외부 의존성 없이

PostgreSQL Event Trigger & NOTIFY 예시

【 SQL 예시 】

-- 1. DDL 변경 이력 자동 기록 (Event Trigger)

```
CREATE TABLE ddl_history (  
  id BIGSERIAL PRIMARY KEY,  
  event_tag TEXT,  
  object TEXT,  
  executed_by TEXT DEFAULT current_user,  
  at TIMESTAMPTZ DEFAULT now()  
);  
  
CREATE OR REPLACE FUNCTION fn_ddl_logger() RETURNS event_trigger AS $$  
DECLARE r RECORD;  
BEGIN  
  FOR r IN SELECT * FROM pg_event_trigger_ddl_commands() LOOP  
    INSERT INTO ddl_history(event_tag, object)  
    VALUES (r.command_tag, r.object_identity);  
  END LOOP;  
END$$ LANGUAGE plpgsql;  
  
CREATE EVENT TRIGGER ddl_logger ON ddl_command_end  
EXECUTE FUNCTION fn_ddl_logger();
```

PostgreSQL Event Trigger & NOTIFY 예시

【 SQL 예시 】

```
-- 2. 실시간 알림 (NOTIFY/LISTEN)
-- 트리거에서 변경 발생 시 앱에 즉시 알림
CREATE OR REPLACE FUNCTION fn_notify_order() RETURNS trigger AS $$
BEGIN
PERFORM pg_notify('order_channel', row_to_json(NEW)::TEXT);
RETURN NEW;
END$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_order_notify AFTER INSERT ON orders
FOR EACH ROW EXECUTE FUNCTION fn_notify_order();

-- 앱에서 수신 (psycopg2 Python)
-- conn.set_isolation_level(ISOLATION_LEVEL_AUTOCOMMIT)
-- cur.execute("LISTEN order_channel")
-- while True: select.select([conn],...); conn.poll(); print(conn.notifies.pop())
```

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

26. Cloud DB 개요

- On-Prem vs DBaaS · AWS RDS & Aurora · GCP Cloud SQL · Azure Database

On-Prem vs DBaaS vs Cloud-Native DB 비교

구분	On-PRem	DBaaS (RDS/Cloud SQL)	Cloud-Native (Aurora/Spanner)
프로비저닝	서버 구매, 설치, 랙	콘솔에서 몇 분 내 생성	스토리지, 복제 구조 전용화 가능
확장성	수직 확장 (교체, 증설 시 느림)	수직 + 수평 (리드 리플리카)	설계부터 분산, 자동 복구, 글로벌
고가용성	DBA가 직접 RAC, AG 구성	Multi-AZ 자동 장애 조치	스토리지 레벨 6-way 복제
운영/패치	DBA, 시스템팀 책임	서비스가 백업, 패치 자동화	더 높은 자동화 수준
비용	CAPEX(선투자) + OPEX	OPEX(사용량 기반)	OPEX + egress 전용 기능 비용
커스터마이징	OS, Filesystem, 확장 자유	슈퍼 유저 제한, 확장 제한	제한이 좀 더 클 수 있음

DBaaS 할 일 vs 우리가 할 일

DBaaS가 해주는 것

- 하드웨어/스토리지 관리
- 장애 디스크 교체
- 백업/스냅샷 스케줄링
- 보관/복구 기능 제공
- 모니터링 지표/알림
- 엔진 소규모 패치(마이너)
- 고가용성 토폴로지
(Multi-AZ, 자동 장애조치) 제공

우리(고객)가 해야 하는 것

- 스키마/인덱스/쿼리 최적화
(핵심! DBaaS라도 책임 면제 아님)
- DB 파라미터 전략
(초기/운영 프로파일)
- 보안정책 설계
(권한/암호화/네트워크 경계)
- 비용/성능 SLO 설계와 관측
(옵서버빌리티)
- 메이저 버전 Upgrade
- 호환성 검증

의사결정 기준

- 규제/데이터 주권
- 외부 반출 금지?
→ On-Prem 우선
- 팀 운영역량 부족?
→ DBaaS 가중치 ↑
- 트래픽 변동 큼?
→ 서버리스/자동 스케일 고려
- 글로벌 사용자, 멀티리전 강일관성?
→ Spanner/다지역 설계

AWS RDS vs Aurora 선택 가이드

항목	AWS RDS	Amazon Aurora
아키텍처	엔진 원본 그대로 관리형 운영	Compute/스토리지 분리, 3AZ 6중 복제
스토리지	EBS 기반	10GB 단위 자동 확장, 최대 128TB
고가용성	Multi-AZ 대기 중	스토리지 동기 복제 + 빠른 장애 조치 가능
읽기 확장	읽기 복제본 추가	리더 엔드포인트, Serverless v2
글로벌	일부 엔진 기능 의존	Global Database (10개 Region, 수십ms)
특화기능	Blue/Green 배포, RDS Proxy	Backtrack·Parallel Query·Serverless v2
선택 기준	엔진 호환성 최우선, 중소규모	읽기 부하 ↑, 고가용성, 빠른 장애 조치 필요

GCP Cloud SQL & Azure Database 핵심

GCP Cloud SQL

- **MySQL·PostgreSQL·SQL Server** 지원
- **Enterprise/Enterprise Plus** 에디션
- **Regional HA**
 - 기본/대기 구성
 - 자동 페일오버
 - **Read Replica** (읽기 확장)
- **IAM DB Auth**
 - 서비스 계정/**IAM** 사용자 토큰 로그인 (**PostgreSQL** 중심)
- 최대 **64TB** 스토리지
- 오토그로스 설정 가능

Azure Database

- **MySQL/PostgreSQL**
 - **Flexible Server**
(**Zonal/Zone-redundant HA** 선택)
- **Zone-redundant HA**
 - **SLA 99.99%**
 - **Burstable** 티어는 HA 미지원
- **Microsoft Entra ID(Azure AD)**
인증 연동 가능

GCP Cloud SQL & Azure Database 핵심

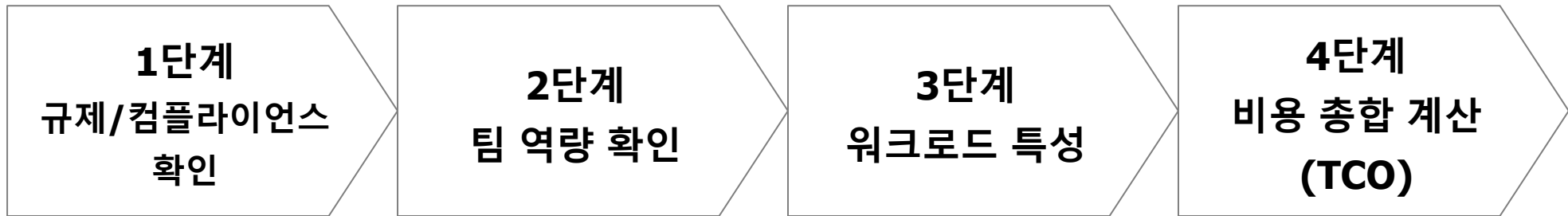
Azure SQL Database (SQL Server 호환 PaaS)

- **Serverless**
 - 자동 일시중지/재개
 - 간헐 트래픽에 비용 효율
- **Hyperscale**
 - 대용량(수TB급) 신속 백업/복구
 - 수평 확장
- **Managed Instance**
 - On-Prem SQL Server 호환성 강조

GCP AlloyDB

- 로그-분리형 아키텍처
- 분석/OLTP 혼합 워크로드 성능 강조

Cloud DB 선택 의사결정 프레임워크



Cloud DB 선택 의사결정 프레임워크

1단계 규제/컴플라이언스 확인

- 데이터 주권(Data Residency)
 - 특정 국가/리전에 데이터 저장 의무?
- GDPR·개인정보보호법·금융 규제
 - 외부 클라우드 반출 금지?
 - 계약 있으면 On-Prem
 - 또는 특정 리전 Cloud

2단계 팀 역량 확인

- DBA 전문가 있는가?
 - On-Prem 가능
- DevOps 팀 작은가?
 - DBaaS 권장 (운영 부담 최소화)

Cloud DB 선택 의사결정 프레임워크

3단계 워크로드 특성

- 트래픽 예측 가능·안정적
→ 고정 인스턴스 (RDS/Cloud SQL)
- 트래픽 급변·간헐적
→ 서버리스 (Aurora Serverless v2·Neon)
- 글로벌 사용자·멀티리전 일관성 필요
→ Spanner/CockroachDB

4단계 비용 총합 계산 (TCO)

- On-Prem
 - CAPEX(서버 구매)
+ OPEX(인건비·전기·유지보수)
- DBaaS
 - OPEX(사용량)
+ Data Egress + IOPS + 백업 스토리지
- Cloud
 - 간과하기 쉬운 비용
→ Egress·IOPS·API 호출 확인

Cloud DB 선택 의사결정 프레임워크

3단계 — 쿼리 개선

- 함수 적용된 컬럼
→ 범위 조건 또는 함수 기반 인덱스로 변경
- 서브쿼리
→ **JOIN** 또는 **CTE**로 변경
- **N+1** 문제
→ 한 번의 **JOIN**으로 해결

4단계 — 아키텍처 개선

- 대용량 집계
→ **Materialized View** 또는 집계 테이블
- 전체 스캔 불가피
→ 파티셔닝으로 스캔 범위 축소
- 읽기 부하
→ **Replica** (읽기 전용 복제본) 분산

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

27. 서버리스 & 분산 Cloud DB

- Aurora Serverless v2 · Google Spanner · Cosmos DB · PlanetScale · Neon · Supabase

서버리스·분산 DB 주요 서비스 4종

Aurora Serverless v2 (MySQL/PG 호환)

- ACU 단위 자동 스케일
- 급증 트래픽 수동 스케일링 불필요
- Global DB·Backtrack 옵션
- 간헐적/변동성 큰 OLTP에 적합
- 스케일 이벤트 시 지연 변화
→ 커넥션 풀·읽기/쓰기 분리 설계

Google Cloud Spanner (True Distributed SQL)

- Paxos/TrueTime 기반
- 글로벌 트랜잭션+외부일관성
- Strict Serializability 보장
- 전역 스키마/SQL
- 멀티리전에서 단일 DB처럼
- 스키마·핫스팟 설계 중요

서버리스·분산 DB 주요 서비스 4종

Azure Cosmos DB (다중 모델·일관성 선택)

- **NoSQL**
 - 문서·키값·Cassandra·MongoDB 등
다중 API 지원
- 5단계 일관성 레벨 선택 가능
- 전 지구적 확장, IoT/이벤트
- 세션 데이터에 적합
- 낮은 일관성→앱 로직 보완

PlanetScale/Neon/Supabase (차세대 개발자 DB)

- **PlanetScale**
 - Vitess 기반
 - 브랜칭, Online DDL 강점
- **Neon**
 - scale-to-zero
 - 스토리지-컴퓨트 분리
- **Supabase**
 - Postgres + Auth + Realtime
+ Edge Functions

서버리스/분산 vs 전통 RDBMS 비교

항목	전통 RDBMS	서버리스/분산 Cloud DB
용량 계획	인스턴스 크기 수동 관리	자동 스케일 (Aurora v2/Neon — scale-to-zero)
일관성 모델	단일 노드에 강한 일관성	글로벌 일관성(Spanner) 선택형 (Cosmos 5단계)
다지역	복제, DR 수동 구성	Managed Global (Spanner·Cosmos·Aurora Global)
스키마 변경 시	Lock, 다운 타임 위험	Online DDL, 브랜칭
비용 모델	고정 (유휴 비용 높음)	사용량 기반 절감 유휴 절감 (scale-to-zero)

MSA 환경 DB 패턴 선택 가이드

패턴	설명	적합 상황(예시)	주의 사항
Database Per Service	서비스별 독립 DB 소유	마이크로서비스 독립 배포	- Cross-service 조인 불가 - 데이터 중복
Saga (Choreography)	이벤트 기반 자율 조정	서비스 간 결합 최소화	- 분산 트랜잭션 복잡 - 보상 트랜잭션 설계 필요
Saga (Orchestration)	중앙 코디네이터가 조정	복잡한 워크플로우 제어	코디네이터 단일 장애 포인트
CQRS	읽기/쓰기 모델 분리	읽기/쓰기 부하 불균형 시	이벤트 발생 시 일관성 허용 필요
Outbox Pattern	트랜잭션 내 이벤트 발행 보장	At-least-once 전달 필요	멱등성 소비자 구현 필요
Event Sourcing	상태가 아닌 이벤트 저장	이력, 감사, 타임 트래블 필요	- 복잡한 쿼리 - 읽기 성능 고려

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

28. 데이터 웨어하우스 & 분석 DB

- OLTP vs OLAP · 파티션 프루닝 · 벡터화 실행 · Redshift · BigQuery · Snowflake · ClickHouse

OLTP vs OLAP 비교

구분	OLTP (운영 DB)	OLAP (분석 DB/DWH)
목적	트랜잭션 처리 (쇼핑, 결제, 재고 등)	다차원 분석, 집계 (매출 등)
워크로드	- 작은 단건 읽기/쓰기 - 낮은 지연	- 대량 스캔 및 집계 - 높은 처리량
스키마	정규화 (보통 3정규화 기준)	De-normalize (스타, 스노우플레이크)
인덱스	B-Tree, PK, UK 포인트 질의에 최적	파티션, 클러스터링, 컬럼형 저장으로 Full Scan 최적
트랜잭션	- 강한 ACID - 짧은 트랜잭션	- 일괄/배치 적재 - 읽기 일관성 (Snapshot)
저장형식	행 지향 (Row store) 중심	컬럼 지향 (Columnar : Parquet/ORC)
최적화	인덱스 튜닝, 조인 순서	파티션 프로닝, 컬럼 프루닝, 벡터화

분석 DB 핵심 기술 3가지

파티션 프루닝 (Partition Pruning)

- 필요한 날짜/범위의 파티션만 읽고 나머지는 무시
- **BigQuery**
 - **PARTITION BY DATE** (event_time)
 - **WHERE** 해당 파티션만 스캔
- **Snowflake**
 - 마이크로파티션 통계로 해당 범위 밖 파일 자동 스킵
- **ClickHouse**
 - **PARTITION BY toDate(ts)**
 - + **ORDER BY**로 범위별 병렬 처리
- 효과 (I/O, 비용 절감)
 - I/O 수십~수백 배, 비용 (BigQuery/Snowflake는 스캔 바이트 과금)

컬럼 프루닝 (Column Pruning)

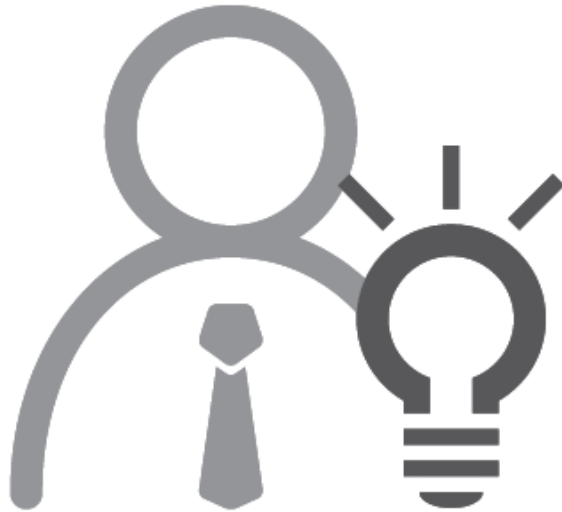
- **SELECT**한 컬럼만 디스크에서 읽고 나머지는 완전히 스킵
- **BigQuery**
 - 스캔 바이트량이 **SELECT** 컬럼 수에 따라 즉시 줄어듦
 - **SELECT *** 절대 금지! 필요한 컬럼만 명시

벡터화 실행 (Vectorized Execution)

- 행 단위 아닌 컬럼 단위 벡터 (1000개씩)로 묶어 **CPU SIMD** 명령어로 처리
- **BigQuery/Snowflake/DuckDB/ClickHouse**
 - 모두 벡터화 실행 엔진 채택
- 효과
 - CPU 효율↑
 - 캐시 효율↑
 - 병렬 처리 용이

잠시 **Solution** 소개를...

지금부터 말하는 **DB**는 저와는 아무 관계가 없음다!



분석 DB 제품별 특징 비교

제품	아키텍처	강점	비용 모델	적합 시나리오(예)
Amazon Redshift	MPP (리더 + Compute slice)	AWS 생태계 (S3, Glue, SageMaker)	노드 기반 + Serverless 옵션	AWS 중심 Data Lake + DWH
Google BigQuery	- 서버리스 - Storage/Compute 완전 분리	- 간단 운영 - ML/GIS/BI Engine 통합	스캔 바이트 과금 (절약 중요!)	- GCP 중심 - 완전한 서버리스 시
Azure Synapse	MPP SQL 풀 + 서버리스 + Spark 통합	MS 환경 (MS PowerBI, AD, Fabric 통합 환경)	일시 중지, 재개	- MS 생태계 - Spark + SQL 혼합
Snowflake	- 스토리지, Compute 분리 - 멀티 가상 WH	- 운영 단순성 - 데이터 웨어링 - VARIANT	- 초 단위 과금 - 일시 중지	- 멀티 클라우드 - 운영 단순화
ClickHouse	- 컬럼형 OLAP - MergeTree 엔진	- 초저지연 실시간 집계 - 비용 효율	- 오픈소스 - 자가호스팅 가능	이벤트, 로그, 시계열에 대한 실시간 분석

스타 스키마 & 데이터 웨어하우스 설계 원칙

스타 스키마 (Star Schema)

- DW 기본형
- 팩트 테이블
 - 거래·이벤트(매출액·수량·클릭수)
 - 크고 **Append-only**
 - 시간 파티션
- 디멘전 테이블
 - 사용자/상품/시간/지역
 - 비교적 작고 **SCD Type2** 이력 적용
- 중앙 팩트를 여러 디멘전이 둘러싸는 별(★) 모양

스타 스키마 설계 규칙

- **Surrogate Key**
 - 디멘전에 정수형 대체키 사용
(자연키는 속성으로 유지)
- 팩트는 얇게 길게
 - 많은 행(**append-only**)
 - 측정값(**measure**)과 FK만
- 역할 차원(**Role-Playing**)
 - 날짜 한 번 만들고
order_date·**ship_date**로 별칭
- **Junk Dimension**
 - Y/N, 소수 플래그들을 묶어
작은 디멘전으로
- **Conformed Dimension**
 - 여러 마트에서 같은 정의로 재사용
(카테고리·지역)

스노우플레이크 스키마 vs 3NF DW

- 스노우플레이크
 - 디멘전을 다시 정규화
→ 카테고리·지역처럼
계층 자주 바뀔 때 선별 사용
- **3NF DW**
 - 원천/코어 레이어는 **3NF**
→ 소비 레이어(**mart**)는 스타
- **Cloud DW (BigQuery·Snowflake)**
 - 조인 비용 낮으면
정규화보다 넓은 테이블 선호

BigQuery 파티션·클러스터 설계 + ClickHouse MergeTree

【 SQL 예시 】

-- Google BigQuery: 파티션 + 클러스터

```
CREATE TABLE mart.fact_sales
```

```
PARTITION BY DATE(event_ts) -- 날짜별 파티션 (스캔 비용 절감)
```

```
CLUSTER BY user_id, item_id AS -- 클러스터: user_id로 정렬 후 item_id
```

```
SELECT
```

```
event_ts, user_id, item_id,
```

```
qty, amount, channel
```

```
FROM stg.sales_cleaned;
```

-- 파티션 프루닝: 특정 날짜만 스캔

```
SELECT SUM(amount) FROM mart.fact_sales
```

```
WHERE event_ts >= '2024-01-01' AND event_ts < '2024-04-01';
```

-- → Q1 파티션만 스캔, 나머지 완전 스킵

BigQuery 파티션·클러스터 설계 + ClickHouse MergeTree

【 SQL 예시 】

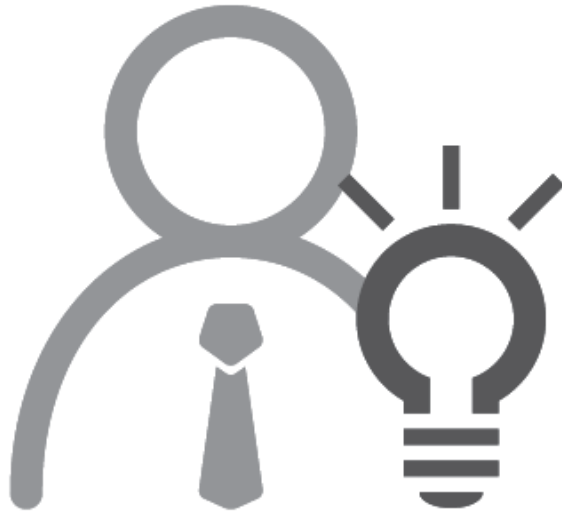
-- ClickHouse: MergeTree 엔진 (초고속 실시간 OLAP)

```
CREATE TABLE fact_events (  
  event_time DateTime,  
  user_id UInt64,  
  action LowCardinality(String), -- 카디널리티 낮은 컬럼 최적화  
  amount Decimal(12,2)  
) ENGINE = MergeTree  
PARTITION BY toDate(event_time) -- 날짜별 파티션  
ORDER BY (user_id, event_time) -- 정렬 키 = 스킵 인덱스  
TTL event_time + INTERVAL 90 DAY; -- 90일 후 자동 삭제
```

-- 실시간 집계 (ms 단위 응답)

```
SELECT user_id, COUNT(), SUM(amount) FROM fact_events  
WHERE action = 'purchase' AND event_time >= today() - 7  
GROUP BY user_id ORDER BY SUM(amount) DESC LIMIT 10;
```

지금부터 말하는 DB는 저와는 아무 관계가 없음다!



요구사항	추천 제품	Why?
완전 서버리스 + GCP 중심	BigQuery	• 서버리스 DWH • 스캔 바이트 과금 • 운영 최소
AWS 생태계 + S3 레이크 + DWH	Redshift	• AWS 생태계 밀착 • Dist/Sort Key 설계로 최적화
MS PowerBI + Spark + SQL 통합	Azure Synapse	• MS 생태계 • Fabric, PowerBI 연동
운영 단순 + Data Sharing	Snowflake	• 멀티 클라우드 • JSON • 자동 튜닝
초저지연 실시간 집계	ClickHouse	• 이벤트, 로그, 시계열 • 비용 효율 • 오픈소스

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

29. 현재의 트렌드

- NewSQL · 시계열 DB · Vector DB & pgvector RAG · AI+DB · GraphQL · Edge DB

NewSQL — SQL + 트랜잭션 + 수평 확장

등장 배경

- 전통 RDBMS
 - 강한 ACID+SQL but 수평 확장 어려움
- NoSQL
 - 수평 확장 쉬움
 - but 트랜잭션/SQL/조인 약함
- NewSQL
 - 3박자 모두
 - Shared-nothing 분산
 - + Raft/Paxos
 - + MVCC

CockroachDB

- 아키텍처
 - 키-값 스토어 위에 SQL 레이어
 - Range 단위 자동 샤딩
 - Raft 그룹 복제
- 강점
 - 단순 운영(모든 노드 동등)
 - 자동 분산/리밸런싱
 - 기본 **SERIALIZABLE**
- 약점
 - 멀티리전 쓰기 시 합의 왕복으로 지연 증가
 - 스키마 설계 최적화 필요

NewSQL — SQL + 트랜잭션 + 수평 확장

TiDB (PingCAP)

- 아키텍처
 - MySQL 호환 SQL(TiDB)
 - + TiKV(분산 KV, Raft)
 - + PD(Placement Driver)
 - + TiFlash(OLAP)
- 강점
 - HTAP(OLTP+OLAP) 동시 지원
 - MySQL 생태계 완전 호환
- 약점
 - 클러스터 구성요소 이해 필요
 - 운영 복잡도

선택 가이드

- 강한 일관성+기본 **SERIALIZABLE** 필요
→ CockroachDB
- MySQL 호환 유지+수평 확장+HTAP
→ TiDB

시계열 DB & Vector DB

TimescaleDB (PostgreSQL 확장)

- Hypertable
 - time + space 파티션 자동화
- Continuous Aggregates
 - 자동 Materialized View (집계 뷰 실시간 갱신)
- Compression, Retention policy 내장
- 장점
 - 표준 SQL 그대로
 - Postgres 에코시스템 (GIS·확장·트랜잭션) 유지

InfluxDB

- 시계열 전용으로 고속 쓰기·압축
- 보존 최적화
- 장점
 - 대량 시계열 ingest
 - 내장 Retention, Downsampling 파이프라인
- 주의
 - SQL 문법과 다름 (Flux/InfluxQL)
 - 표준 조인·트랜잭션 개념 제한
- 선택
 - SQL 생태계 유지
 - TimescaleDB 고속 ingest 특화
 - InfluxDB

Vector DB AI 시대의 유사도 검색

- pgvector(PostgreSQL 확장)
 - vector 타입
 - + HNSW/IVF 인덱스 / SQL
 - + 트랜잭션
 - + JOIN 그대로
- Pinecone
 - 완전 관리형
 - 초대규모 클러스터 운영, 필터링, 멀티 테넌시
- Weaviate
 - 오픈소스+매니지드
 - BM25+벡터 하이브리드
 - GraphQL/REST API
- DB 내 SQL+조인 → pgvector
 - 초대규모+운영단순 → Pinecone
 - 오픈소스+API유연 → Weaviate

pgvector — RAG 하이브리드 검색 패턴

【 SQL 예시 】

```
-- pgvector 설치 및 테이블 구성
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pg_trgm; -- 전문검색

CREATE TABLE knowledge_base (
  id BIGSERIAL PRIMARY KEY,
  title TEXT NOT NULL,
  content TEXT,
  chunk_idx INT DEFAULT 0,
  embedding VECTOR(1536), -- OpenAI text-embedding-3-small
  search_ts TSVECTOR, -- 한국어 FTS
  created_at TIMESTAMPTZ DEFAULT now()
);
```

Checkpoint

- ✓ **RAG** = 문서를
임베딩으로 **DB**에
저장
→ 질문 임베딩으로
유사 문서 검색
→ **LLM** 컨텍스트
주입

pgvector — RAG 하이브리드 검색 패턴

【 SQL 예시 】

```
-- HNSW 인덱스 (코사인 유사도 기준)
CREATE INDEX ON knowledge_base USING HNSW (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
CREATE INDEX ON knowledge_base USING GIN (search_ts);

-- 하이브리드 검색 (벡터 70% + 키워드 30%)
WITH vector_search AS (
  SELECT id, 1 - (embedding <=> $1::VECTOR) AS vec_score
  FROM knowledge_base ORDER BY embedding <=> $1::VECTOR LIMIT 20
),
keyword_search AS (
  SELECT id, ts_rank(search_ts, plainto_tsquery('korean', $2)) AS kw_score
  FROM knowledge_base WHERE search_ts @@ plainto_tsquery('korean', $2)
)
SELECT kb.title, kb.content,
COALESCE(vs.vec_score,0)*0.7 + COALESCE(ks.kw_score,0)*0.3 AS hybrid_score
FROM knowledge_base kb
LEFT JOIN vector_search vs ON vs.id = kb.id
LEFT JOIN keyword_search ks ON ks.id = kb.id
WHERE vs.id IS NOT NULL OR ks.id IS NOT NULL
ORDER BY hybrid_score DESC LIMIT 5;
```

Checkpoint

- ✓ **RAG** = 문서를
임베딩으로 **DB**에
저장
→ 질문 임베딩으로
유사 문서 검색
→ **LLM** 컨텍스트
주입

AI/ML + DB 통합 패턴 3가지

패턴 1 : DB 안에서 ML (in-DB ML)

- BigQuery ML,
Snowflake Snowpark
SQL Server ML Services,
PostgreSQL MADlib
- SQL로 회귀/분류/시계열
/클러스터링 모델 직접 실행
- 장점
 - 데이터 이동 없음 (보안/규정 준수)
 - 재현성/권한 통합
- 단점
 - 모델 다양성/최신성은
PyTorch/TF 전용 프레임워크
대비 제한

패턴 2 : RAG (Retrieval-Augmented Generation)

- 파이프라인
 - 원문 → 임베딩
→ Vector DB 저장
→ 질문 유사 문서 검색
→ LLM 컨텍스트 주입
- 품질 핵심
 - Chunking/필터/재순위
(re-ranking)
 - 최신성 갱신
 - 출처 보강
- pgvector + PostgreSQL
 - SQL+트랜잭션+조인으로
완전한 RAG 파이프라인 구현
- 추후 과정에서 상세 진행 예정
(Vector DB 등)

패턴 3 : 피쳐 스토어 + 실시간 예측

- Feast 같은 피쳐 스토어로
오프라인/온라인 일관성 유지
(훈련/서빙 스키마 동일)
- DB/DWH
 - 오프라인 피쳐 소스
 - 캐시(Redis) : 온라인 피쳐 역할
- RDBMS/Cloud DB
(BigQuery·Snowflake·Redshift)
 - 내장 ML/UDTF/외부 함수

GraphQL + DB & Edge DB — 최신 트렌드

GraphQL + DB 패턴

- 문제
 - REST는 Over-fetch(필요 이상)
 - Under-fetch(필요 이하) 문제
- GraphQL
 - 클라이언트가 필요한 필드만 질의 → 불필요한 데이터 전송 없음
- Hasura
 - PostgreSQL 위에서 즉시 GraphQL API 생성 + 권한·RLS 연동·구독(실시간)
- PostGraphile
 - 스키마로 GraphQL 자동 생성 가능, PostgreSQL 기능 깊게 활용
- Prisma
 - 타입 세이프 ORM, GraphQL 서버와 궁합 좋음 (Next.js 등)
- 주의
 - N+1 쿼리 방지 (DataLoader), 권한·멀티테넌시, CDN 캐싱 전략 필요

GraphQL + DB & Edge DB — 최신 트렌드

Edge DB 패턴

- 동기
 - 전 세계 사용자에게 낮은 지연(ms 단위), 엣지 함수와 가까운 데이터
- Turso
 - libSQL/SQLite 분산 → 엣지 노드에서 SQLite
- Cloudflare D1
 - SQLite 기반, Workers와 통합
- Neon/PlanetScale
 - 서버리스 PG/MySQL, 분리형 스토리지·브랜치·자동 스케일
- 패턴
 - 읽기=로컬 엣지 캐시 + 쓰기=중앙 합의 (최종 일관성)
- 주의
 - GDPR/데이터 거주성, 일관성 선택·충돌 해결 (CRDT) 전략

TimescaleDB — PostgreSQL 기반 시계열 DB

【 SQL 예시 】

```
-- TimescaleDB 설치 및 Hypertable 생성
CREATE EXTENSION IF NOT EXISTS timescaledb;

-- 일반 테이블 생성 후 Hypertable로 변환
CREATE TABLE metrics (
  ts TIMESTAMPTZ NOT NULL,
  host TEXT NOT NULL,
  cpu_pct DOUBLE PRECISION,
  mem_mb INT
);
SELECT create_hypertable('metrics', 'ts'); -- 시간 파티션 자동화

-- 대량 삽입 (10만 건)
INSERT INTO metrics
SELECT now() - (random() * INTERVAL '30 days'),
'host_' || (random()*10)::INT,
random() * 100,
(random() * 8192)::INT
FROM generate_series(1, 100000);
```

Checkpoint

- ✓ TimescaleDB
 - PostgreSQL 그대로
 - + 시계열 최적화
 - IoT, 금융 시계열에 최적

TimescaleDB — PostgreSQL 기반 시계열 DB

【 SQL 예시 】

-- Continuous Aggregate: 5분 단위 평균 자동 집계

```
CREATE MATERIALIZED VIEW metrics_5m WITH (timescaledb.continuous) AS
SELECT time_bucket('5 minutes', ts) AS bucket,
host, AVG(cpu_pct) AS cpu_avg, AVG(mem_mb) AS mem_avg
FROM metrics
GROUP BY bucket, host;
```

-- 보존 정책: 30일 초과 데이터 자동 삭제

```
SELECT add_retention_policy('metrics', INTERVAL '30 days');
```

-- 조회 (자동으로 Continuous Aggregate 활용)

```
SELECT * FROM metrics_5m WHERE host = 'host_3' ORDER BY bucket DESC LIMIT 10;
```

Checkpoint

✓ TimescaleDB

- PostgreSQL

그대로

+ 시계열 최적화

- IoT, 금융 시계열에
최적

AI 시대 DB 엔지니어의 역할 변화

전통 DBA 역할

- 스키마 설계, 인덱스 최적화, 백업, 복구, 성능 튜닝
- 쿼리 실행계획 분석, Lock 관리, 장애 대응

AI 시대 추가 역할

- **Vector DB 설계**
임베딩 차원, 인덱스, 하이브리드 검색 구현
- **Feature Store 관리**
학습/서빙 피처 일관성 보장
- **AI 파이프라인 DB 통합**
ETL/ELT·Streaming·in-DB ML 연동
- **LLM 기반 SQL 자동 생성·검증**
(AI가 만든 쿼리의 성능·보안 검토)

AI 시대 DB 엔지니어

인프라 지식

- **인프라 운영**
아키텍처 설계·비용 최적화·파라미터 튜닝
- 온프레미스 → 멀티 클라우드 DB 전략 수립
- 단일 DBMS → 목적별 DB 혼용

목적별 DB 선택

데이터 유형	적합 DB(예시)	이유	예시
트랜잭션 데이터	- PostgreSQL - MySQL	ACID 보장, 관계 무결성	주문, 결제, 사용자
세션, 캐시	- Redis - Memcached	인메모리, 초저지연	장바구니, 세션, Rate Limit
검색 인덱스	- Elasticsearch - Opensearch	역방향 인덱스, 전문 검색	상품 검색, 로그 분석
시계열 데이터	- TimescaleDB - InfluxDB	타임 파티션, 집계 최적화	IoT 센서, APM 메트릭
임베딩/벡터	- pgvector - Qdrant	ANN 인덱스, 유사도 검색	RAG, 추천, 이미지 검색 (Vector DB 시간에 자세히)
그래프 데이터	- Neo4j - Amazon Neptune	관계 탐색, 경로 쿼리	SNS 친구 관계, 사기 탐지
분석/리포팅	- BigQuery - Snowflake	컬럼형, 대용량 스캔	BI 대시보드, 데이터마트

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

30. 보안 및 권한 관리

- 인증·인가·SQL Injection·암호화·Row-Level Security·감사

DB 보안 전체 구조

인증(Authentication)

- 누가 접속했는가 (아이디/비번, AD/IAM)

인가(Authorization)

- 그 사람이 무엇을 할 수 있나 (권한: **SELECT/INSERT** 등)

입력검증/파라미터 바인딩

- 악성 입력으로부터 **SQL**을 보호

암호화

- 네트워크 전송(**TLS**)과 저장(디스크/**TDE**/컬럼)에서 데이터 보호

감사(Audit)

- 누가 언제 무슨 쿼리 실행했는지 기록



- 최소 권한 원칙(Least Privilege)
— 필요한 것만 준다

사용자·역할·권한

Postgres
(가장 직관적)

```
-- 역할 생성
CREATE ROLE app_reader NOLOGIN;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_reader;

-- 사용자 생성 및 역할 부여
CREATE ROLE app_user LOGIN PASSWORD 'secret';
GRANT app_reader TO app_user;
```

MySQL
(8.0 이상 권장 방식)

```
CREATE ROLE 'app_reader';
GRANT SELECT ON mydb.* TO 'app_reader';
CREATE USER 'app_user'@'%' IDENTIFIED BY 'secret';
GRANT 'app_reader' TO 'app_user'@'%';
```

사용자·역할·권한

SQL Server
(Windows
/AD 연동 가능)

```
CREATE ROLE app_reader;  
GRANT SELECT ON SCHEMA::dbo TO app_reader;  
CREATE LOGIN app_user WITH PASSWORD='secret';  
CREATE USER app_user FOR LOGIN app_user;  
EXEC sp_addrolemember 'app_reader','app_user';
```

Oracle
(권한이 매우 세세함)

```
CREATE ROLE APP_READER;  
GRANT CREATE SESSION TO APP_READER;  
GRANT SELECT ON HR.EMPLOYEES TO APP_READER;  
CREATE USER APP_USER IDENTIFIED BY secret;  
GRANT APP_READER TO APP_USER;
```

Row-Level Security(행 단위 보안) — 멀티테넌시

【 목적 】

기능

- 같은 테이블에서 사용자가 볼 수 있는 행(데이터)을 제한

Postgres

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;  
CREATE POLICY customer_policy ON orders  
FOR SELECT USING (customer_id =  
current_setting('app.customer_id')::int);
```

```
-- 앱이 세션에 고객ID를 설정하면 그 고객 데이터만 조회 가능  
SELECT set_config('app.customer_id','42', true);
```

SQL Server는 SESSION_CONTEXT를 이용한 비슷한 패턴

SQL Injection — 원인과 한 줄 방어법

【 원인 】

- 개발자가 사용자 입력을 직접 SQL 문자열에 붙여넣음
→ 공격자가 SQL 구조 변경 가능

코드

위험: 문자열 포맷

(절대 금지!!!!!!)

```
sql = f"SELECT * FROM users WHERE name = '{user_input}'"  
cur.execute(sql)
```



psycopg2 (Postgres)

(파라미터 바인딩)

```
cur.execute("SELECT * FROM users WHERE name = %s",  
(user_input,))
```

절대 문자열 이어붙이기(문자열 포맷)로 쿼리 만들지 마라
→ 항상 파라미터 바인딩(Prepared Statement) 사용

SQL Injection — 원인과 한 줄 방어법

【 원인 】

- 개발자가 사용자 입력을
직접 **SQL** 문자열에 붙여넣음
→ 공격자가 **SQL** 구조 변경 가능

정렬·컬럼 이름 같은 경우

ORDER BY에 사용자 입력을 직접 넣으면 위험
→ 화이트리스트(허용 목록)로 검증 후 사용



```
if order not in ('name','created_at'):  
    order = 'created_at'  
sql = f"SELECT * FROM items ORDER BY {order}"
```

ORM(예: SQLAlchemy)은 안전한가?

대부분의 **ORM**은 파라미터 바인딩을 자동으로 해줘 기본적으로 안전

하지만 **ORM**에서 제공하는 **raw SQL**이나
.execute(text(...))에 입력을 직접 포맷팅하면 취약

결론: ORM 사용 시도 ORM의 파라미터 바인딩 API만 사용하라

암호화(세부: TLS, TDE, 컬럼 암호화)

전송 암호화 (TLS)

- 클라이언트↔DB 연결을 암호화
- **항상 켜자**

디스크 암호화 / TDE (Transparent Data Encryption)

- DB 파일(데이터파일, 백업)을 자동으로 암호화 → 앱은 모름
- 장점: 복구·백업까지 암호화
- 단점: 컬럼 단위 제어 불가
- Oracle, SQL Server, 대부분의 클라우드 DB가 TDE 또는 스토리지 암호화 제공

암호화(세부: TLS, TDE, 컬럼 암호화)

컬럼 암호화 (민감컬럼만 암호화)

- **PostgreSQL: pgcrypto 사용**
→ 데이터를 암호화해서 **bytea**로 저장
- **SQL Server: Always Encrypted**
→ 클라이언트 측 암호화 키로 암호화되어
DB는 암호문만 가짐
- **장점:** 민감한 데이터만 타겟, 검색 통제 가능
- **단점:** 인덱스/검색 제약, 성능 저하.

-- Postgres pgcrypto 예 (간단)

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;  
INSERT INTO customers (name, ssn_enc)  
VALUES ('Alice', pgp_sym_encrypt('111-22-3333', 'mypassword'));  
-- 복호화  
SELECT pgp_sym_decrypt(ssn_enc, 'mypassword') FROM customers;
```


클라우드 DB에서 달라지는 점

키 관리

- 클라우드는 **KMS**(키 관리 서비스)를 제공
- 키 권한을 잃으면 백업 복구 불가 → **키 관리 중요!**

IAM/AD 연동

- 사용자를 **DB 계정** 대신 클라우드 계정(**IAM/Azure AD**)으로 관리 가능
→ 중앙관리 편함

감사 & 로깅

- 클라우드 로그(**Audit**)를 쉽게 연결해 자동 보관·알림 가능
 - AWS RDS: IAM 인증, KMS 키
 - GCP Cloud SQL: Cloud KMS, IAM
 - Azure SQL: Azure AD, Key Vault

클라우드에서는 DB 권한 + 클라우드 권한을 함께 설계

흔히 하는 실수 & 해결방법

애플리케이션에 **DB 관리자 권한을 줌**

역할 분리, 최소권한

클라우드 **KMS** 키 권한을 삭제

키 권한 정책 문서화, 백업 키 관리

로그에 민감정보(**SSN, 비밀번호**) 평문으로 남김

마스킹/로그 필터링 및 민감컬럼 암호화

ORDER BY 같은 파라미터를 검증 없이 사용

화이트리스트로 검증

핵심 체크리스트

TLS 강제 적용

앱은 **Prepared Statement/ORM**
안전 **API**만 사용

DB 사용자·역할 설계
→ 최소권한 적용

민감데이터는 컬럼 암호화 또는
TDE 적용 결정

클라우드 키 관리는
팀 문서화 + 백업키 보관

감사 로그(누가 언제) 설정 및
모니터링

PostgreSQL 사용자(Role) 개념

【 사용자 개념 】

- 역할(Role) = 사용자(User)
- PostgreSQL에서는 User와 Role을 구분하지 않고 "Role"로 통합
- 단, 로그인 권한 여부(LOGIN)에 따라 사용 가능한지 결정됨



Role 속성

Role 속성	설명
LOGIN	실제 사용자로 로그인 가능
NOSUPERUSER	슈퍼유저 아님 (기본)
CREATEDB, CREATEROLE	DB나 Role 생성 가능 여부
INHERIT	부모 Role 권한 상속 가능

사용자 및 역할 생성/삭제

사용자 생성

```
-- 단순 로그인 가능한 사용자 생성
CREATE ROLE analyst LOGIN PASSWORD 'analyst_pw';

-- 개발자 역할: DB 생성 권한 추가
CREATE ROLE dev_user LOGIN PASSWORD 'dev_pw' CREATEDB;

-- 읽기 전용 사용자
CREATE ROLE readonly_user LOGIN PASSWORD 'readonly_pw' NOSUPERUSER;
```

사용자 삭제

```
DROP ROLE readonly_user;
```

사용자 존재 여부 확인

```
□du    -- psql 명령
-- 또는
SELECT rolname, rolsuper, rolcreatedb, rolcanlogin
FROM pg_roles;
```

권한 부여/회수_GRANT / REVOKE 구문 정리

【 주요 GRANT 권한 종류 (대상별) 】

대상	부여 가능한 권한	예시
Database	CONNECT, TEMP	GRANT CONNECT ON DATABASE db TO user;
Schema	USAGE, CREATE	GRANT USAGE ON SCHEMA public TO user;
Table	SELECT, INSERT, UPDATE, DELETE, REFERENCES	GRANT SELECT ON sales TO analyst;
Function	EXECUTE	GRANT EXECUTE ON FUNCTION calc_tax() TO user;

[실무 예시] 역할 기반 권한 분리 설계

【 권한 분리 설계표 】

역할	로그인 가능	권한 요약
admin_user	O	모든 권한, SUPERUSER
dev_user	O	CREATE TABLE, INSERT, SELECT, UPDATE
analyst_user	O	SELECT only
etl_user	O	TRUNCATE, INSERT, BULK 작업 권한
readonly_app	O	VIEW 또는 SELECT만 허용 (API용)

etl_user에 특정 작업 권한만 부여

```
CREATE ROLE etl_user LOGIN PASSWORD 'etl_pw';

GRANT CONNECT ON DATABASE your_database TO etl_user;
GRANT USAGE ON SCHEMA public TO etl_user;
GRANT INSERT, TRUNCATE ON TABLE sales TO etl_user;
```

권한 확인 및 모니터링

```
-- 사용자의 테이블 권한 확인
SELECT grantee, privilege_type, table_name
FROM information_schema.role_table_grants
WHERE grantee = 'analyst';

-- 현재 로그인한 사용자 확인
SELECT current_user;
```

[실무 예시] RBAC 권한 설계안

【 역할(Role)별 최소 권한 설정 – 데이터 엔지니어, 분석가, API 사용자 기준 】

역할명 (ROLE)	설명	권한 구성 요약
data_engineer	데이터 적재/정제 담당자	INSERT, UPDATE, TRUNCATE, SELECT, 특정 CREATE 권한
data_analyst	리포트 및 분석 전용	SELECT (READ-ONLY), EXECUTE on 분석 함수
api_user	프론트엔드/백엔드 연결 계정	SELECT on VIEW 또는 제한된 TABLE, 직접 테이블 접근 불허
etl_user	스케줄러 등 ETL 자동화 계정	INSERT, TRUNCATE, SELECT on 대상 테이블
admin_user	시스템 관리용	ALL PRIVILEGES + CREATEDB, CREATEROLE, LOGIN, SUPERUSER

일반적으로 **api_user, analyst_user**는 직접 테이블에 접근시키지 않고 **VIEW**를 통해 간접적으로 제한된 데이터만 제공

[실무 예시] 권한 모니터링용 SQL 쿼리

현재 역할(Roles) 목록 및 속성

```
SELECT rolname, rolsuper, rolcreatedb, rolcreaterole, rolcanlogin
FROM pg_roles
ORDER BY rolname;
```

사용자별 테이블 권한 확인 (information_schema 기준)

```
SELECT grantee, table_schema, table_name, privilege_type
FROM information_schema.role_table_grants
WHERE grantee IN ('data_engineer', 'data_analyst', 'api_user')
ORDER BY grantee, table_name;
```

특정 사용자가 가진 함수 권한

```
SELECT grantee, routine_schema, routine_name, privilege_type
FROM information_schema.role_routine_grants
WHERE grantee = 'data_analyst';
```

[실무 예시] 권한 모니터링용 SQL 쿼리

특정 테이블에 대해 어떤 권한이 부여되었는지

```
SELECT *  
FROM information_schema.role_table_grants  
WHERE table_name = 'sales';
```

현재 세션의 사용자 정보 확인

```
SELECT current_user, session_user;
```

[실무 예시] VIEW 기반 보안

【 VIEW 기반 보안 】

- 민감한 데이터가 있는 테이블의 경우,
- VIEW**를 통해 부분 필드만 **SELECT** 허용하거나
- row-level** 필터링을 적용하는 것이 일반적



View 구분

View	SQL
분석가용	CREATE VIEW v_employee_basic AS SELECT emp_id, emp_name, department FROM employee;
API 사용자용 View (급여, 이름 제거)	CREATE VIEW v_employee_api AS SELECT emp_id, department FROM employee;

```
-- 분석가에게 SELECT 권한 부여
GRANT SELECT ON v_employee_basic TO data_analyst;

-- API 사용자에게는 v_employee_api만 접근 허용
GRANT SELECT ON v_employee_api TO api_user;

-- 테이블 직접 접근은 차단되어야 함
REVOKE ALL ON employee FROM data_analyst, api_user;
```

RDB(User/Privilege) 제어 핵심 구조

MySQL

- 계정 생성/삭제
 - CREATE USER / DROP USER
- 권한 부여/회수
 - GRANT, REVOKE, SHOW GRANTS로 권한 조회
- 권한 범위
 - 글로벌, 데이터베이스, 테이블, 컬럼, 객체(뷰/프로시저/함수)
- 주요 권한
 - SELECT, INSERT, UPDATE, DELETE, EXECUTE, SUPER, PROCESS, FILE, SHUTDOWN 등
- 역할(Role)
 - CREATE ROLE, 역할에 권한 부여/회수, 사용자에게 역할 할당 후 계층적 권한 적용 가능
- 동적 권한
 - 서버 내 플러그인/특정 기능별 추가 권한(예: BACKUP_ADMIN, FIREWALL_ADMIN)

RDB(User/Privilege) 제어 핵심 구조

MySQL

- **User 및 Host 기반 관리**
 - 사용자 계정은 **user@host** 형태, 호스트를 지정해 접속 위치 제한
- **관리/보안 방법**
 - **CREATE USER, ALTER USER, DROP USER**로 계정 생성/변경/삭제
 - **SHOW GRANTS, INFORMATION_SCHEMA**로 권한 상태 주기적 감사, 비밀번호 정책 및 만료 관리
 - **최소권한 원칙, 서비스/관리 계정 분리, audit_log 활성화 등 권장**

RDB(User/Privilege) 제어 핵심 구조

PostgreSQL

- 계정/롤
 - CREATE ROLE로 사용자 및 그룹 생성
 - LOGIN 속성 지정 시 사용자
- 권한 부여
 - GRANT privilege ON object TO role;
 - REVOKE로 회수
- 주요 권한
 - SELECT, INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES, TRIGGER 등
- 세분화
 - 테이블/시퀀스/함수/스키마/데이터베이스 단위로 권한 지정
 - 그룹/역할 간 계층적 관리

RDB(User/Privilege) 제어 핵심 구조

PostgreSQL

- 관리/보안 방법
 - pg_hba.conf로 접속 인증 방식 제어
 - 권한 변경·감사 쿼리: pg_roles, pg_catalog으로 role/권한 정보 조회

RDB(User/Privilege) 제어 핵심 구조

Oracle

- 사용자(User)

- CREATE USER
- 삭제 : DROP USER

- Role 중심

- CREATE ROLE로 역할 생성 후 권한 부여 후 사용자에게 역할 할당
- 세분화된 역할(시스템 역할·객체 역할)을 통한 권한 관리, 역할에 권한을 부여하고 사용자는 역할을 할당

- 권한 부여

- GRANT, REVOKE로 사용자·역할에 system privilege(예: CREATE SESSION) 및 object privilege(예: SELECT ON employees) 부여/회수
- 내장 역할(CONNECT, RESOURCE, DBA) 활용 및 ORA_AUDIT 활성화로 역할 변동/사용 추적

- 세분화

- 시스템 권한(데이터베이스 수준), 객체 권한(테이블/뷰 등), 역할 권한, RBAC 지원

RDB(User/Privilege) 제어 핵심 구조

Oracle

- **Cloud 연동**
 - IAM/클라우드 통합 시 RBAC, 세분화된 관리 가능
- **관리/보안 방법**
 - 세밀한 역할 분할 및 각 역할에 대한 목적/권한 문서화, 분기점/승인 프로세스 도입
 - Active Directory 연동, role 관리 자동화/주기적 감사,
"WITH ADMIN OPTION" 최소화 권장
 - 변경 및 권한 동향에 대한 감사/모니터링 필수

RDB(User/Privilege) 제어 핵심 구조

SQL Server

- 로그인·사용자·Role 분리
 - 로그인을 통해 인증, DB 사용자 계정을 각 데이터베이스 별로 매핑 가능
- 권한 부여/회수
 - 내장/커스텀 server/database role(예: db_owner, db_datareader, AuditViewer) 이용, 각 role별로 granular 권한 관리
 - GRANT, DENY, REVOKE 명령으로 세부 권한 및 기능 제어
 - Active Directory 연동 및 서비스 계정+개인 계정 분리
- 관리/보안 방법
 - 최소권한 할당(principle of least privilege), 역할 기반 권한 관리(RBAC), 분리된 책임(SOD)
 - strong password/multi-factor authentication, 사용자/권한 주기적 감사 및 audit/monitoring 활성화
 - 불필요 계정 정리, 권한 변경 이력 기록

CloudDB(분산 DB, Cloud Native DB) 권한 관리

Google Cloud Spanner

- **IAM 기반 Role/Permission**

- Cloud IAM에서 Spanner Admin, Database Admin, Database User 등 역할(소유/편집자/읽기전용/백업권한/세분화된 FGAC 등)을 지정
- 세부 작업 권한까지 구분

- **세분화**

- Fine-grained access control(FGAC)로 DB 내부 개체 단위까지 세밀 관리
- IAM과 DB 역할 연계

- **권한 관리 관점**

- 프로젝트/인스턴스/DB/테이블 단위 모든 리소스에 Role별 할당

CloudDB(분산 DB, Cloud Native DB) 권한 관리

Amazon Aurora, RDS

- **IAM과 연동**
 - RDS 액세스 관리(Option Group/Security Group)로 인스턴스, DB 수준 접근 제어
- **SQL 기반 권한 관리**
 - MySQL, PostgreSQL 등 RDB 엔진별 자체 권한 관리와 IAM 접근관리 동시 활용 가능

기타 Cloud /NoSQL (RBAC 대표)

- **Role 기반 제어**
 - ScyllaDB, Cassandra 등은 RBAC(Role Based Access Control)로 계층적 역할/권한 부여, 역할 간 상속구조(조직/직책별)
- **클라우드 관리 콘솔**
 - 리소스/서비스별 사용자 구분, REST API/SDK 통한 권한 배포 및 회수

대표 명령어/관점 비교

DBMS	계정(사용자) 생성/삭제	권한 부여/회수	역할(Role) 관리	세분화 권한 단위	사용자/권한 관리 구조
MySQL	CREATE/DROP USER	GRANT/REVOKE	CREATE ROLE	글로벌/DB/테이블/컬럼/객체	SQL+Static /Dynamic Privileges
PostgreSQL	CREATE ROLE	GRANT/REVOKE	CREATE ROLE	DB/스키마/테이블/시퀀스/함수/그룹(롤)	SQL(Role 기반, RBAC)
Oracle	CREATE USER	GRANT/REVOKE	CREATE ROLE	시스템/객체/롤 단위	SQL+RBAC+Cloud 연동
SQL Server	CREATE LOGIN/USER	GRANT/REVOKE/DENY	CREATE ROLE	서버/DB/스키마/객체/시스템 단위	SQL+Role+Windows /Azure RBAC
Cloud Spanner	IAM 계정	IAM Role 지정	DB Role+IAM	프로젝트/인스턴스/DB/테이블/열	IAM+FGAC (Role 기반 및 세분화)
Aurora /RDS	IAM 계정/그룹	SQL+IAM 권한	DB Role+IAM	DB 인스턴스/객체/DB /PaaS-Security Group	IAM+SQL 혼합 관리 구조

정리/활용 방안

- 전통 **RDBMS**는 **SQL** 기반 명령으로, 글로벌~객체 단위 권한 구분 및 역할 기반 일원화된 관리가 가능
- **CloudDB/DistributedDB**는 **IAM(Identity & Access Management)**와 **DB** 내부 역할 제어를 통합해 조직 규칙에 따라 세분화, 관리 편의성과 보안성 향상
- 세분화/자동화
 - **RBAC** 계층, 정책 기반/자동 배포, 관리 콘솔, **API/CLI**의 혼합으로 대규모 기업 작업 환경에 적합한 구조 구성 가능

최소 권한 설계 패턴

- 시스템 계정/스키마 소유자 분리
- 역할 단위로 권한 부여
- 애플리케이션 계정은 역할만 사용
- **DDL 권한 분리(운영 금지)**
- 감사·모니터링

[예시] 역할 정의와 권한 부여_PostgreSQL

```
-- 역할 정의
CREATE ROLE app_reader NOLOGIN;
GRANT CONNECT ON DATABASE appdb TO app_reader;
GRANT USAGE ON SCHEMA public TO app_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_reader;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT ON TABLES TO app_reader;

-- 애플리케이션 계정
CREATE ROLE app_user LOGIN PASSWORD '***';
GRANT app_reader TO app_user;
```


[예시] 역할 정의와 권한 부여_MySQL 8.0

```
CREATE ROLE 'app_reader';  
GRANT SELECT ON appdb.* TO 'app_reader';  
CREATE USER 'app_user'@'%' IDENTIFIED BY '***';  
GRANT 'app_reader' TO 'app_user'@'%';  
SET DEFAULT ROLE 'app_reader' TO 'app_user'@'%';
```

[예시] 역할 정의와 권한 부여_MSSQL

```
CREATE ROLE app_reader;  
GRANT SELECT ON SCHEMA::dbo TO app_reader;  
CREATE LOGIN app_user WITH PASSWORD='***';  
CREATE USER app_user FOR LOGIN app_user;  
EXEC sp_addrolemember 'app_reader', 'app_user';
```

[예시] 역할 정의와 권한 부여_Oracle

```
CREATE ROLE APP_READER;  
GRANT CREATE SESSION TO APP_READER;  
GRANT SELECT ANY TABLE TO APP_READER; -- (권장 X, 스키마 단위 세분화 권장)  
-- 더 안전하게:  
GRANT SELECT ON HR.EMPLOYEES TO APP_READER;  
  
CREATE USER APP_USER IDENTIFIED BY "***";  
GRANT APP_READER TO APP_USER;
```

[예시] Row-Level Security(행 보안)_PostgreSQL

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;  
CREATE POLICY by_customer ON orders  
  FOR SELECT USING (customer_id = current_setting('app.customer_id')::int);  
-- 앱 커넥션 직후:  
SELECT set_config('app.customer_id','42', true);
```

[예시] Row-Level Security(행 보안)_MSSQL

```
CREATE SCHEMA Security;  
CREATE FUNCTION Security.fnOrderPredicate(@cust_id int)  
RETURNS TABLE WITH SCHEMABINDING AS  
    RETURN SELECT 1 AS fn WHERE @cust_id =  
    CAST(SESSION_CONTEXT(N'customer_id') AS int);  
  
CREATE SECURITY POLICY OrderFilter  
ADD FILTER PREDICATE Security.fnOrderPredicate(customer_id) ON dbo.Orders  
WITH (STATE = ON);  
  
-- 앱 접속 시:  
EXEC sp_set_session_context @key=N'customer_id', @value=42;
```

SQL Injection 방어: Prepared Statement, ORM

취약 원인: 사용자 입력을 문자열 이어붙이기로 바로 SQL에 삽입

1차 방어: 파라미터 바인딩(Prepared Statement) — DB가 SQL과 값의 경계를 분리

2차 보강: 화이트리스트 검증(특히 ORDER BY, LIMIT 컬럼/정렬방향),
ORM의 Query Builder 안전 API

주의: 저장 프로시저/뷰만으로 자동 방어 아님.
내부에서 동적 SQL(concat) 만들면 동일 취약.

취약 vs 안전_Python + psycopg

```
sql = f"SELECT * FROM users WHERE name = '{user_input}'"  
cur.execute(sql) # ' OR '1'='1 공격 가능
```

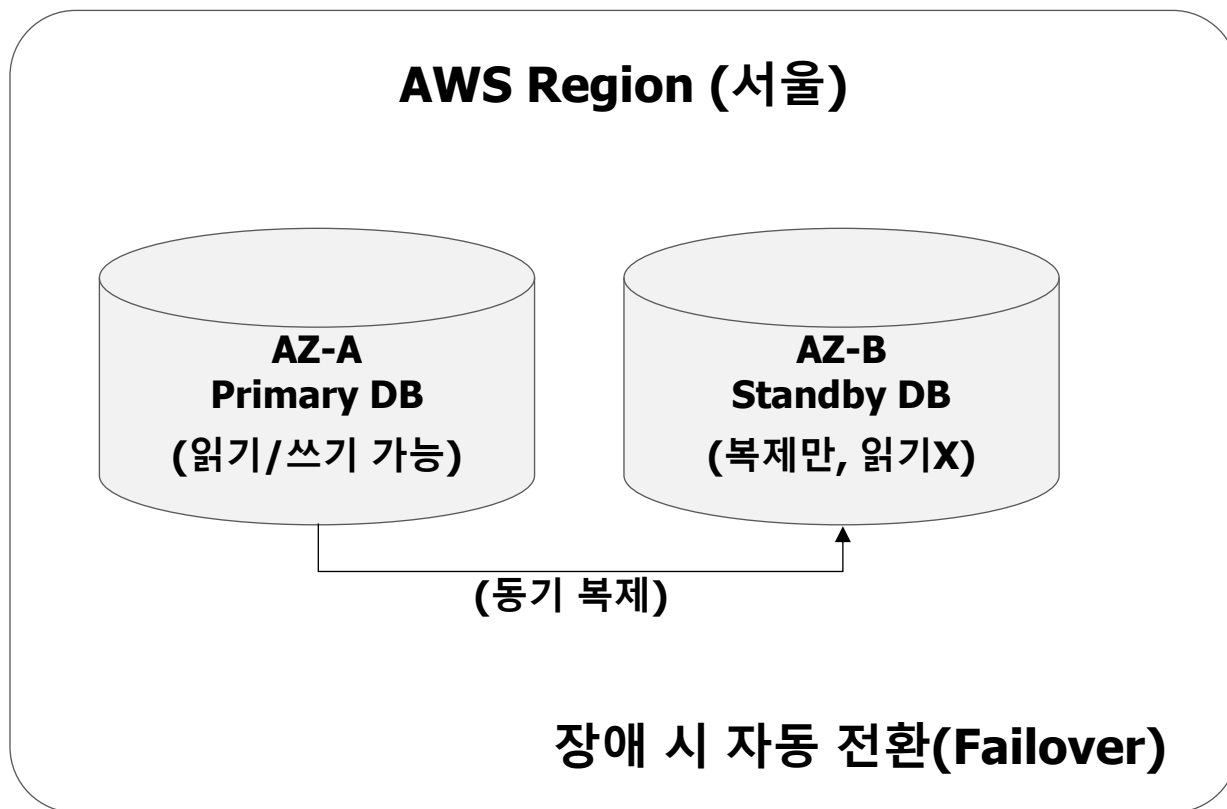
AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

31. 백업·복구 & 고가용성

- Full/Incremental/PITR · Replication · Failover · Cloud 백업

백업 유형 비교

【 Multi-AZ (Availability Zone) 구조 】



- 두 개의 **AZ**에 **DB**를 복제해서 동시에 운영
- **Primary DB**
 - 실제로 애플리케이션이 접속해서 읽고 쓰는 주 **DB**
- **Standby DB**
 - 동기 복제로 항상 같은 데이터 유지
- 동기 복제
 - 쓰기가 완료되려면, 두 **DB** 모두 쓰기가 끝나야 “성공”
→ 그래서 **RPO**(데이터 손실) ≈ 0
- **Failover**
 - **Primary**가 죽으면 **AWS**가 자동으로 **Standby**를 승격함
→ 그래서 **RTO**(복구 시간)는 수 분 내

백업 유형 비교

백업 유형	설명	특징	DBMS 구현
Full Backup	전체 데이터 스냅샷	- 가장 느림 - 가장 완전	pg_dump mysqldump RMAN
Incremental	마지막 백업 이후 변경분만	- 빠름 - 복구 시 Full + Incremental 조합	WAL 아카이브 Binlog RMAN
Differential	마지막 Full 이후 전체 변경분	Incremental보다 복구 간단	SQL Server 지원
PITR (시점 복구)	특정 시점으로 복구	오염된 트랜잭션 직전으로 복구	WAL + 기록 시점 지정
논리적 백업	SQL 덤프 (INSERT 문)	- 이식성 높음 - 느림	pg_dump mysqldump
물리적 백업	데이터 파일 직접 복사	- 빠름 - 동일 환경 필요	pg_basebackup xtrabackup

PITR & Streaming Replication — PostgreSQL

PITR (Point-In-Time Recovery)

- 오염 이전 시점으로 복구
- Base backup + WAL 아카이브 보관
→ `recovery_target_time` 설정
- 실수로 **DELETE**한 직전 시점으로 복구 가능
- `postgresql.conf`
 - `archive_mode=on,`
`archive_command`
`= 'cp %p /backup/wal/%f'`

Streaming Replication

- **Primary→Standby** 실시간 복제
- **WAL 스트리밍 방식**
→ **Standby가 Primary와**
거의 동일한 데이터 유지
- **Hot Standby**
 - **SELECT** 쿼리 허용 (읽기 분산)
- **Synchronous**
 - **COMMIT** 시 **Standby** 응답 대기
(**RPO=0**, 성능 저하)
- **Asynchronous**
 - **COMMIT** 즉시 응답
(**RPO>0**, 기본값, 성능 우수)

PITR & Streaming Replication — PostgreSQL

Failover 전략

- **Patroni**
 - Python 기반 HA 클러스터 매니저
 - Primary 장애 시 자동 Standby 승격
- **pg_auto_failover / Repmgr**
 - 대안 HA 도구
- **Cloud**
 - RDS Multi-AZ / Aurora
 - 장애 조치 자동 (CNAME 전환, 수십 초)

DR (재해 복구) 목표

- **RPO (Recovery Point Objective)**
 - 최대 허용 데이터 손실 시간
- **RTO (Recovery Time Objective)**
 - 최대 허용 서비스 다운 시간

DBMS별 백업·복구·HA 비교

DBMS	백업 도구	HA 솔루션	Cloud 옵션
PostgreSQL	- pg_dump - pg_basebackup - WAL	- pg_dump - pg_basebackup - WAL	- RDS Multi-AZ - Aurora - Cloud SQL HA
MySQL	- mysqldump - xtrabackup - mysqlpump	- Group Replication - ProxySQL - MHA	- RDS Multi-AZ - Aurora - Cloud SQL
Oracle	RMAN (Recovery Manager)	- RAC - Data Guard - GoldenGate	- Autonomous DB - Exadata Cloud
SQL Server	- SSMS Backup - Maintenance Plan	- Always On AG - Failover Cluster	- Azure SQL MI - RDS for SQL Server
Cloud (공통)	- 스냅샷 - PITR UI (슬라이더)	- Multi-AZ - Region 자동 장애조치	크로스 리전 Read Replica / Global DB

고가용성 솔루션 심화

Oracle RAC (Real Application Clusters)

- 여러 인스턴스가
공유 스토리지(ASM)를 동시 접근
- **Cache Fusion**
 - 노드 간 데이터 블록 캐시
일관성 유지
- 대형 엔터프라이즈, 무정지 요구,
읽기/쓰기 모두 확장 필요 시
- **AWS RDS for Oracle**
 - RAC 미지원
(Exadata on AWS는 가능)

SQL Server Always On AG

- DB 집합 단위 HA/DR
+ 읽기 스케일아웃
- 동기식(HA)
 - RPO=0, 성능 약간 저하
- 비동기식(DR)
 - 지연 허용
- 보조 복제본에서 읽기 전용 리포팅
쿼리 가능
- **RDS Multi-AZ**는 내부적으로
AG 활용

PostgreSQL FDW (Foreign Data Wrapper)

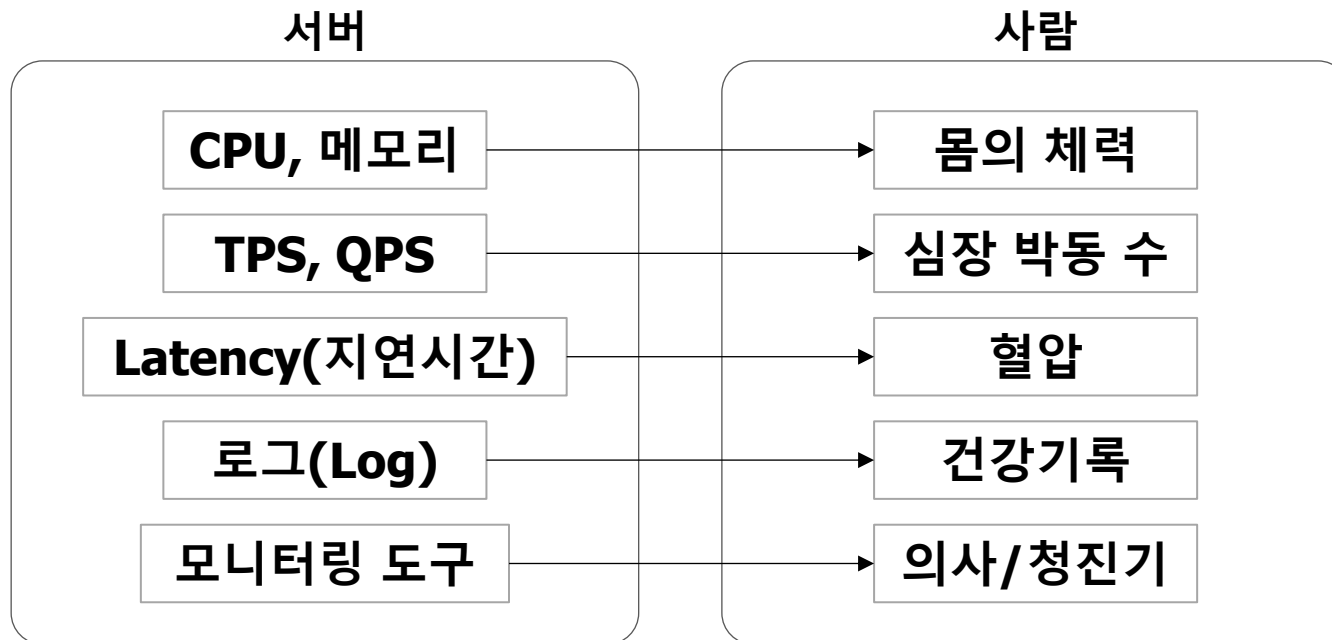
- **postgres_fdw**
 - 원격 PostgreSQL 서버의
테이블을 로컬처럼 참조
- 이기종 DB 조인, 데이터 가상화,
점진적 마이그레이션에 활용
- **Logical Replication**
 - 테이블 단위 Pub/Sub 복제
- 용도
 - 테이블 단위 증분 복제
 - 무중단 마이그레이션
 - 읽기 분

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

32. 모니터링 및 운영

- TPS·QPS·Latency · Slow Query · Prometheus+Grafana · Cloud 모니터링

“모니터링”이란?



**DB 모니터링은 DB의 건강을 체크하고,
이상 징후를 미리 발견하는 일**

기본 지표 (TPS, QPS, Latency)

용어	뜻	표현	예시
TPS (Transactions Per Second)	초당 트랜잭션 수	"1초에 몇 건의 거래가 끝났는가"	쇼핑몰 결제 100건/초
QPS (Queries Per Second)	초당 쿼리 수	"1초에 몇 번 DB에 물어봤는가"	SELECT/INSERT/UPDATE 등 포함
Latency (지연시간)	요청~응답 걸린 시간	"응답 속도"	쿼리 실행에 0.05초 걸림

- ✓ **QPS**는 얼마나 많은 일을 처리하는지
- ✓ **TPS**는 얼마나 많은 거래가 완료되는지
- ✓ **Latency**는 얼마나 빠른지

카페에서
 QPS → 초당 주문 수
 TPS → 초당 결제 완료 수
 Latency → 한 잔 나오기까지 시간

도구	역할	설명
Prometheus	데이터 수집	DB나 서버에서 지표를 주기적으로 모아 저장
Grafana	시각화	Prometheus 데이터를 예쁜 그래프로 보여줌
Datadog	통합 모니터링 SaaS	DB, 서버, 네트워크를 한 번에 관리 (유료)
CloudWatch	AWS 전용 모니터링	RDS나 EC2의 상태를 자동으로 보여줌

Prometheus + Grafana 예시

1. Prometheus가 DB에서 “현재 CPU 몇 %, TPS 몇?” 같은 데이터를 수집
2. Grafana가 그 데이터를 그래프로 Dashboard 화
 - 초당 쿼리(QPS) 그래프
 - p95 지연시간 그래프
 - 복제 지연(replica lag) 그래프
 - Active connection 수



DB용 심전도 모니터

Cloud DB 예시

Cloud	모니터링 서비스	특징
AWS RDS / Aurora	CloudWatch + Performance Insights	그래프+상위 느린 쿼리 자동 분석
GCP Cloud SQL	Cloud Monitoring + Query Insights	쿼리별 평균 응답시간 바로 확인 가능
Azure DB	Azure Monitor + Workbooks	시각화 대시보드 템플릿 제공

**Cloud에서는 직접 설정 안 해도 자동 수집되지만,
세세한 로그를 보려면 "Logs Export"를 켜야 함**

이상 징후 확인

증상	원인 가능성	RDBMS	Cloud DB
쿼리가 갑자기 느려짐	인덱스 없음, 통계 오래됨	EXPLAIN, ANALYZE로 실행 계획 확인	동일하게 SQL 실행계획 확인 가능 (Cloud 콘솔이나 내부 쿼리로도 가능)
CPU 100%	느린 쿼리, 풀스캔	OS 모니터링 도구(top, htop) + Slow Query Log로 원인 확인	CloudWatch / Cloud Monitoring / Azure Monitor에서 CPU 그래프 확인 후 느린 쿼리 로그 확인
Connection Full	커넥션 누수, 풀 미사용	SHOW PROCESSLIST, Threads_connected로 확인	RDS/Cloud SQL 등에서도 max_connections 초과 알람 발생, 콘솔에서 연결 수 그래프 제공
복제 지연	네트워크 문제, 대용량 DML	Replica 서버에서 SHOW SLAVE STATUS 또는 pg_last_xact_replay_timestamp()로 확인	Cloud에서는 ReplicaLag 지표 자동 제공 (RDS: CloudWatch, GCP: Cloud Monitoring)
디스크 꽉 참	로그 미삭제, 임시파일	df -h나 스토리지 모니터링으로 직접 확인	클라우드 콘솔에서 FreeStorageSpace 또는 Disk Usage 지표로 확인 가능

이상 징후 확인 – Cloud DB와 차이

구분	RDBMS	Cloud DB(RDS, Cloud SQL, Azure DB 등)
로그 접근	직접 파일(/var/log/...) 접근 가능	콘솔에서 로그 Export (CloudWatch Logs, Cloud Logging 등)
리소스 지표	OS 명령어로 직접 확인(top, vmstat 등)	클라우드 모니터링 서비스에서 자동 수집
복제 모니터링	수동 스크립트 필요	ReplicaLag 지표 기본 제공
알람 설정	직접 Prometheus/Grafana로 구성	콘솔에서 클릭 몇 번으로 CloudWatch/Alerting 설정
수정 권한	거의 모든 설정 직접 변경 가능	제한적 (파라미터 그룹, 모니터링 기능만 변경 가능)

알람(Alert) 설정 예시

상황	경고 기준	행동
평균 응답 시간 > 500ms	5분 이상 지속	쿼리 분석 시작
Replica Lag > 30초	10분 지속	복제 트래픽 점검
CPU 사용률 > 80%	10분 지속	인덱스, 캐시 점검
Free Storage < 10%	즉시	로그 정리, 용량 증설

On-Prem DB vs 클라우드 DB 차이

항목	On-Prem (직접 설치)	Cloud (DBaaS)
설치/운영	직접 설정 필요	자동 설치/패치
로그 접근	직접 파일 확인	콘솔/로그 서비스에서 조회
모니터링 도구	Prometheus/Grafana 직접 설치	CloudWatch, Query Insights 자동 제공
튜닝 자유도	높음(파라미터 직접 수정)	제한적(파라미터 그룹만 변경 가능)
알람 설정	직접 설정	콘솔에서 클릭 몇 번이면 설정

내가 다 하는 것 vs. 대부분 자동, 하지만 세밀 제어는 제한

개념	의미	도구/방법
TPS / QPS / Latency	DB의 “속도와 양”	모니터링 그래프, SHOW STATUS, pg_stat_database
Slow Query Log	느린 쿼리 기록	MySQL: /var/log/mysql/slow.log
pg_stat_activity	pg_stat_activity	PostgreSQL 내부 뷰
Prometheus + Grafana	시각화	On-Prem 용
CloudWatch / Query Insights	자동 모니터링	클라우드용

- ✓ **TPS/QPS = DB가 얼마나 바쁘게 일하는가**
- ✓ **Latency = 얼마나 빨리 일하는가**
- ✓ **로그 = 왜 느렸는가의 증거**
- ✓ **모니터링 도구 = 그걸 한눈에 보여주는 창**

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

33. 종합실습 - 4

- ecommerce 매출 분석 및 정리

종합 실습 - 시나리오



- 당신은 **E-Commerce** 데이터팀 주니어 엔지니어
- 월간 매출 리포트/**AOV**, 카테고리별 성과, 재구매·**RFM** 분석, 재고 이슈 탐지를 만들고, 실행 계획(성능)까지 개선하는 것
 - 채널: **web, mobile, marketplace**
 - 주문 상태: **created / paid / shipped / delivered / cancelled / refunded**
 - 가격은 가격이력(**SCD2**)로 관리
(**product_prices, valid_from~valid_to**)
 - 카테고리: 트리 구조(재귀 **CTE**)
 - 재고: 재주문 시점 관리(**reorder_point**)
 - 리뷰: 평점 **1~5**, 효자상품 판별
 - 쿠폰: **SAVE10** 등

종합 실습 - 환경설정



- **Schema 생성**
 - 종합실습4_ecom_schema_postgres.sql
- **데이터 적재**
 - 종합실습4_ecom_seed_postgres.sql

종합 실습 - 실습 문제

Q1) 지난 한 달간 실제 팔린 총 금액 보기 (paid+shipped+delivered)

Q2) 월별로 몇 개 주문이 있었고, 얼마 벌었는지, 주문당 평균 금액은 얼마인지

Q3) 최근 90일 카테고리 Top10

Q4) 제품별 누적매출 RANK() Top20

Q5) 고객이 얼마나 최근에, 얼마나 자주, 얼마나 많이 샀는지 (최근성/빈도/금액)

종합 실습 - 실습 문제

Q6) 첫 구매 후 30일 내 재구매율

Q7) 재고가 임계치보다 낮은 상품 찾아내기 (곧 품절될 위험이 있는 상품은?)

Q8) 리뷰 4.5↑ & 50개↑ 효자상품 (리뷰가 많고 평가도 좋은 효자상품 찾기)

Q9) 쿠폰 사용 영향(쿠폰을 쓴 주문과 안 쓴 주문의 평균 주문 금액 비교)

Q10) 상위 1% 고객의 최근 60일 매출

Q11) 0으로 나누어도 에러 안 나는 나눗셈 함수 써보기
→ 안전하게 평균 계산하기 (0으로 나누기 방지)

종합 실습 - 실행 계획 비교 & 성능 개선



- **EXPLAIN ANALYZE**로 병목 파악
- 인덱스 추가/개선
- **Join 전략 비교**
: Hash Join vs Nested Loop, Bitmap Heap Scan 등
- **Materialized View 활용**
: mv_daily_gmv로 리포트 질의 가속, 갱신 전략 검토
- **(Option) 엔진별 특징 정리**
: Postgres / MySQL / Oracle / SQL Server 옵티마이저 차이

종합 실습 - 실행 계획 비교 & 성능 개선



- **Materialized View 활용**

: mv_daily_gmv로 리포트 질의 가속, 갱신 전략 검토

매일매일 총 판매금액을 조회할 수 있게
단, 매번 JOIN 해서 SUM 하면
속도가 느리므로 **Materialized View** 사용

mv_daily_gmv라는 **Materialized View**를 만들어서
리포트 쿼리를 빠르게 실행하고,
데이터가 얼마나 자주 바뀌는지에 맞춰
갱신 주기를 설계 (**오후 3시 기준으로 설정**)

종합 실습 - 제출물

실습 제출물
(당일 제출 원칙)

SQL Query 및 결과

- 각 문항별 쿼리와 실행결과 화면
(쿼리와 실행결과가 나온 화면 Screenshot)

제출 방법 및 배점

- Slack 내 댓글 제출 (제출기한 금일 중)
- 채점 기준
 - 요구사항 반영 여부 + Query 내 컬럼 선택 (불필요한 컬럼 선택 확인)
 - DB 프로그래밍에 대한 주석처리 여부



수고하셨습니다.

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.